

Reti (Computer Networks)

Capitolo 3 Livello di trasporto: UDP e TCP

Docente: Paolo Casari

TA: Andrea Rosani

Capitolo 3: Livello di trasporto

Obiettivi:

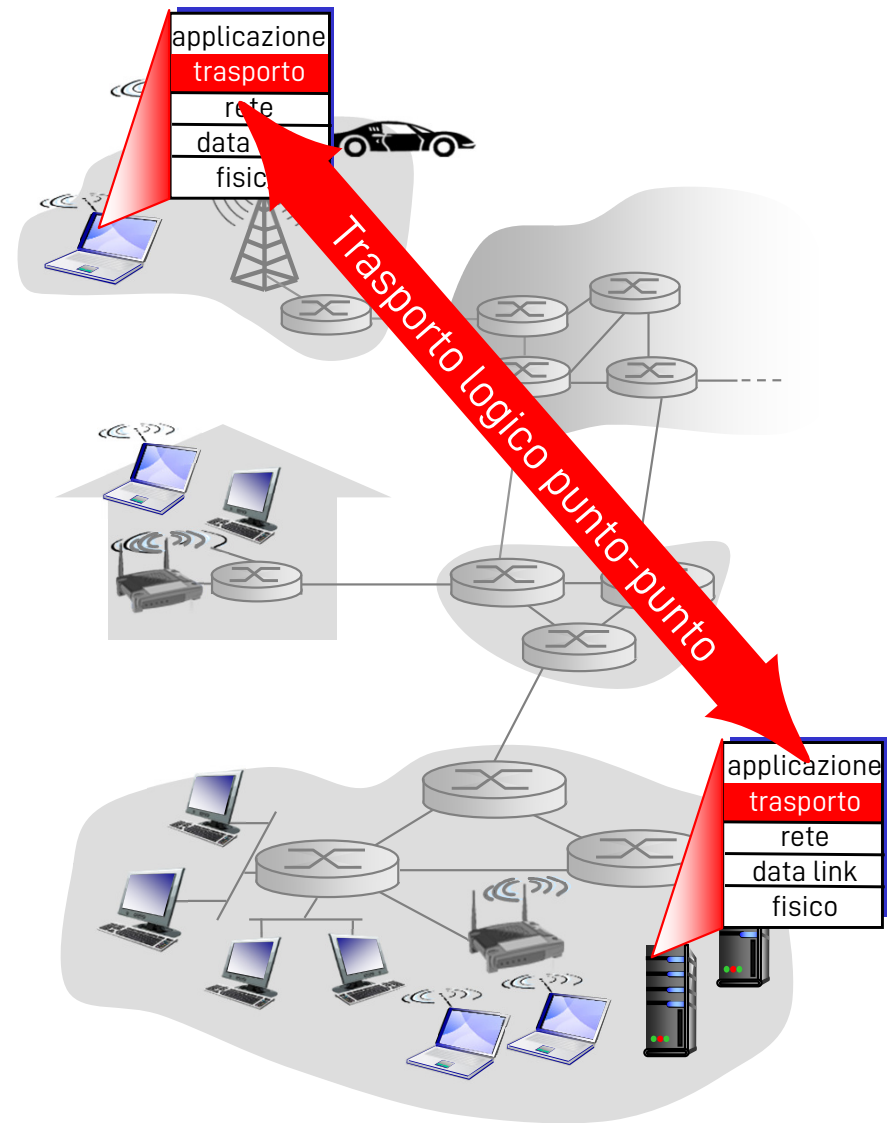
- ❑ Capire i principi che sono alla base dei servizi del livello di trasporto:
 - Multiplexing/demultiplexing
 - Trasferimento dati affidabile
 - Controllo di flusso
 - Controllo di congestione
- ❑ Descrivere i protocolli del livello di trasporto di Internet:
 - UDP: trasporto senza connessione
 - TCP: trasporto orientato alla connessione
 - Controllo di congestione TCP

Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - Struttura dei segmenti
 - Trasferimento dati affidabile
 - Controllo di flusso
 - Gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

Servizi e protocolli di trasporto

- ❑ Forniscono la comunicazione logica tra processi applicativi di host differenti
- ❑ I protocolli di trasporto vengono eseguiti nei sistemi terminali
 - ❖ Lato invio: scinde i messaggi in segmenti e li passa al livello di rete
 - ❖ Lato ricezione: riassembla i segmenti in messaggi e li passa al livello di applicazione
- ❑ Più protocolli di trasporto sono a disposizione delle applicazioni
 - ❖ Internet: TCP e UDP



Relazione tra livello di trasporto e livello di rete

❑ Livello di rete:

Comunicazione logica tra host

❑ Livello di trasporto:

Comunicazione logica tra processi

- Si basa sui servizi del livello di rete e li potenzia

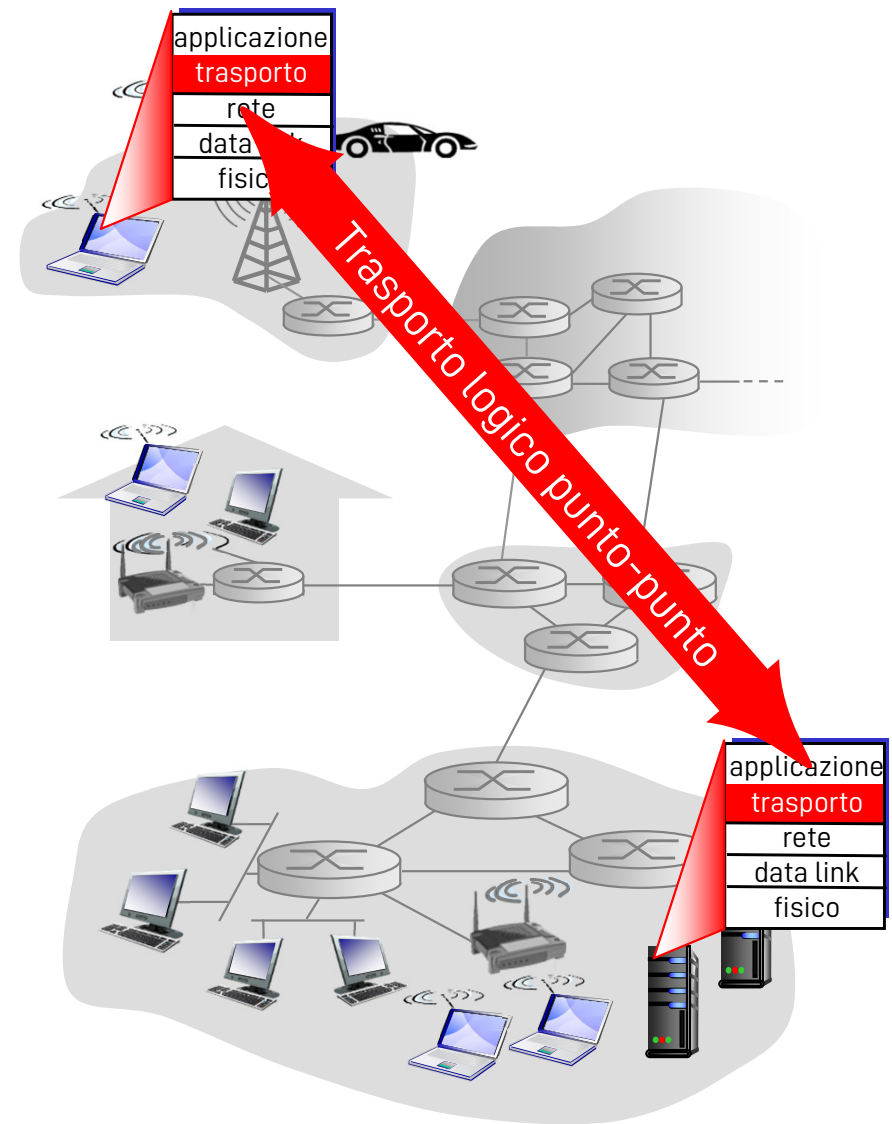
Analogia con la posta ordinaria:

12 studenti a Torino inviano lettere a 12 studenti a Enna

- ❑ Processi = studenti
- ❑ Messaggi delle applicazioni = Lettere nelle buste
- ❑ Host = Abitazioni
- ❑ Protocollo di trasporto = sorella maggiore Anna e fratello maggiore Andrea
- ❑ Protocollo del livello di rete = Servizio postale

Protocolli del livello di trasporto in Internet

- ❑ Affidabile, consegne nell'ordine originario: TCP
 - ❖ Controllo di congestione
 - ❖ Controllo di flusso
 - ❖ Setup della connessione
- ❑ Inaffidabile, consegne senza ordine: UDP
 - ❖ Estensione senza fronzoli del servizio di consegna best effort
- ❑ Servizi non disponibili:
 - ❖ Garanzia su ritardi
 - ❖ Garanzia su ampiezza di banda

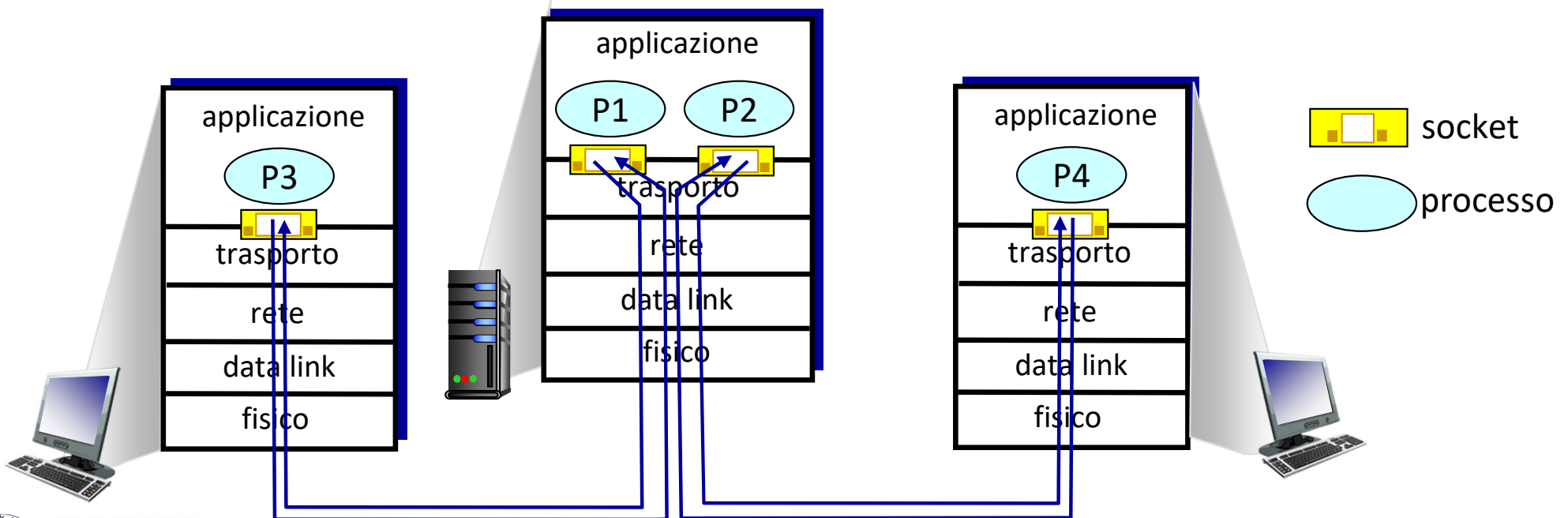


Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - Struttura dei segmenti
 - Trasferimento dati affidabile
 - Controllo di flusso
 - Gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

Multiplexing/demultiplexing

- ❑ Multiplexing al trasmettitore: gestione dati da diverse socket, aggiunta header con PCI del livello transport (ne ha bisogno il ricevitore per il demultiplexing)
- ❑ Demultiplexing al ricevitore: utilizza le PCI nell'header per consegnare i segmenti ricevuti alle socket giuste

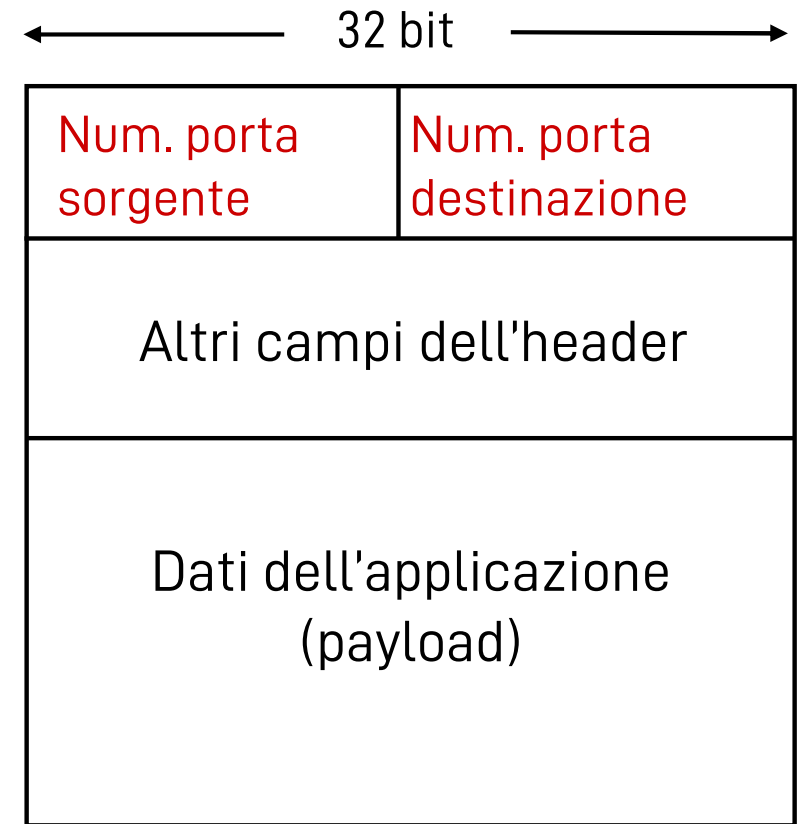


Come funziona il demultiplexing

❑ L'host riceve i datagrammi IP

- ❖ Ogni datagramma ha un indirizzo IP di origine e un indirizzo IP di destinazione
- ❖ Ogni datagramma trasporta 1 segmento a livello di trasporto
- ❖ Ogni segmento ha un numero di porta di origine e un numero di porta di destinazione

❑ L'host usa gli indirizzi IP e i numeri di porta per inviare il segmento alla socket appropriata



Struttura di un segmento
TCP / UDP

Porte TCP e UDP

- ❑ La destinazione finale di un segmento non è un host, ma un **processo** che gira sull'host
- ❑ L'interfaccia tra l'applicazione e il livello di trasporto è la "**porta**"
 - ❖ Identificata da un intero di 16 bit
 - ❖ Mappatura biunivoca tra porta $\leftrightarrow$ processo
 - ❖ I servizi standard vengono esposti da un server utilizzando numeri di porta standard o "well-known", con valori < 1024
 - Esempio: porta 80 per HTTP, porta 25 per SMTP, porta 53 per DNS
 - ❖ I processi non-standard e le connessioni in ingresso a un client usano invece numeri di porta più alti, fino a 65535 ($=2^{16}-1$)

Porte TCP e UDP

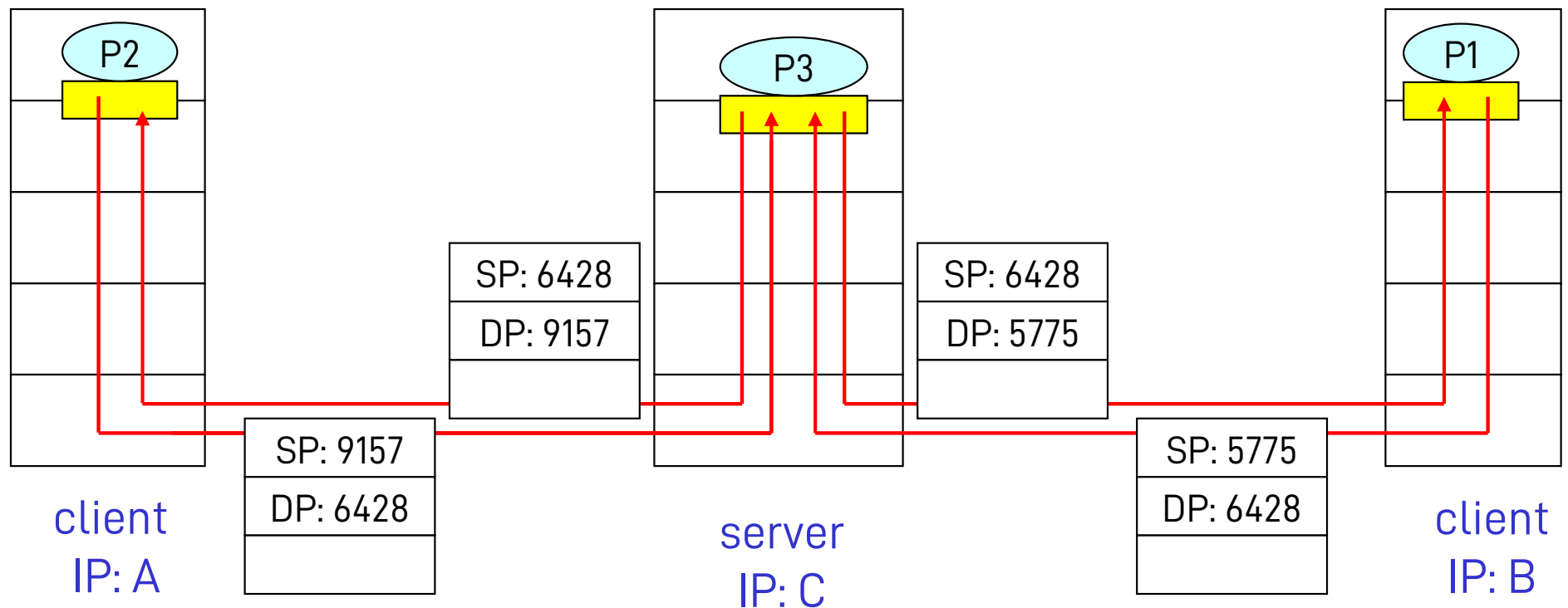
- ❑ I numeri di porta si possono anche classificare come
 - ❖ Statici (altro nome delle porte "well-known")
 - Associati con applicazioni standard (email, web, DNS, ...)
 - ❖ Dinamici (o "ephemeral")
 - Assegnati automaticamente dal sistema operativo quando si apre una connessione o si crea una socket
- ❑ Le porte sorgente e destinazione non sono le stesse (**D**: perché?)
- ❑ Concetto di flusso (flow) e connessione
 - ❖ Gruppo di dati che appartengono alla stessa comunicazione logica
 - ❖ Un'applicazione può aprire multiple connessioni e veicolare molteplici flussi

Demultiplexing senza connessione

- ❑ Crea le socket con i numeri di porta:
 - ❑ `DatagramSocket mySocket1 = new DatagramSocket(12534);`
 - ❑ `DatagramSocket mySocket2 = new DatagramSocket(12535);`
- ❑ La socket UDP è identificata da 2 parametri:
(indirizzo IP di destinazione,
numero della porta di destinazione)
- ❑ Quando l'host riceve il segmento UDP:
 - Controlla il numero della porta di destinazione nel segmento
 - Invia il segmento UDP alla socket con quel numero di porta
- ❑ Datagrammi IP con indirizzi IP di origine e/o numeri di porta di origine differenti vengono inviati alla stessa socket

Demultiplexing senza connessione (continua)

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```



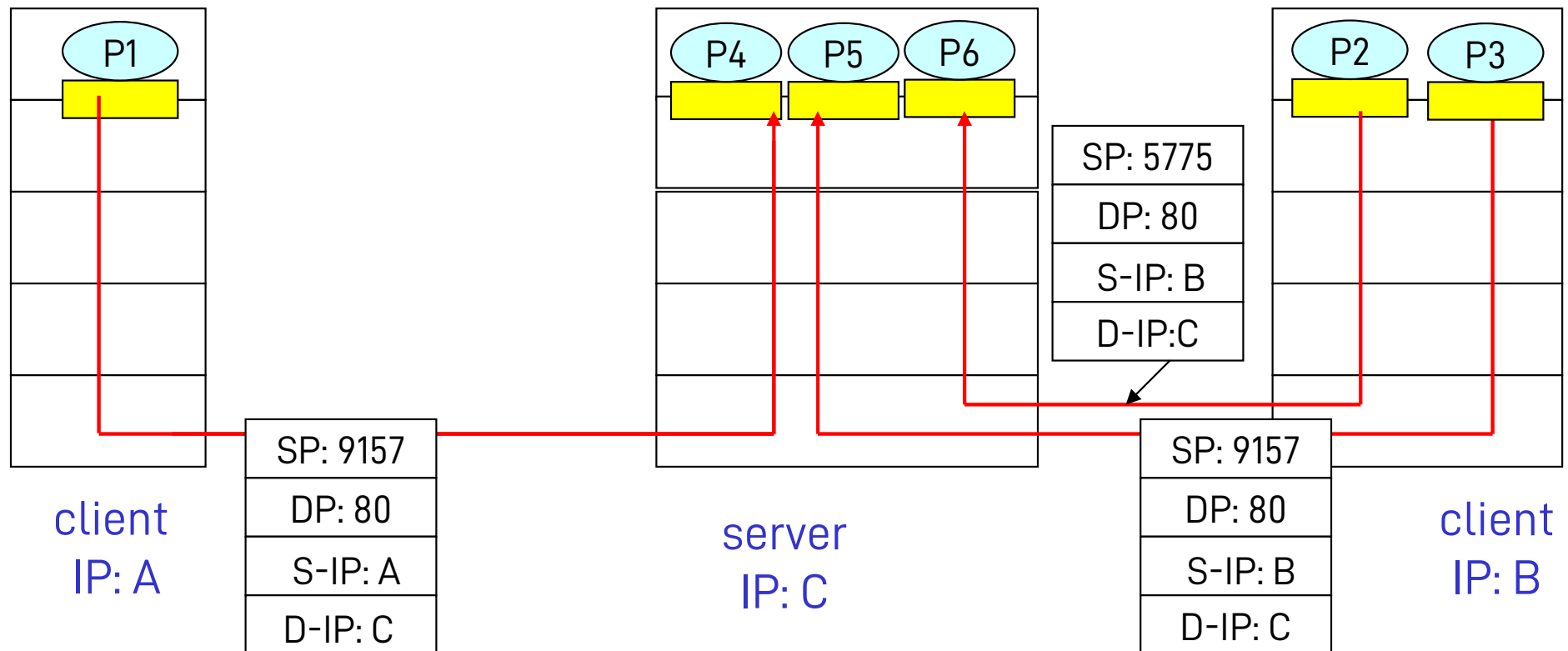
SP = source port
DP = destination port

SP fornisce "l'indirizzo di ritorno a livello 4"

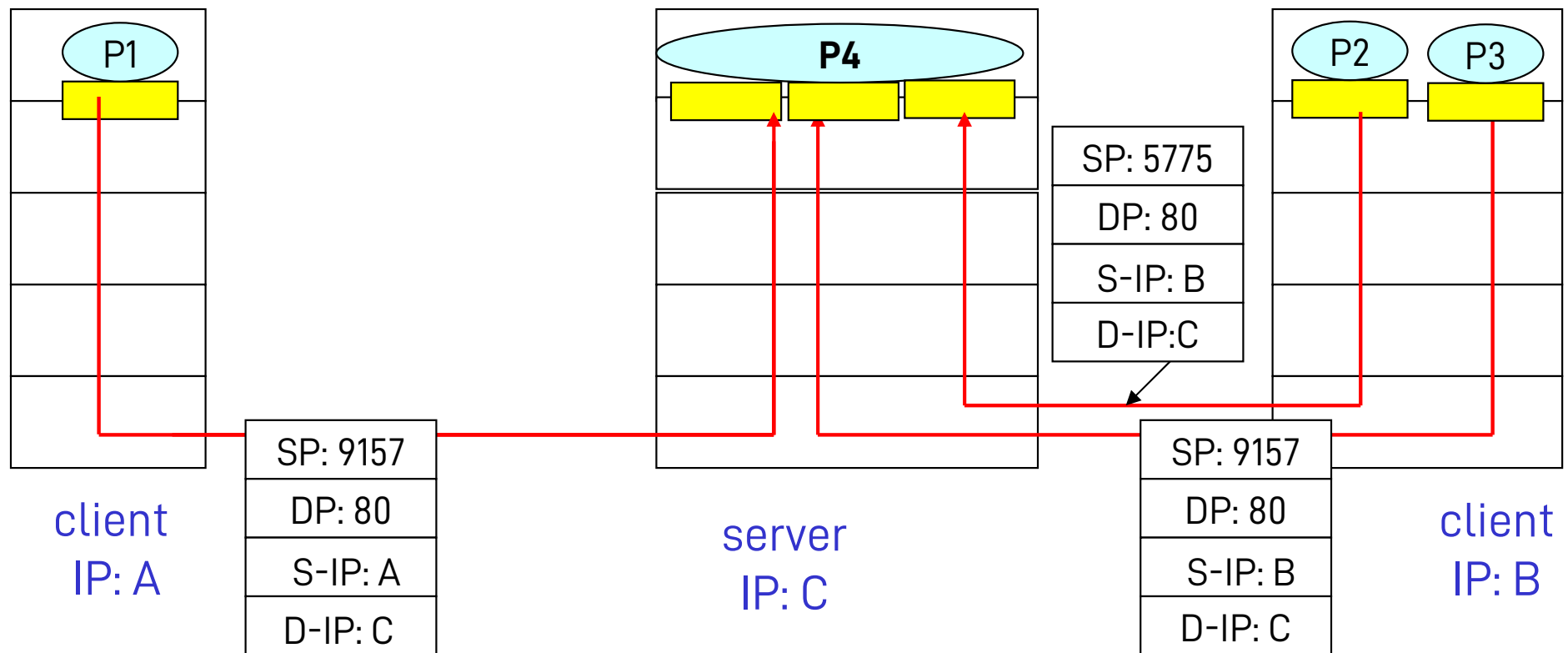
Demultiplexing orientato alla connessione

- ❑ La socket TCP è identificata da 4 parametri:
 - indirizzo IP di origine
 - numero di porta di origine
 - indirizzo IP di destinazione
 - numero di porta di destinazione
- ❑ L'host ricevente usa i quattro parametri per inviare il segmento alla socket appropriata
- ❑ Un host server può supportare più socket TCP contemporanee:
 - Ogni socket è identificata dai suoi 4 parametri
- ❑ I server web hanno socket differenti per ogni connessione client
 - Con HTTP non-persistente si avrà una socket differente per ogni richiesta

Demultiplexing orientato alla connessione (cont.)



Demultiplexing orientato alla connessione (cont.)



Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - struttura dei segmenti
 - trasferimento dati affidabile
 - controllo di flusso
 - gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

UDP: User Datagram Protocol [RFC 768]

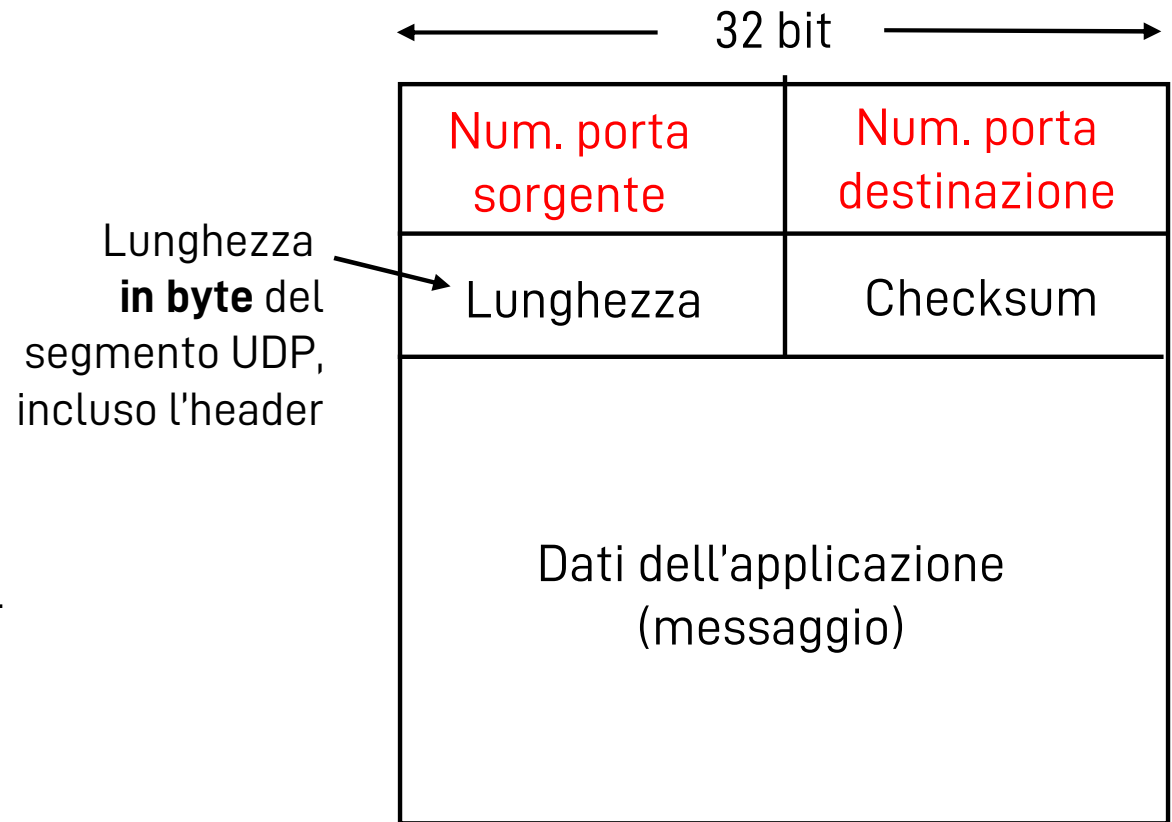
- ❑ Protocollo di trasporto "senza fronzoli"
- ❑ Servizio di consegna "a massimo sforzo" (**best effort**)
- ❑ I segmenti UDP possono essere:
 - Perduti
 - Consegnati fuori sequenza all'applicazione
- ❑ **Senza connessione:**
 - No handshaking tra mittente e destinatario UDP
 - Ogni segmento UDP è gestito indipendentemente dagli altri

Perché esiste UDP?

- ❑ Non richiede di stabilire una connessione (che potrebbe aggiungere un ritardo)
- ❑ Semplice: nessuno stato di connessione nel mittente e destinatario
- ❑ Header di segmento corti
- ❑ Senza controllo di congestione: UDP può inviare raffiche di pacchetti dati

UDP: ulteriori informazioni

- ❑ Utilizzato spesso nelle applicazioni multimediali
 - Tolleranza piccole perdite
 - Sensibile alla frequenza
- ❑ Altri impieghi di UDP
 - DNS
 - SNMP
- ❑ Trasferimento affidabile con UDP: aggiungere affidabilità al livello di applicazione



Struttura del segmento UDP

Checksum UDP

Obiettivo: rilevare gli “errori” (bit alterati)
nel segmento trasmesso

Mittente:

- ❑ Tratta il contenuto del segmento come una sequenza di interi da 16 bit
- ❑ checksum: somma le parole di 16 bit nel segmento e calcola il complemento a 1 della somma
- ❑ Il mittente pone il valore ottenuto nel campo checksum del segmento UDP

Ricevente:

- ❑ Somma tutte le parole di 16 bit del segmento, incluso il checksum
- ❑ Controlla se il risultato è una parola di 16 bit tutti uguali a 1
 - Sì – Nessun errore rilevato.
Ma potrebbero esserci errori nonostante questo?
Lo scopriremo più avanti ...
 - No – errore rilevato

Checksum UDP: esempio

Esempio: somma di due parole di 16 bit ciascuna

1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 Parola 1

1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 Parola 2

Riporto
sommato

1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1

+

Somma

1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0

Checksum

0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

(complemento a 1)

Nota: quando si sommano i numeri, se c'è un riporto dal bit più significativo, questo deve essere sommato al risultato

Checksum UDP: al ricevitore

[illegible]

Sommare la somma delle parole di 16 bit e il complemento a 1 della somma stessa produce una parola di tutti 1 (se non ci sono bit errati nel segmento ricevuto)

Nota su rilevamento e correzione di errori

- ❑ Bit di parità: permette di rilevare un numero dispari di bit errati

0	1	1	0	1	0	1	0	0
0	1	0	0	1	0	1	0	1

- ❑ Codice a ripetizione: permette voti di maggioranza, e correzione di qualche errore

0	1	1	0	1	0	1	0
0	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0
0	1	1	0	1	0	1	0

Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - struttura dei segmenti
 - trasferimento dati affidabile
 - controllo di flusso
 - gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

ARQ

- ❑ Automatic Repeat reQuest
 - ❖ Classe di protocolli che cerca di recuperare le perdite di pacchetti
- ❑ Usa pacchetti speciali per notificare il trasmettitore di una avvenuta ricezione corretta
 - ❖ Acknowledgments (ACK)
- ❑ Esempi di protocolli basati su forme di ARQ
 - ❖ Stop-and-Wait
 - ❖ Go-back-N
 - ❖ Selective Repeat
 - ❖ TCP
 - ❖ Il protocollo MAC (livello 2, data link) dei sistemi WiFi

Stop-and-Wait

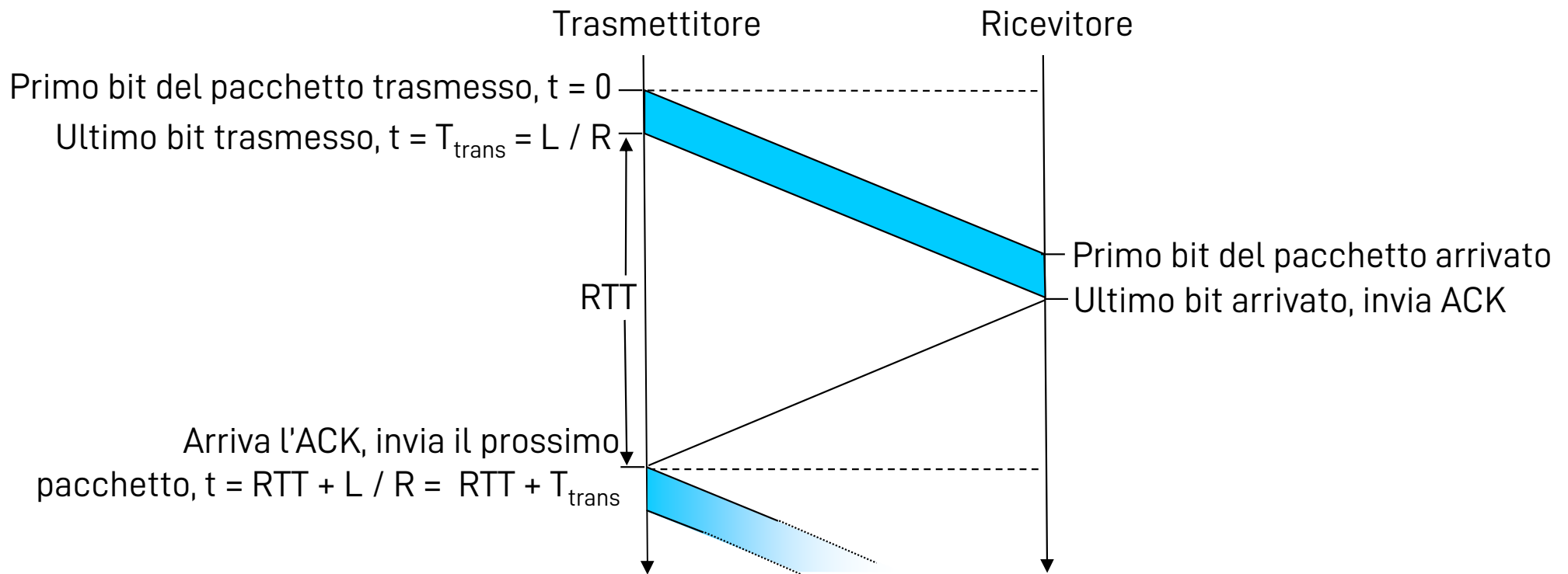
❑ Trasmettitore:

- ❖ Invia una PDU (e mantiene una copia locale)
- ❖ Imposta un timeout
- ❖ Attende la ricezione del rispettivo ACK
 - Se non riceve alcun ACK entro il timeout, invia nuovamente la stessa PDU
 - Se riceve l'ACK
 - Controlla che l'ACK non contenga errori (checksum)
 - Controlla il numero di sequenza
 - Se è tutto OK (quindi l'ACK si riferiva alla PDU inviata) procede con l'invio della PDU successiva

❑ Il ricevitore, quando riceve una PDU:

- ❖ Controlla se ci sono errori (checksum)
- ❖ Controlla il numero di sequenza
 - Se è corretto e in ordine, invia l'ACK e passa l'SDU ai layer superiori
 - Se il checksum o il numero di sequenza sono errati, cancella la PDU (*drop*)

Come funziona Stop-and-Wait



❑ Assunzione: la durata del messaggio ACK è trascurabile

Efficienza di Stop-and-Wait

❑ Assumiamo

- ❖ $R = 1 \text{ Gbit/s}$
- ❖ 15 ms di ritardo di propagazione ($RTT = 30 \text{ ms}$)
- ❖ $L = 8000 \text{ bit}$
- ❖ $T_{\text{trans}} = L/R = 8000 [\text{bit}] / 10^9 [\text{bit/s}] = 8 [\mu\text{s}]$

❑ Throughput percepito a livello applicazione

- ❖ $L / (T_{\text{trans}} + RTT) = 8000 [\text{bit}] / (30.008 [\text{ms}]) = 33 [\text{kByte/s}]$
- ❖ Ehi! Ma non avevo pagato per un link da 1 Gbit/s?? ☹

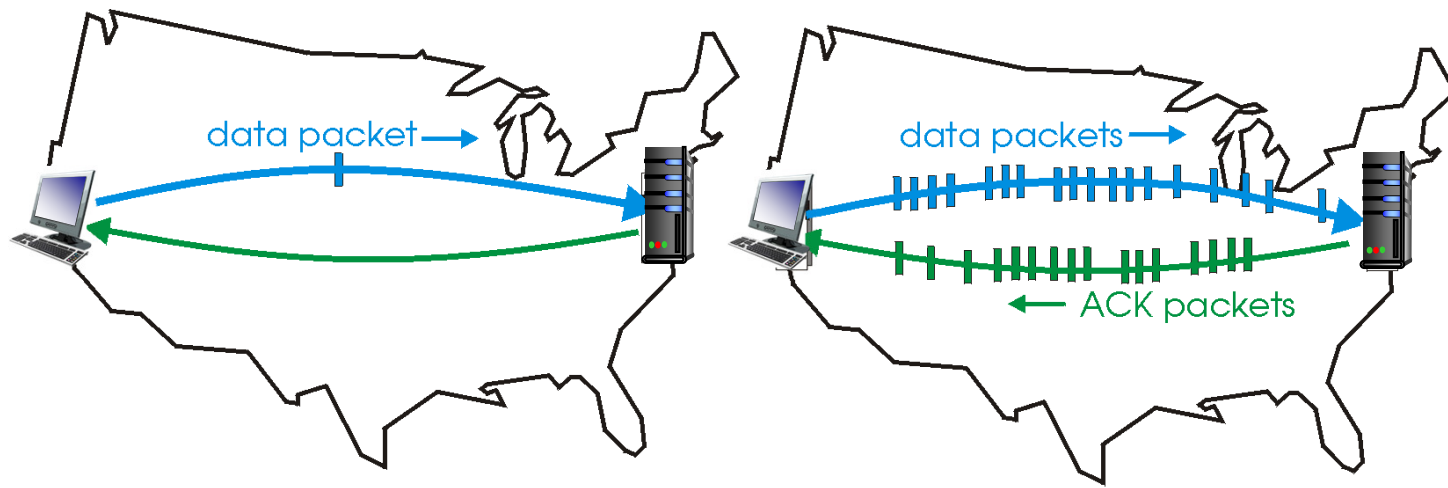
❑ Efficienza

- ❖ $T_{\text{trans}} / (T_{\text{trans}} + RTT) = 0.008 [\text{ms}] / 30.008 [\text{ms}] = 0.00027 (0.027 \%)$

❑ I protocolli di trasporto limitano l'uso delle risorse fisiche

Protocolli con pipelining

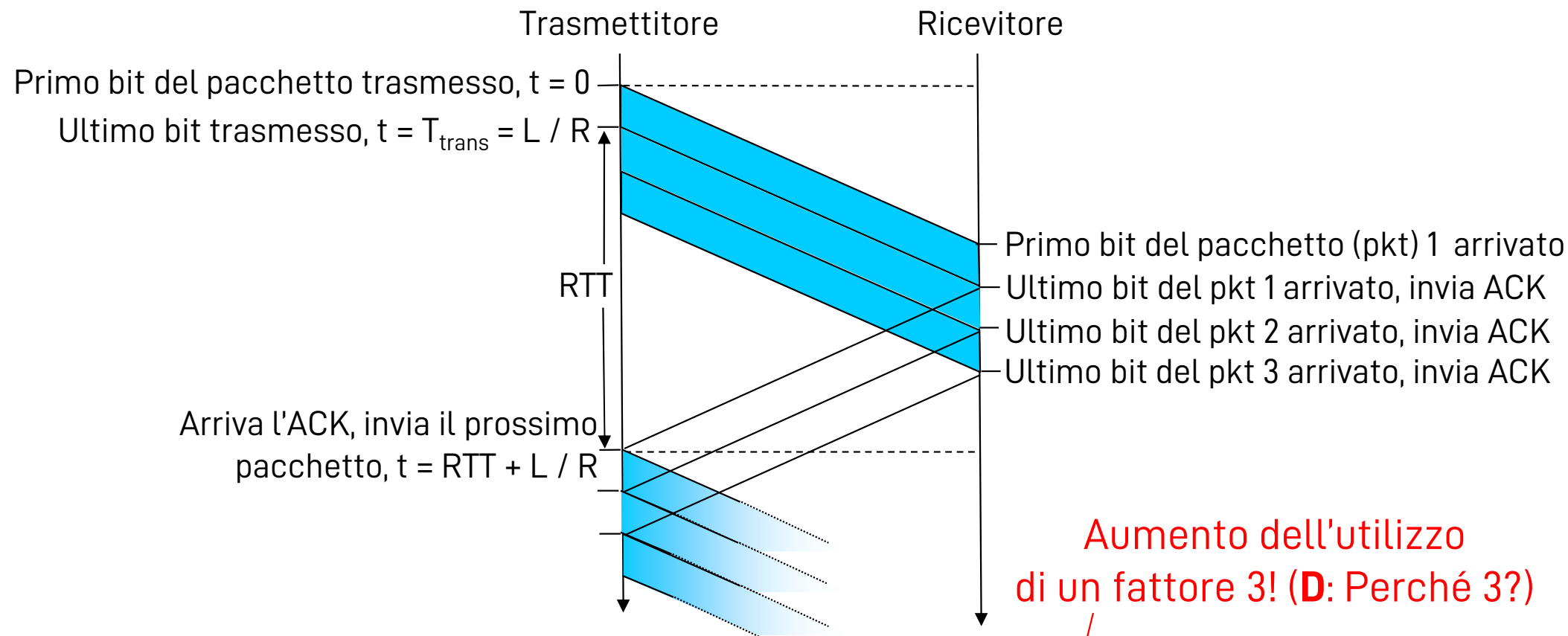
- ❑ Pipelining: il trasmettitore accetta che ci siano diversi pacchetti “in volo”, cioè per i quali non ha ancora ricevuto un ACK
 - ❖ Va tenuta traccia dei numeri di sequenza dei pacchetti “in volo”
 - L'intervallo di numeri di sequenza accettabili si allarga
 - ❖ Bisogna memorizzare i segmenti sia al trasmettitore sia al ricevitore
- ❑ Due forme di protocolli con pipelining: Go-back-N, selective repeat



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

Il pipelining aumenta l'utilizzo di un link

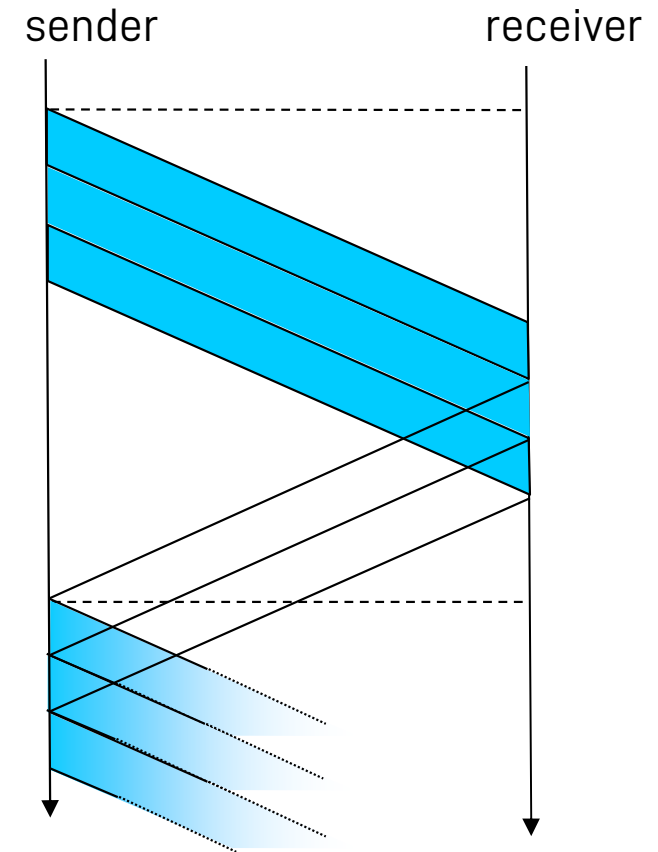


❑ Application layer throughput

$$\diamond 3L / (T_{trans} + RTT) = 24000 \text{ b} / (30.008 \text{ ms}) = 100 \text{ kByte/sec}$$

Throughput in presenza di pipelining

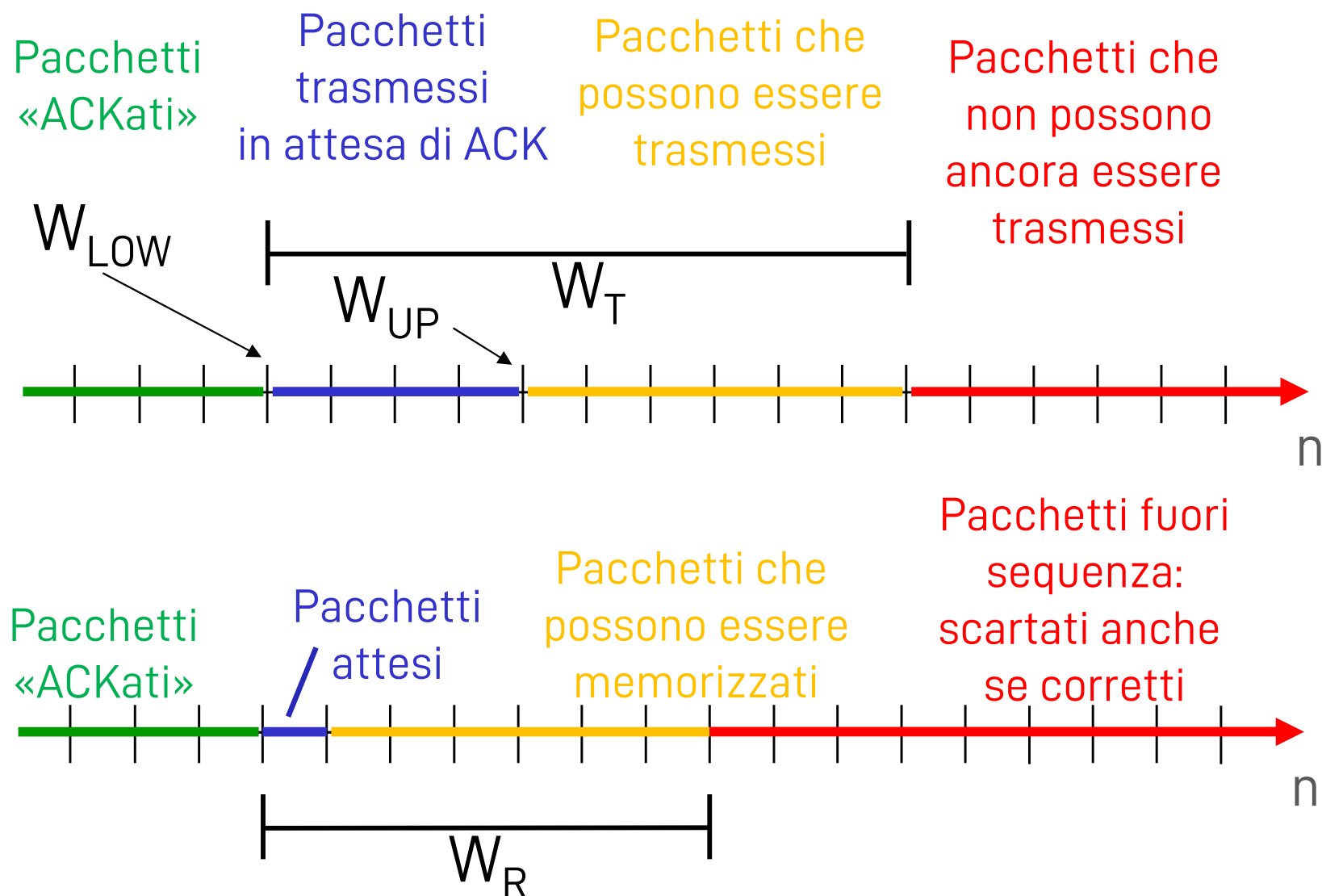
- ❑ 1 pacchetto inviato
(dimensione L) per RTT:
 - ❖ $\text{Throughput} = L / (\text{RTT} + T_{\text{trans}})$
- ❑ 3 pacchetti inviati per RTT:
 - ❖ $\text{Throughput} = 3L / (\text{RTT} + T_{\text{trans}})$
-
- ❑ N pacchetti inviati per RTT:
 - ❖ $\text{Throughput} = N \times L / (\text{RTT} + T_{\text{trans}})$
 - ❖ (Sempre che il tempo necessario per trasmettere N pacchetti sia $< \text{RTT}$)
- ❑ Il valore di N è chiamato “dimensione della finestra”



Protocolli con pipelining: definizioni

- ❑ **Finestra di trasmissione** W_T : insieme di PDU che il trasmettitore può trasmettere senza avere ancora ricevuto l'ACK corrispondente
 - ❖ Al massimo tanto grande quanto la memoria allocata dal sistema operativo del trasmettitore
 - ❖ $|W_T|$ (cardinalità di W_T) indica la dimensione della finestra
- ❑ **Finestra di ricezione** W_R : insieme di PDU che il ricevitore può accettare e immagazzinare
 - ❖ Al massimo tanto grande quanto la memoria allocata dal sistema operativo del ricevitore
- ❑ **Puntatore low** W_{LOW} : puntatore al **primo** pacchetto nella finestra di trasmissione W_T
- ❑ **Puntatore up** W_{UP} : puntatore all'**ultimo** pacchetto **già** trasmesso
 - ❖ Potrebbe non coincidere con l'ultimo pacchetto di W_T

Finestre di trasmissione e ricezione



Acknowledgements

A seconda del protocollo, si possono usare diversi tipi di ACK

☐ ACK individuale

- ❖ Indica la ricezione corretta di un pacchetto specifico
- ❖ ACK(n) significa "Ho ricevuto il pacchetto n"

☐ ACK cumulativo

- ❖ Indica la ricezione corretta di tutti i pacchetti fino a un certo indice
- ❖ ACK(n) significa "Ho ricevuto tutto fino al pacchetto n (**escluso**)"

☐ ACK negativo (NACK)

- ❖ Richiede la ritrasmissione di un pacchetto singolo
- ❖ NACK(n) significa "Inviarmi di nuovo il pacchetto n"

☐ "Piggybacking"

- ❖ Inserimento di un ACK data in un pacchetto dati

Protocolli con pipelining: sommario

Go-back-N:

- ❑ Il mittente può avere fino a N pacchetti senza ACK in pipeline
- ❑ Il ricevente invia solo ACK cumulativi
 - ❖ Non dà l'ACK di un pacchetto se c'è un gap
- ❑ Il mittente ha un timer per il più vecchio pacchetto senza ACK
 - ❖ Se il timer scade, ritrasmette tutti i pacchetti senza ACK

Selective Repeat:

- ❑ Il mittente può avere fino a N pacchetti senza ACK in pipeline
- ❑ Il ricevente dà gli ACK solo ai singoli pacchetti
- ❑ Il mittente mantiene un timer per ciascun pacchetto che non ha ancora ricevuto ACK
 - ❖ Quando il timer scade, ritrasmette solo i pacchetti che non hanno avuto ACK

Nota: qui l'ACKn cumulativo riscontra i pacchetti
 fino a n incluso per comodità di spiegazione
 (in rosso i veri ACK cumulativi)

Go-back-N in action

Finestra di trasmissione ($|W_T|=4$)

Trasmittitore

Ricevitore

Finestra di ricezione ($|W_R|=1$)
 (dopo la ricezione)

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

Invia pkt0
 Invia pkt1
 Invia pkt2
 Invia pkt3
 (attendi)

X perdita

Ricevi pkt0, invia ack0(ack1)
 Ricevi pkt1, invia ack1(ack2)

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

Ricevi pkt3, scartalo,
 (re)invia ack1(ack2)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8 Ricevi ack0, invia pkt4
 0 1 2 3 4 5 6 7 8 Ricevi ack1, invia pkt5

Ricevi pkt4, e scartalo,
 (re)invia ack1(ack2)

0 1 2 3 4 5 6 7 8

Ricevi pkt5, e scartalo,
 (re)invia ack1(ack2)

0 1 2 3 4 5 6 7 8

Ignora l'ACK duplicato



0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

Invia pkt2
 Invia pkt3
 Invia pkt4
 Invia pkt5

Ricevi pkt2, invia ack2(ack3)
 Ricevi pkt3, invia ack3(ack4)
 Ricevi pkt4, invia ack4(ack5)
 Ricevi pkt5, invia ack5(ack6)

0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8
 0 1 2 3 4 5 6 7 8

Nota: qui gli ACKn si riferiscono
a un singolo pacchetto,
non sono cumulativi

Selective repeat in action

Finestra di trasmissione
($|W_T|=4$)

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

Trasmittitore

Invia pkt0
Invia pkt1
Invia pkt2
Invia pkt3
(attendi)

Ricevitore

Ricevi pkt0, invia ack0
Ricevi pkt1, invia ack1

Ricevi pkt3, mantieni
in buffer, invia ack3

Ricevi pkt4, mantieni
in buffer, invia ack4

Ricevi pkt5, mantieni
in buffer, invia ack5

Finestra di ricezione ($|W_R|=4$)
(dopo la ricezione)

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8 Ricevi ack0, invia pkt4
0 1 2 3 4 5 6 7 8 Ricevi ack1, invia pkt5

Registra l'arrivo di ack3



invia pkt2

Registra l'arrivo di ack4

Registra l'arrivo di ack5

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

D1: quando vengono consegnati i pacchetti all'applicazione del ricevitore?
D2: cosa succede quando arriva ack2?

Relazione tra dimensione delle finestre e numeri di sequenza in selective repeat

- ❑ Numero di sequenza di 2 bit

❖ 0, 1, 2, 3

- ❑ $|W_T| = |W_R| = 3$


pkt0 timeout
(re)invia pkt0

Finestra di trasmissione

0123012
0123012
0123012

Finestra di ricezione
(dopo la ricezione)

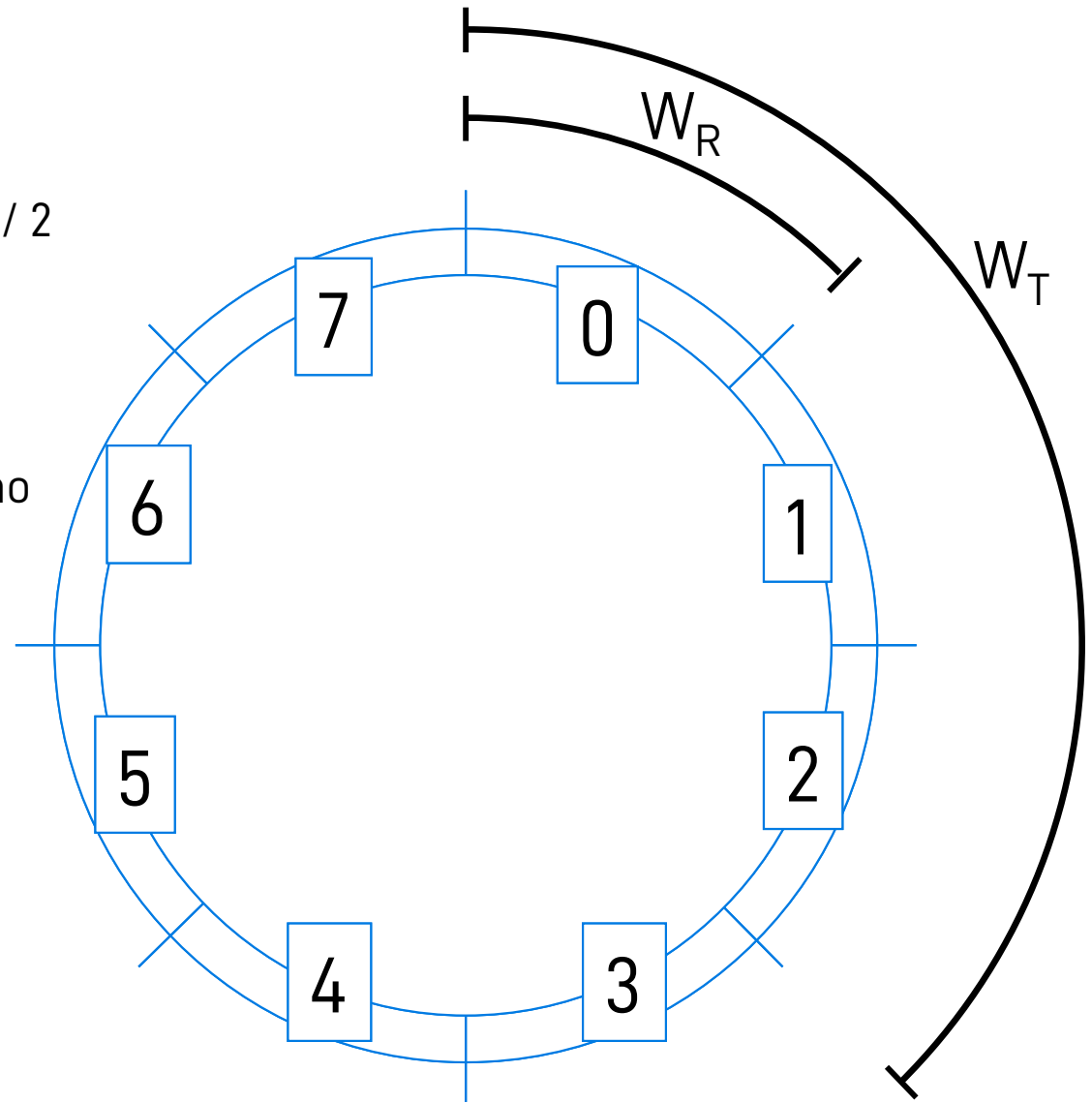
0123012
0123012
0123012

X
X
X

pkt0 è un duplicato,
ma viene erroneamente
accettato come
pacchetto nuovo!

Spazio dei numeri di sequenza

- ❑ Il numero di sequenza è ciclico
 - ❖ Con k bit si ha un periodo pari a 2^k
 - ❖ Se $|W_T| = |W_R|$, serve $|W_T| \leq 2^{k-1} = 2^k / 2$
 - ❖ **Regola generale:** $|W_T| + |W_R| \leq 2^k$
 - ❖ Idea di base: se **consegno correttamente tutti i segmenti, ma perdo tutti gli ACK**, W_T e W_R devono rimanere al massimo *contigue*, senza mai sovrapporsi
- ❑ Nell'esempio:
 - ❖ $k = 3 \rightarrow 2^k = 8$
 - ❖ $|W_T| = 3$, $|W_R| = 1$
 - ❖ Altrettanto validi:
 - $|W_T| = 7$, $|W_R| = 1$
 - $|W_T| = 3$, $|W_R| = 5$



D1: nel Go-Back-N vale la stessa cosa?

Capitolo 3: Livello di trasporto

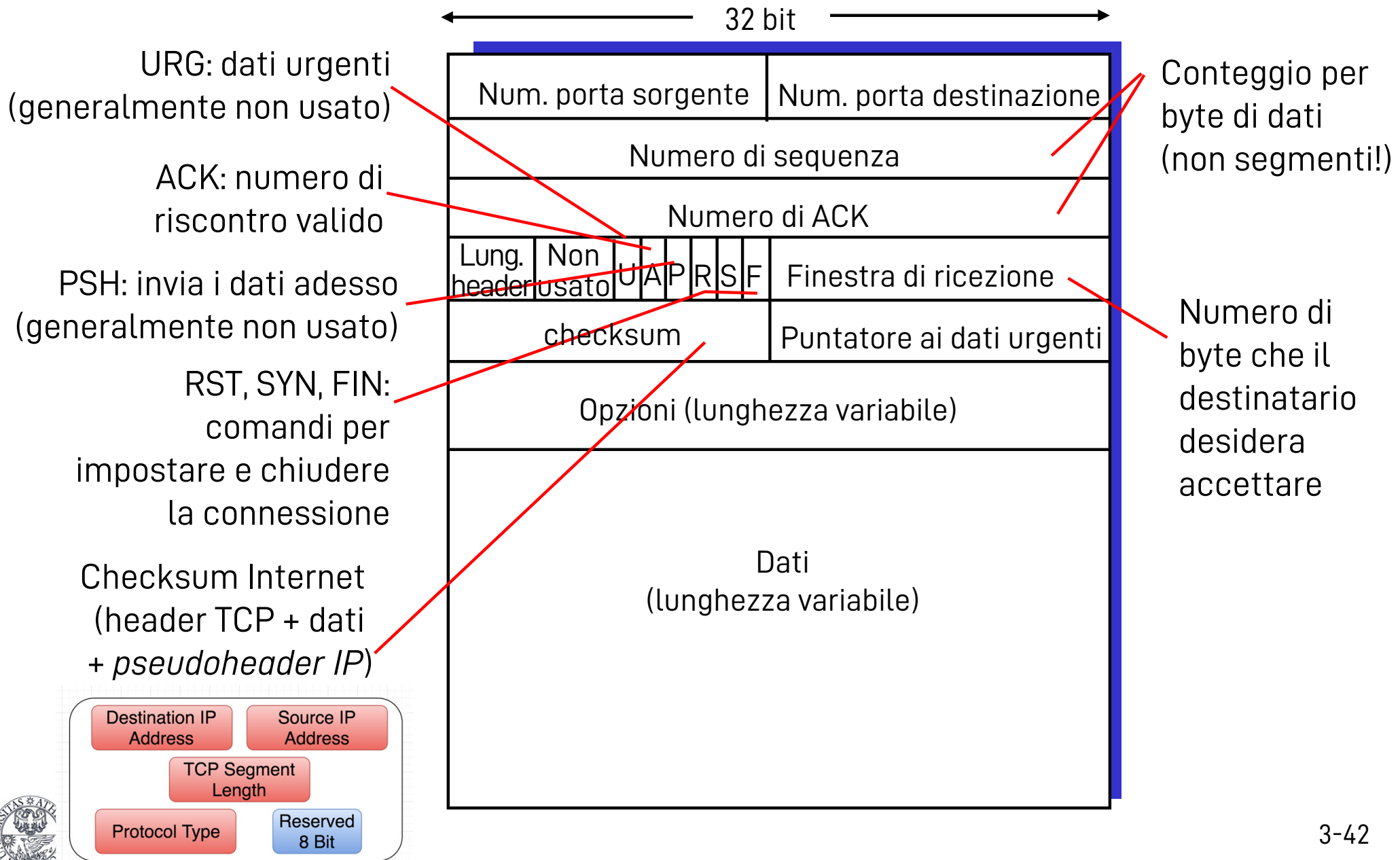
- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - struttura dei segmenti
 - trasferimento dati affidabile
 - controllo di flusso
 - gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

TCP: Panoramica

RFCs: 793, 1122, 1323, 2018, 2581, ...

- ❑ Protocollo punto-punto:
 - ❖ un mittente, un destinatario
- ❑ Flusso di byte affidabile e consegnato in ordine
 - ❖ Nessun "limite di messaggi"
- ❑ Con pipelining:
 - ❖ Meccanismi di controllo di flusso e di congestione TCP definiscono la dimensione della finestra
 - ❖ Le dimensioni delle finestre sono dinamiche (sia al mittente sia al destinatario)
 - ❖ Usa ACK cumulativi
- ❑ Il controllo di congestione evita di saturare la rete
- ❑ Full duplex:
 - ❖ Flusso di dati bidirezionale nella stessa connessione
 - ❖ MSS: dimensione massima di un segmento (maximum segment size)
- ❑ Orientato alla connessione:
 - ❖ Setup della connessione tramite handshaking (scambio di messaggi di controllo)
 - ❖ Inizializza lo stato di mittente e destinatario prima dello scambio di dati
- ❑ Flusso controllato:
 - ❖ Il trasmettitore non sovraccaricherà il ricevitore

Struttura dei segmenti TCP



(Dimensione della) Finestra di ricezione (RWND)

❑ RWND è un campo di 16 bit

- ❖ Indica il numero di byte che il ricevitore può immagazzinare
- ❖ Di conseguenza, rappresenta anche la massima quantità di dati in transito durante un RTT
- ❖ Valore massimo: 2^{16} Byte = 65536 Byte = 64 kByte

❑ Assumiamo RTT = 100 ms

- ❖ Dipende dalla velocità del link quanti dati possono essere "in volo"
(ricordate il bandwidth-delay product!!)

Bandwidth	Bandwidth-delay product
T1 (1.5 Mbps)	18 kB
Ethernet (10 Mbps)	122 kB
T3 (45 Mbps)	549 kB
FastEthernet (100 Mbps)	1.2 MB
STS-48 (2.5 Gbps)	29.6 MB

- ❖ RWND può essere aumentato usando un meccanismo di "scalatura"
(ovvero decidendo che il numero non conteggia Byte, ma multipli di Byte)

Numeri di sequenza e ACK di TCP

Numeri di sequenza:

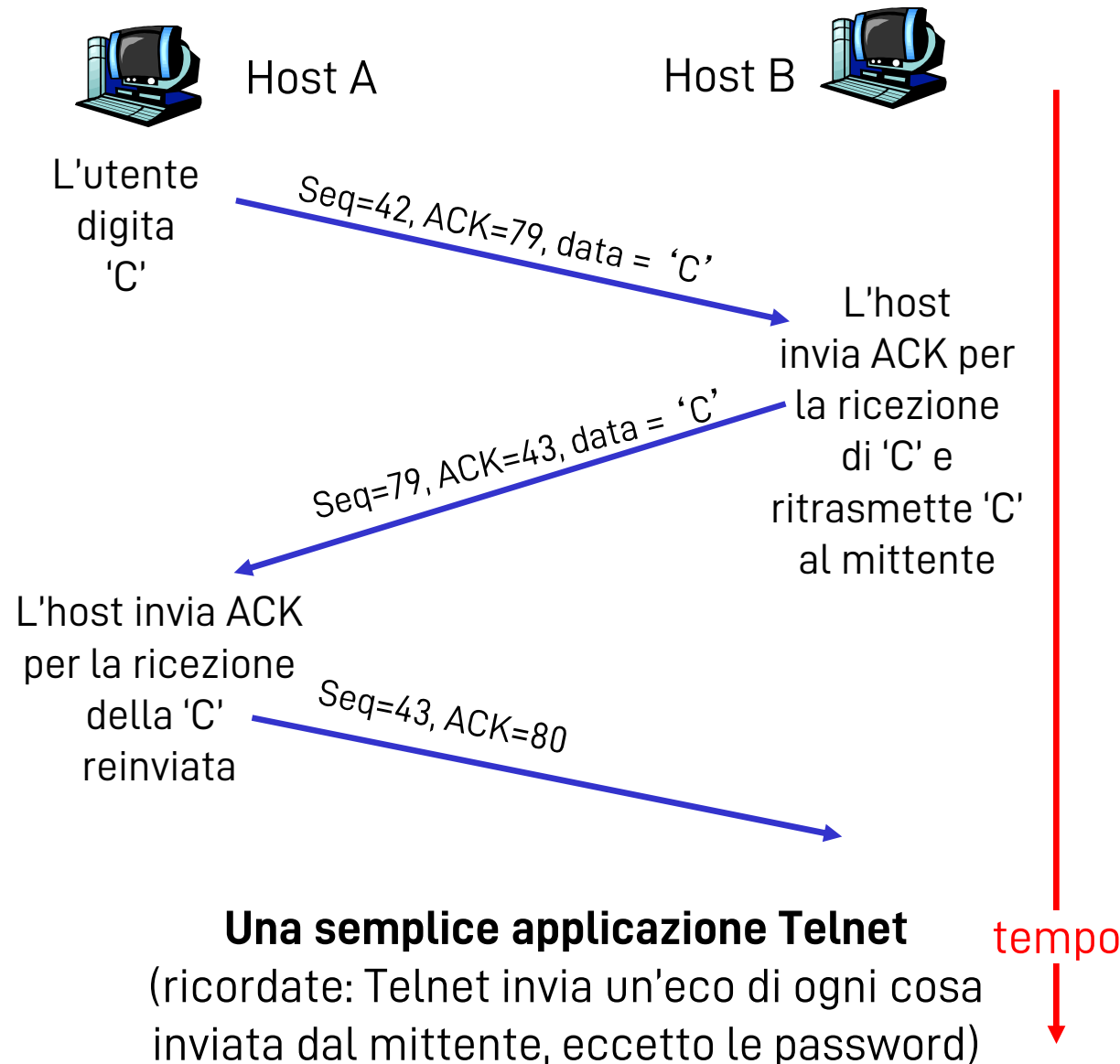
- "Numero" del primo byte del segmento nel flusso di byte

ACK:

- Numero di sequenza del prossimo byte atteso dall'altro lato
- ACK cumulativo

D: Come fa il destinatario a gestire segmenti fuori sequenza?

- R: la specifica TCP non lo dice – dipende dall'implementazione



tempo

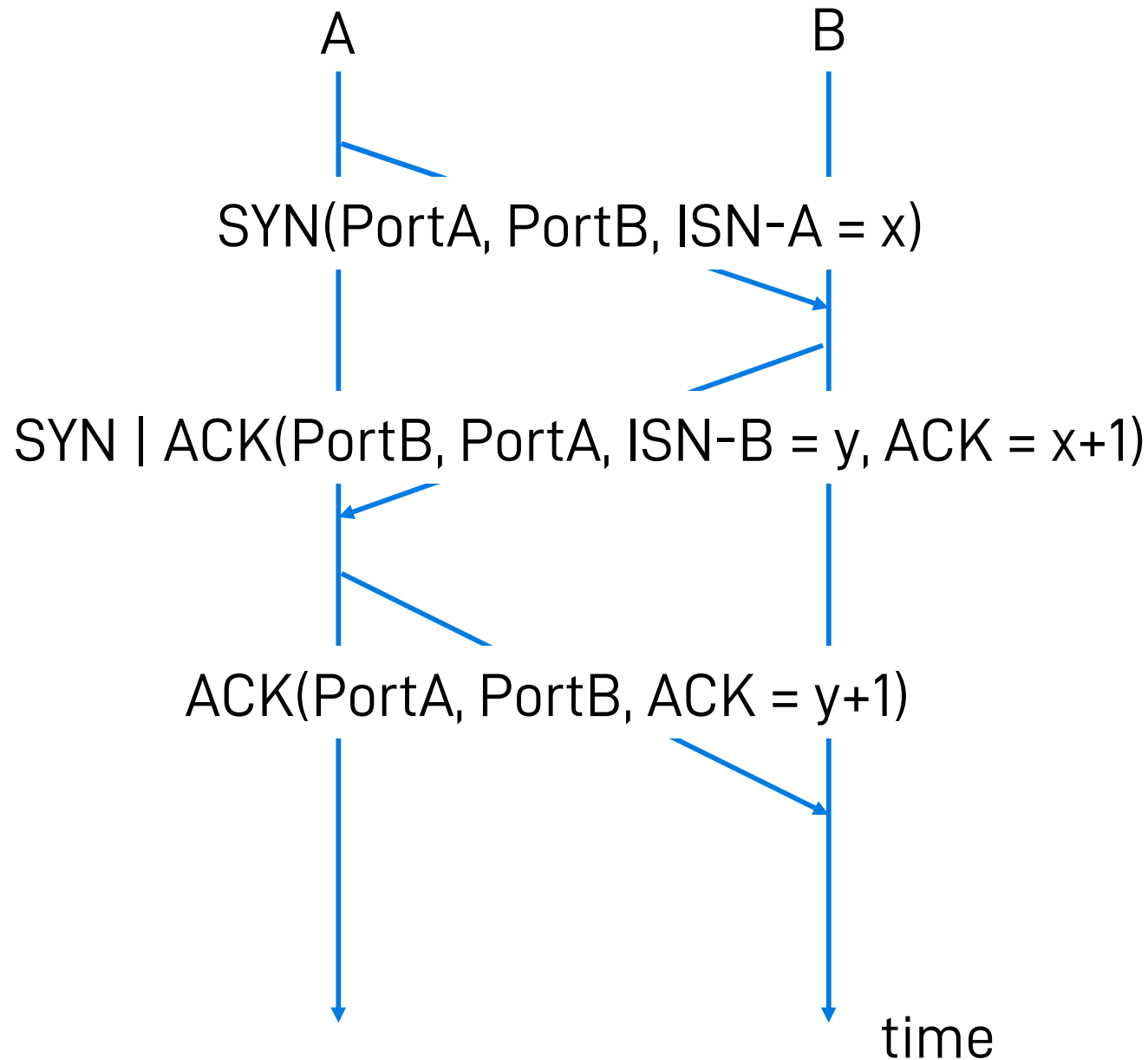
Setup della connessione TCP

- ❑ La procedura di setup della connessione TCP si chiama "three-way handshake"
- ❑ **Host A** (il client che inizia la connessione): segmento con flag SYN a 1
 - ❖ Porta sorgente (= A), Porta destinazione (= B)
 - ❖ Numero di sequenza iniziale (= x)
- ❑ **Host B** (server in attesa di connessioni): segmento con flag SYN e ACK a 1
 - ❖ Porta sorgente (= B), Porta destinazione (= A)
 - ❖ Numero di sequenza iniziale (= y)
 - ❖ Numero di ACK (= x + 1)
- ❑ **Host A**: segmento con il flag ACK a 1
 - ❖ Porta sorgente (= A), Porta destinazione (= B)
 - ❖ Numero di ACK (= y + 1)

D: perché l'handshake è proprio a 3 vie?

TCP connection setup

ISN =
Initial
Sequence
Number



- ▷ Frame 1 (74 bytes on wire, 74 bytes captured)
- ▷ Ethernet II, Src: AsustekC_b3:01:84 (00:1d:60:b3:01:84), Dst: Actionte_2f:47:87 (00:1d:60:2f:47:87)
- ▷ Internet Protocol, Src: 192.168.1.2 (192.168.1.2), Dst: 174.143.213.184 (174.143.213.184)
- ▼ Transmission Control Protocol, Src Port: 54841 (54841), Dst Port: http (80), Seq: 0
 - Source port: 54841 (54841)
 - Destination port: http (80)
 - [Stream index: 0]
 - Sequence number: 0 (relative sequence number)
 - Header length: 40 bytes
 - ▼ Flags: 0x02 (SYN)
 - 0... = Congestion Window Reduced (CWR): Not set
 - .0.. = ECN-Echo: Not set
 - ..0. = Urgent: Not set
 - ...0 = Acknowledgement: Not set
 - 0... = Push: Not set
 -0.. = Reset: Not set
 - ▷1. = Syn: Set
 -0 = Fin: Not set
 - Window size: 5840
 - ▷ Checksum: 0x85f0 [validation disabled]
 - ▷ Options: (20 bytes)

Offset	Hex	ASCII
0000	00 26 62 2f 47 87 00 1d 60 b3 01 84 08 00 45 00	.&b/G... ..E.
0010	00 3c 47 65 40 00 40 06 ad 64 c0 a8 01 02 ae 8f	.<Ge@.@. .d.....
0020	d5 b8 d6 39 00 50 f6 1c 6c be 00 00 00 00 a0 02	...9.P.. l.....
0030	16 d0 85 f0 00 00 02 04 05 b4 04 02 08 0a 00 0d	
0040	2b db 00 00 00 00 01 03 03 07	+.....

▶ Frame 2 (74 bytes on wire, 74 bytes captured)
 ▶ Ethernet II, Src: Actionte_2f:47:87 (00:26:62:2f:47:87), Dst: AsustekC_b3:01:84
 ▶ Internet Protocol, Src: 174.143.213.184 (174.143.213.184), Dst: 192.168.1.2 (192.168.1.2)
 ▼ Transmission Control Protocol, Src Port: http (80), Dst Port: 54841 (54841), Seq: 1234567890
 Source port: http (80)
 Destination port: 54841 (54841)
 [Stream index: 0]
 Sequence number: 0 (relative sequence number)
 Acknowledgement number: 1 (relative ack number)
 Header length: 40 bytes
 ▼ Flags: 0x12 (SYN, ACK)
 0... = Congestion Window Reduced (CWR): Not set
 .0... = ECN-Echo: Not set
 ..0. = Urgent: Not set
 ...1 = Acknowledgement: Set
 0... = Push: Not set
 0.. = Reset: Not set
 ▶1. = Syn: Set
 0 = Fin: Not set
 Window size: 5792
 ▶ Checksum: 0x4ff1 [validation disabled]
 ▶ Options: (20 bytes)
 ▶ [SEQ/ACK analysis]

0020	01 02 00 50 d6 39 fa 58 9c 88 f6 1c 6c bf a0 12	...P.9.Xl..
0030	16 a0 4f f1 00 00 02 04 05 b4 04 02 08 0a 12 cc	..0.....
0040	8c 71 00 0d 2b db 01 03 03 06	.q..+... ..

Lunghezza massima di un segmento TCP (Maximum Segment Size, MSS)

- ❑ TCP lavora per byte, ma non manda mai un singolo byte alla volta a meno che non sia forzato a farlo (tramite l'algoritmo di Nagle)
 - ❖ Cerca invece di inviare segmenti lunghi quanto un MSS
- ❑ L'MSS dipende da un parametro del livello di rete sottostante (es. IP) che si chiama Maximum Transfer Unit (MTU)
 - ❖ A sua volta, l'MTU del livello di rete dipende dall'MTU del livello data link
 - ❖ I parametri sopra dipendono dai link lungo *tutto* il percorso in Internet
- ❑ L'MSS si riferisce alla lunghezza del solo payload! (i dati trasportati dal segmento)
- ❑ Come scegliere l'MSS?
 - ❖ Non ci sono meccanismi per comunicarlo
 - ❖ "Trial and error": si tenta con MSS sempre più grandi, finché non ci si accorge che qualche segmento viene perso
 - In molti casi si riceve una comunicazione esplicita di incompatibilità, ma in molti altri casi... no

Calcolo MSS e valori tipici MTU

❑ Default

- ❖ MTU di Ethernet = 1500 Byte (payload dati inseribile in un frame a livello 2)
- ❖ Tale payload contiene i dati incapsulati dei protocolli di livello 3 e 4
- ❖ Header IP = 20 Byte
- ❖ Header TCP = 20 Byte

❑ MSS di TCP = 1460 Byte

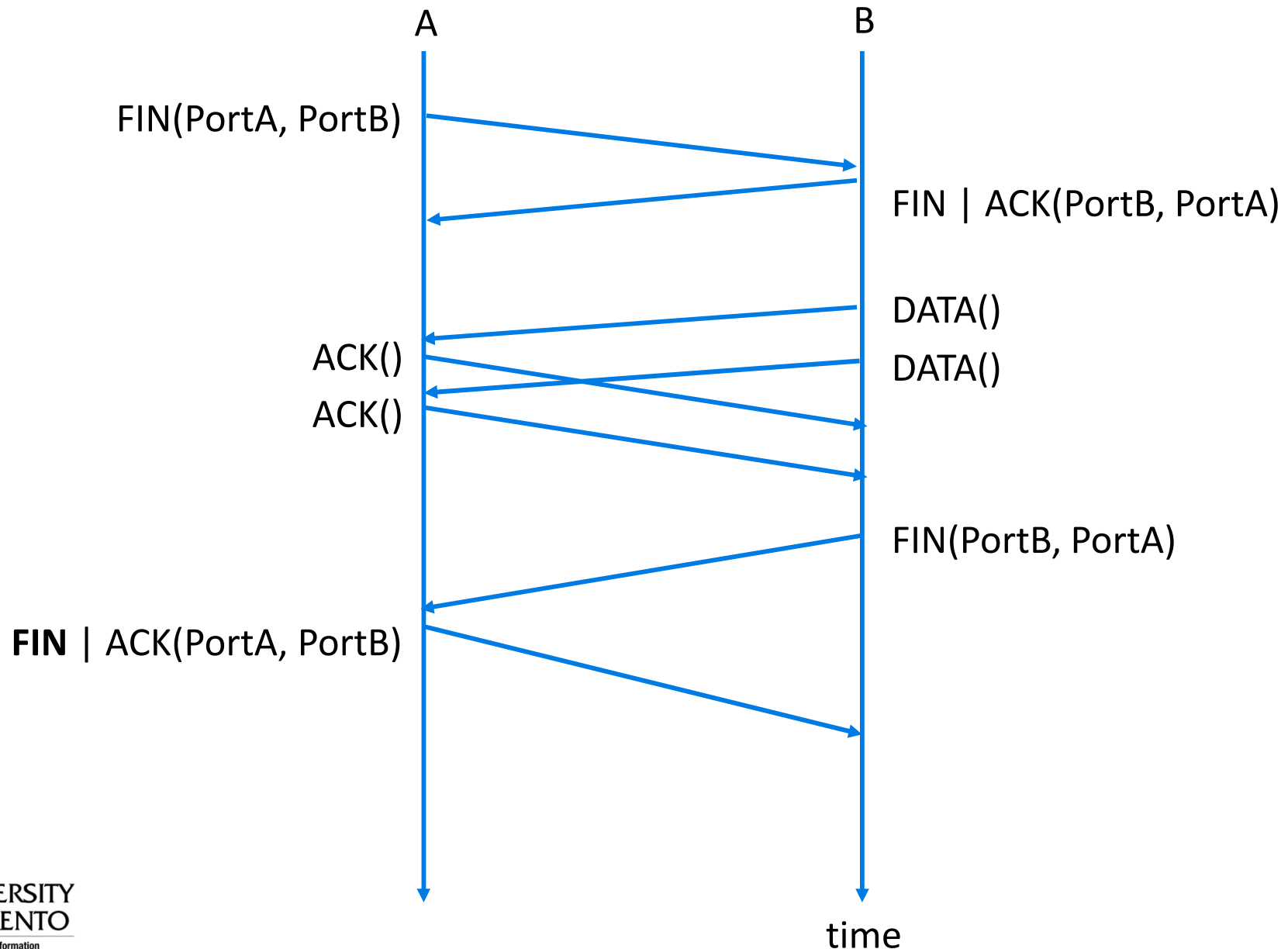
❑ “Least maximum”: MSS = 536 Byte

- ❖ La più piccola MTU impostabile per IP è 576 Byte
- ❖ ...meno 20 Byte header IP
- ❖ ...meno 20 Byte header TCP

Chiusura (“tear-down”) di una connessione TCP

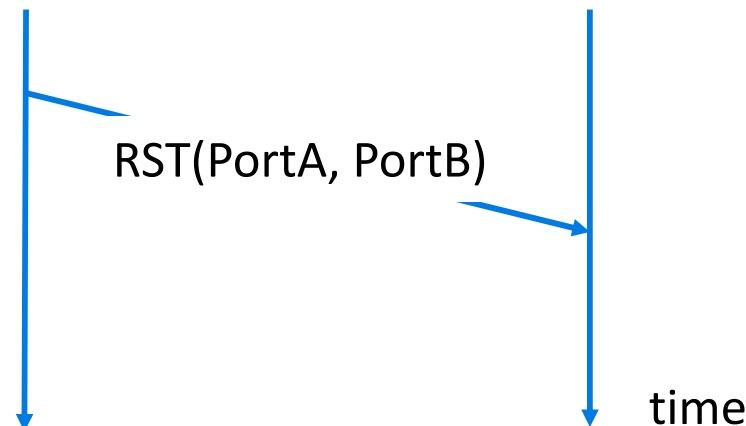
- ❑ Le connessioni TCP sono sempre bidirezionali
 - ❖ Bisogna chiuderle in entrambe le direzioni
- ❑ Procedura “gentile”: inviare un segmento TCP con flag FIN a 1
 - ❖ Il ricevitore invia un ACK
 - ❖ La connessione è semi-chiusa (“half-closed”)
- ❑ Si possono ancora mandare dati nella direzione opposta
 - ❖ Gli ACK inviati su una connessione semi-chiusa non costituiscono traffico dati
- ❑ La chiusura è completa solo quando si invia un FIN e si riceve un ACK anche nell'altra direzione

Chiusura ("tear-down") di una connessione TCP



Chiusura (“tear-down”) di una connessione TCP

- ❑ Chiusura “brusca”: reset (RST)
- ❑ RST si usa per resettare connessioni non gestibili o che si trovano in uno stato errato
 - ❖ Esempio: ricevo un ACK per una connessione mai aperta
- ❑ Uno degli host invia un segmento con flag RST a 1
 - ❖ Entrambi gli host liberano le risorse allocate dal sistema operativo
 - ❖ **D**: E se il RST si perde?
- ❑ I server possono usare il RST per chiudere connessioni velocemente



TCP: tempo di andata e ritorno e Retransmission TimeOut (RTO)

D: come impostare il valore del timeout di TCP?

- ❑ Più grande di RTT
 - Ma RTT varia
- ❑ Troppo piccolo:
timeout prematuro
 - Ritrasmissioni non necessarie
- ❑ Troppo grande:
reazione lenta alla perdita dei segmenti

D: come stimare RTT?

- ❑ **SampleRTT**: tempo misurato dalla trasmissione del segmento fino alla ricezione di ACK
 - Ignora le ritrasmissioni
- ❑ **SampleRTT** varia, quindi occorre una stima “più livellata” di RTT
 - Media di più misure recenti, non semplicemente il valore corrente di **SampleRTT**

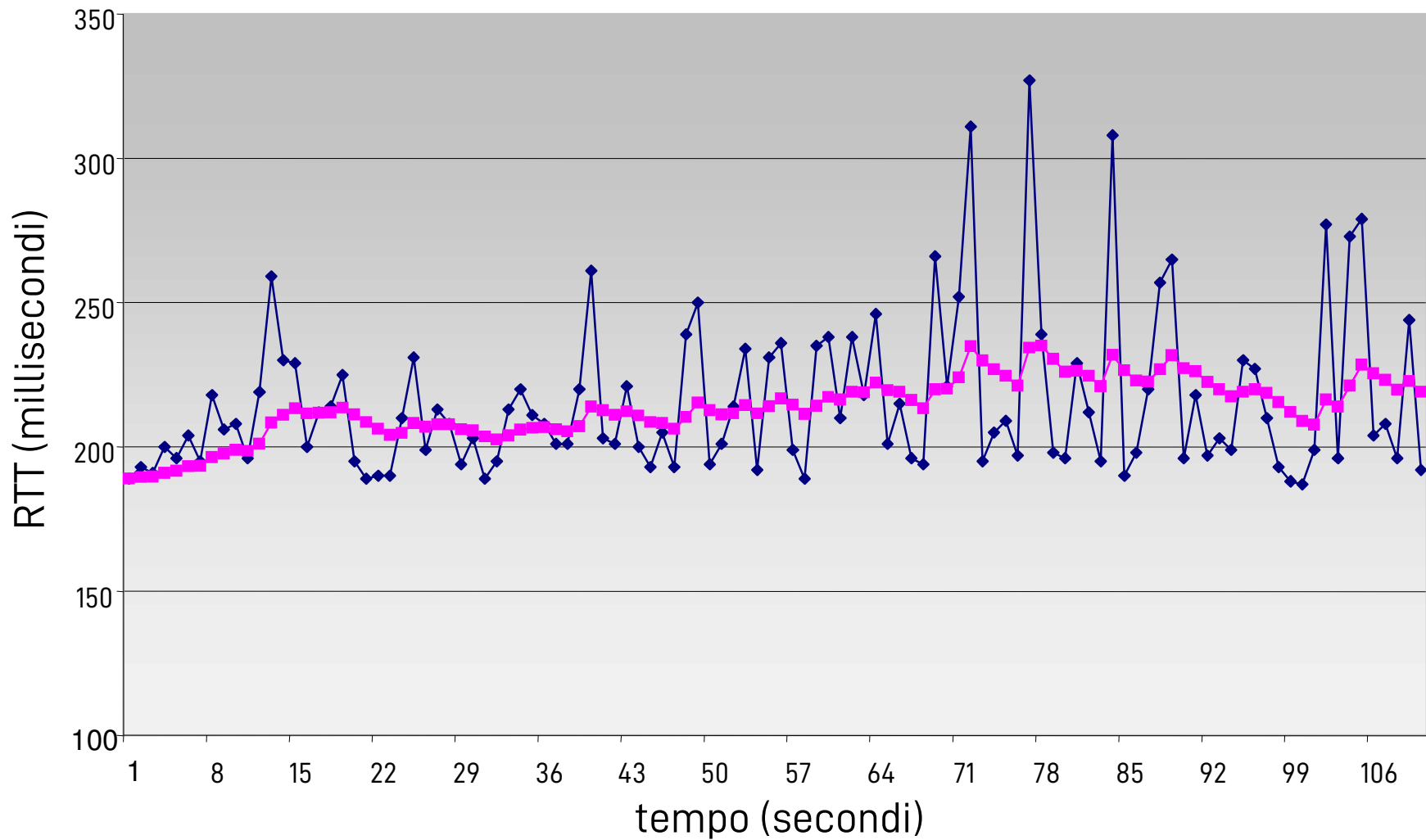
TCP: tempo di andata e ritorno e Retransmission TimeOut (RTO)

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❑ Media mobile esponenziale ponderata
- ❑ L'influenza dei vecchi campioni diminuisce esponenzialmente
- ❑ Valore tipico: $\alpha = 0,125$

Esempio di stima di RTT

RTT tra **gaia.cs.umass.edu** e **fantasia.eurecom.fr**



—◆— Campione RTT —■— Stime livellate di RTT

TCP: tempo di andata e ritorno e Retransmission TimeOut (RTO)

Impostazione dell'RTO

- ❑ **EstimatedRTT** più un “margine di sicurezza”
 - Grande variazione di **EstimatedRTT** → margine di sicurezza maggiore
- ❑ Stimare innanzitutto di quanto **SampleRTT** si discosta da **EstimatedRTT**:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(tipicamente, $\beta = 0.25$)

Poi impostare l'intervallo di timeout:

$$\text{RTO} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

Inizializzazione (RFC 6298)

- ❑ Retransmission Timeout (RTO) = 1 secondo
 - ❖ 0 un qualunque valore maggiore di questo

- ❑ Quando si ha una sola misura di RTT, si pone:
 - ❖ $\text{EstimatedRTT} = \text{SampleRTT}$
 - ❖ $\text{DevRTT} = \text{RTT} / 2$

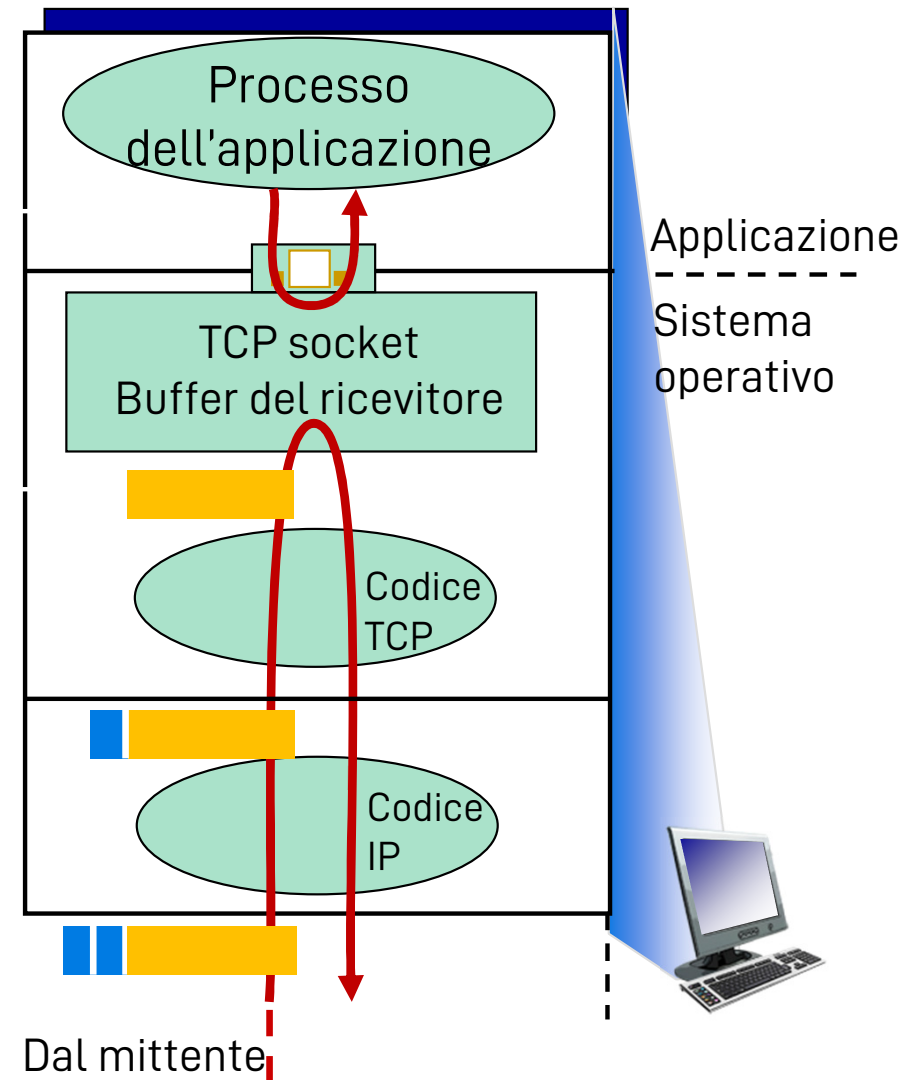
TCP: controllo di flusso

L'applicazione potrebbe prelevare i dati dal buffer TCP più *lentamente*...

... di quanto il codice che esegue le regole di TCP sia capace di inviarli

❑ Controllo di flusso

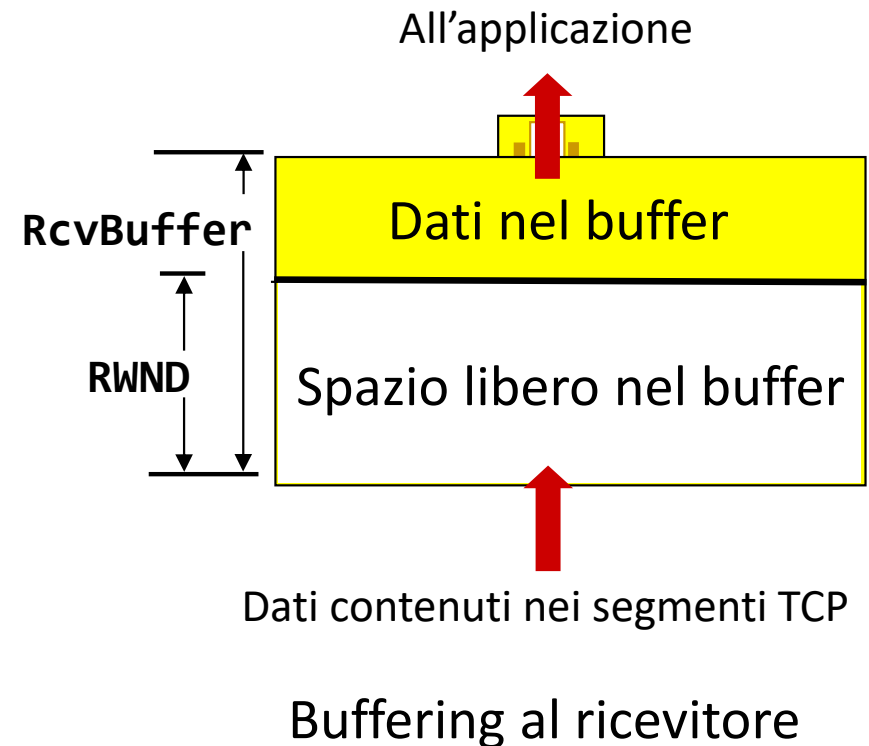
- ❖ Premette al ricevitore di controllare la velocità di trasmissione del mittente
- ❖ Troppi dati, troppo in fretta



Pila ("stack") protocollare al ricevitore

TCP: controllo di flusso

- ❑ Il ricevitore comunica quanto spazio libero ha nel proprio buffer di ricezione, includendo il valore della RWND nell'header TCP di ogni segmento che invia al mittente
- ❑ Il mittente limita la quantità di dati "in volo" (ancora non ACKati) al valore di RWND
- ❑ Garantisce che il buffer del ricevitore non vada in overflow (lo costringerebbe a scartare pacchetti corretti per mancanza di spazio)



Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - struttura dei segmenti
 - trasferimento dati affidabile
 - controllo di flusso
 - gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

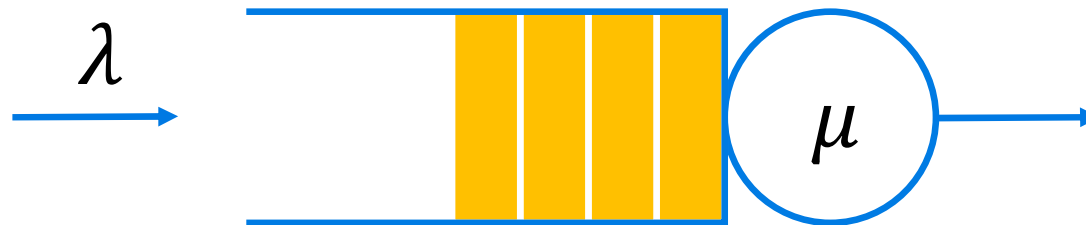
Principi del controllo di congestione

Congestione:

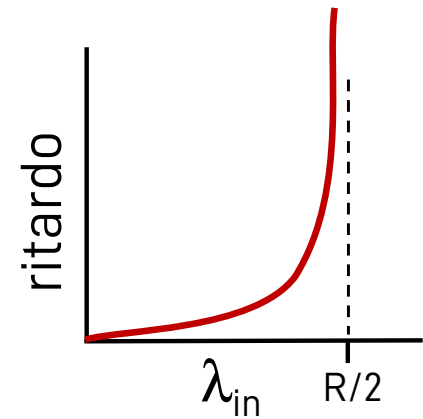
- ❑ Informalmente: "troppi trasmettitori stanno mandando troppi dati e la rete non riesce a gestire tutto questo traffico"
- ❑ E' diverso dal controllo di flusso!
- ❑ Come si manifesta:
 - ❖ Pacchetti persi (buffer overflow nei router)
 - ❖ Ritardi lunghi (accodamento nei buffer dei router)
- ❑ Nella top-10 dei problem dell'informatica moderna

Modelli per sistemi a coda (non materiale d'esame)

- ❑ Un sistema a coda comprende una “fila d'attesa” e un server
 - ❖ Tasso (frequenza) di arrivo λ : numero medio di pacchetti (o più in generale: di unità di lavoro) che entra nella coda per unità di tempo
 - ❖ Tasso (frequenza) di servizio μ : tempo medio richiesto dal server per trasmettere un pacchetto (in generale: concludere un lavoro)
- ❑ Arrivi e servizi avvengono secondo una distribuzione statistica
 - ❖ Es.: M/M/1: Arrivi $\sim \exp\left(\frac{1}{\lambda}\right)$, Tempi servizio $\sim \exp\left(\frac{1}{\mu}\right)$, 1 server
 - ❖ M = “Markovian” $\rightarrow$ si riferisce alla distribuzione esponenziale



Modelli per sistemi a coda (non materiale d'esame)



❑ Si possono calcolare le proprietà del sistema:

❖ Per un sistema M/M/1:

- Definiamo il carico del server come $\rho = \frac{\lambda}{\mu}$
- La lunghezza media della coda è: $L = \frac{\rho^2}{1-\rho}$ (per $\rho < 1$)
- La probabilità che ci siano N pacchetti in coda è: $\pi_N = (1 - \rho)\rho^N$

❖ Es.: $\lambda = 0.8$ pkt/s, $\mu = 1$ pkt/s (M/M/1: medie di distribuzioni esponenziali)

- $L = 3.2$ pkt
- $\pi_0 = 0.2, \pi_1 = 0.16, \pi_2 = 0.128, \pi_3 = 0.1024, \dots$

❑ Ricordate: λ e μ sono solo delle medie statistiche, non delle costanti

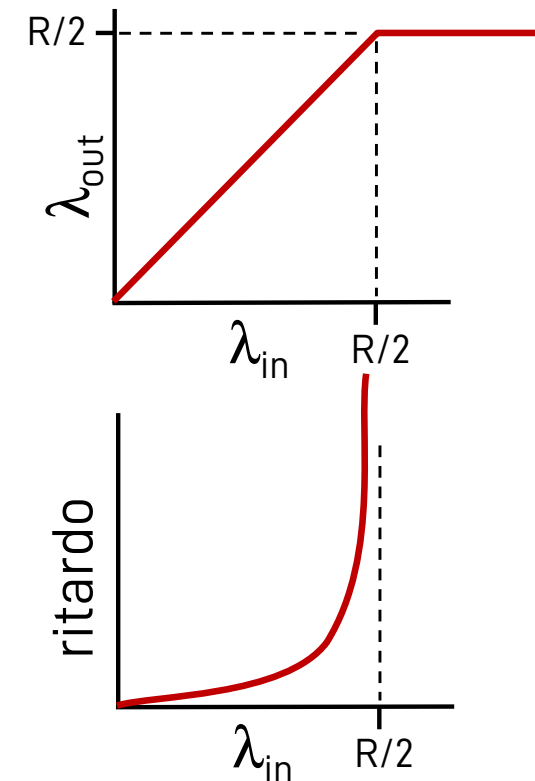
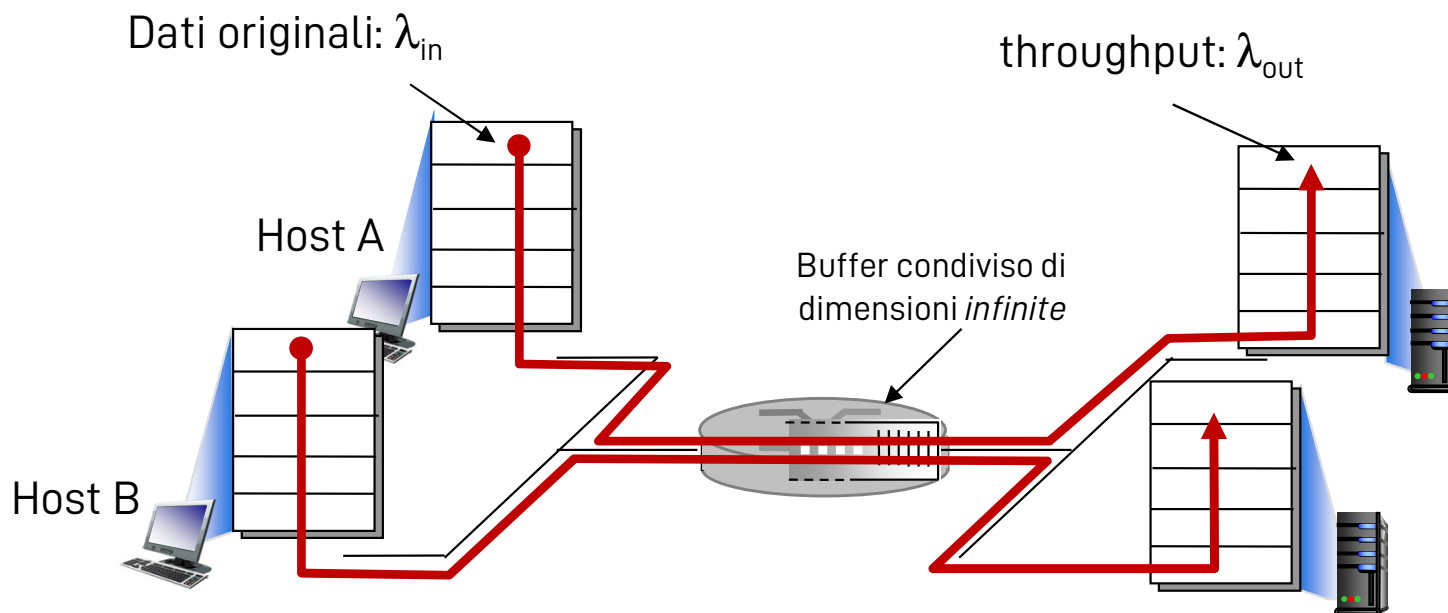
❖ Qualche volta passa più tempo o meno tempo tra arrivi e tra servizi

❑ Anche se λ è molto piccolo e μ molto grande, c'è una probabilità $\neq 0$ (magari bassa) che i pacchetti si accodino e magari vengano scartati

Cause/costi della congestione:

Scenario 1

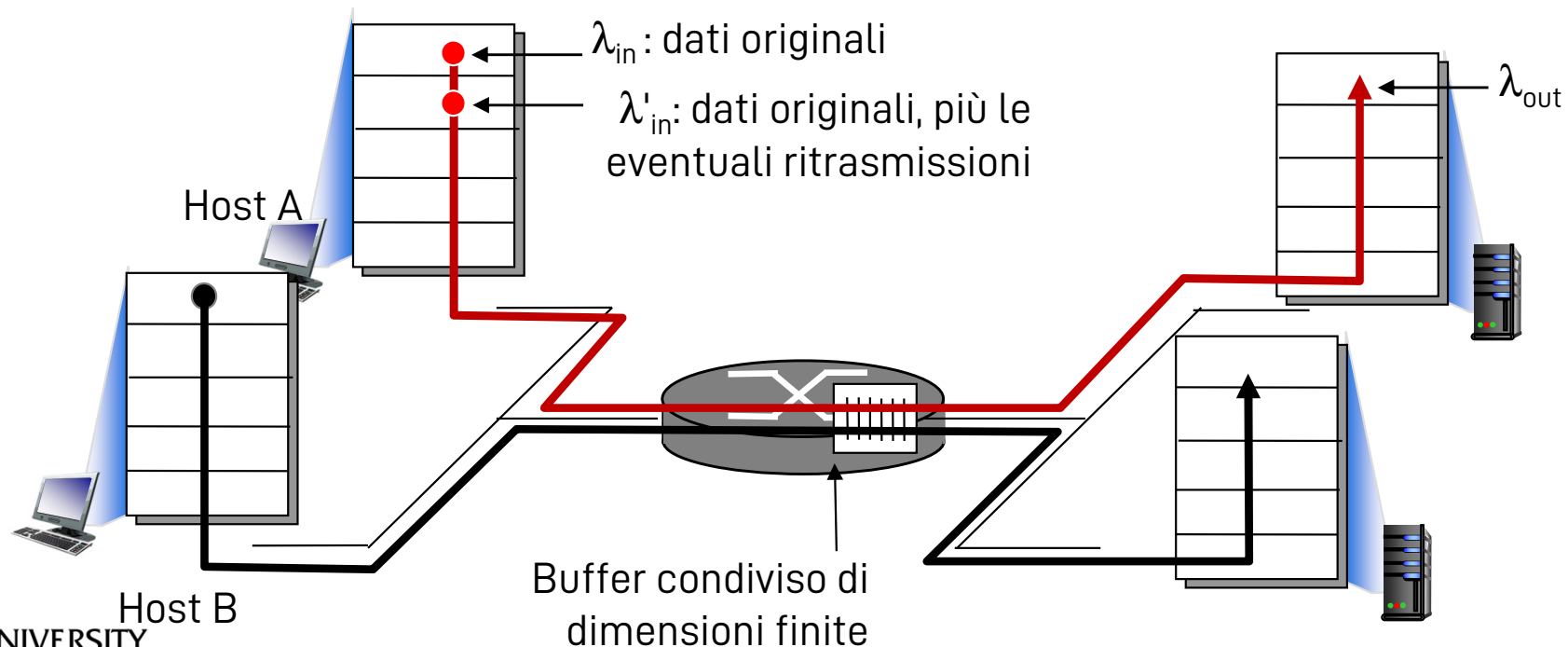
- ❑ Due trasmettitori, due ricevitori
- ❑ Un router, buffer infinito
- ❑ Capacità del link di uscita: R
- ❑ No ritrasmissioni
- ❑ Throughput massimo per connessione: $R/2$
- ❑ Ritardi elevati se il tasso di arrivo λ_{in} si avvicina a $R/2$



Cause/costi della congestione:

Scenario 2

- ❑ Un router, buffer di dimensione finita
- ❑ Il mittente ritrasmette i pacchetti finiti in timeout
 - ❖ Tasso di arrivo dall'applicazione al mittente: λ_{in} ,
tasso percepito dall'applicazione del destinatario: λ_{out}
 - ❖ Al livello di trasporto, il tasso di arrivo **include** le ritrasmissioni! $\lambda'_{in} > \lambda_{in}$

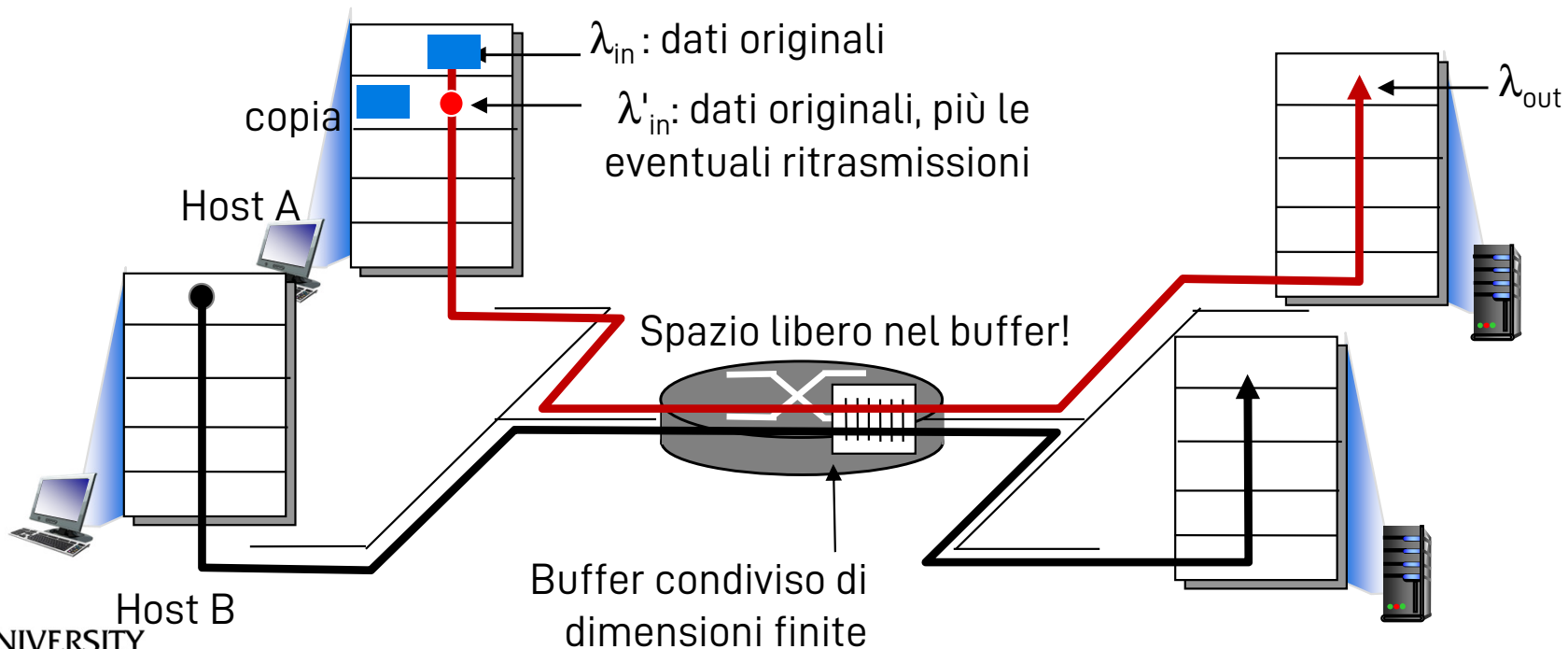
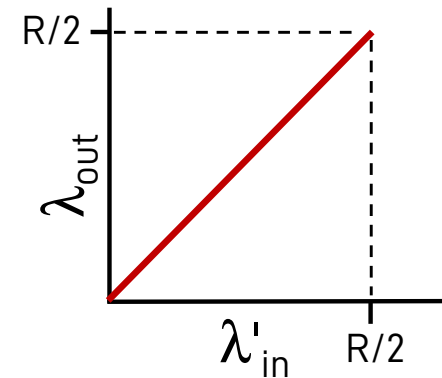


Cause/costi della congestione:

Scenario 2

Caso ideale: conoscenza perfetta della situazione

- ❑ Il mittente invia dati solo se il router ha spazio nel buffer
- ❑ Risultato ideale

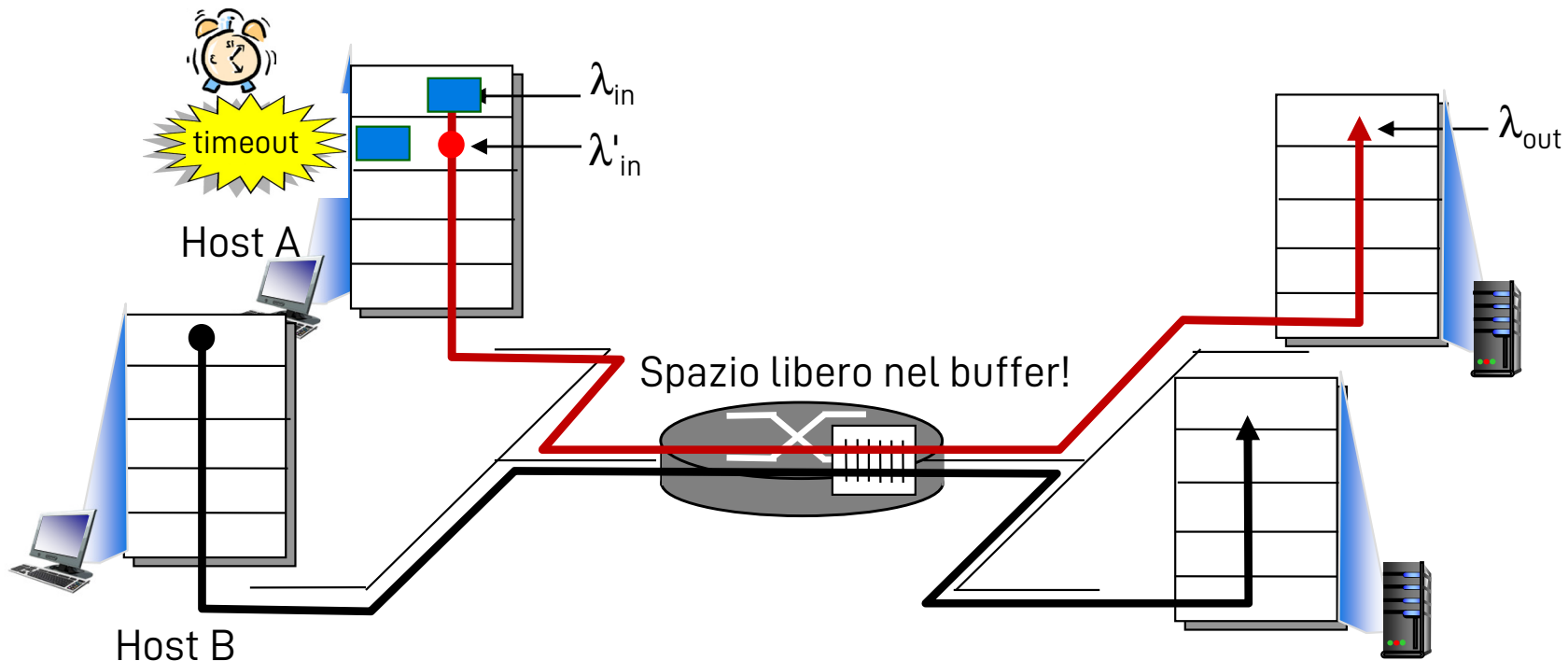
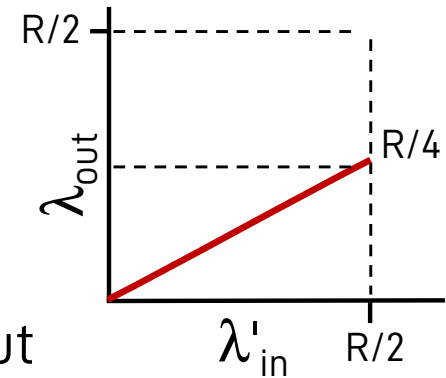


Cause/costi della congestione:

Scenario 2

Caso realistico: conoscenza limitata della rete

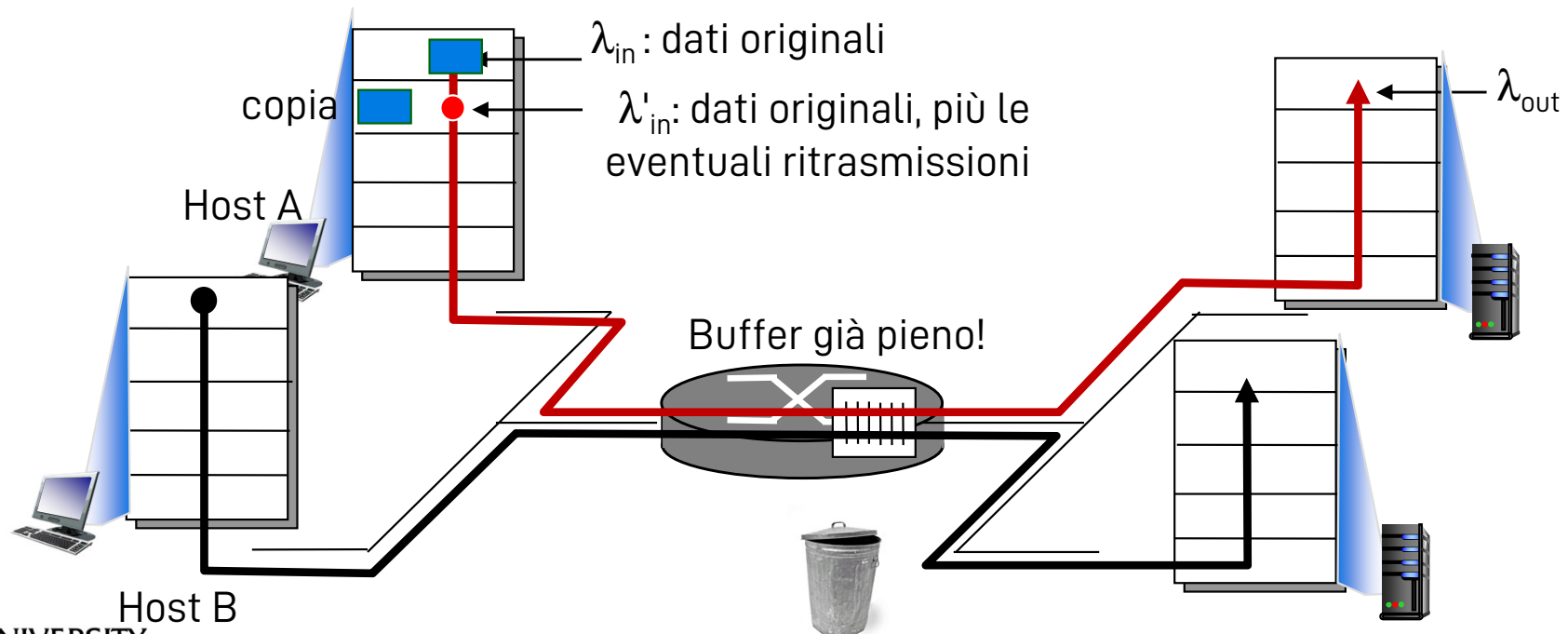
- ❑ La congestione causa ritardi
- ❑ Se al mittente scatta un timeout, il mittente invia una seconda copia dello stesso pacchetto
 - ❖ Vengono consegnate entrambe, dimezzando il throughput



Cause/costi della congestione:

Scenario 2

Idealizzazione: il mittente reinvia pacchetti solo se è sicuro che si siano persi lungo la strada (ovvero, i timeout sono molto lunghi)

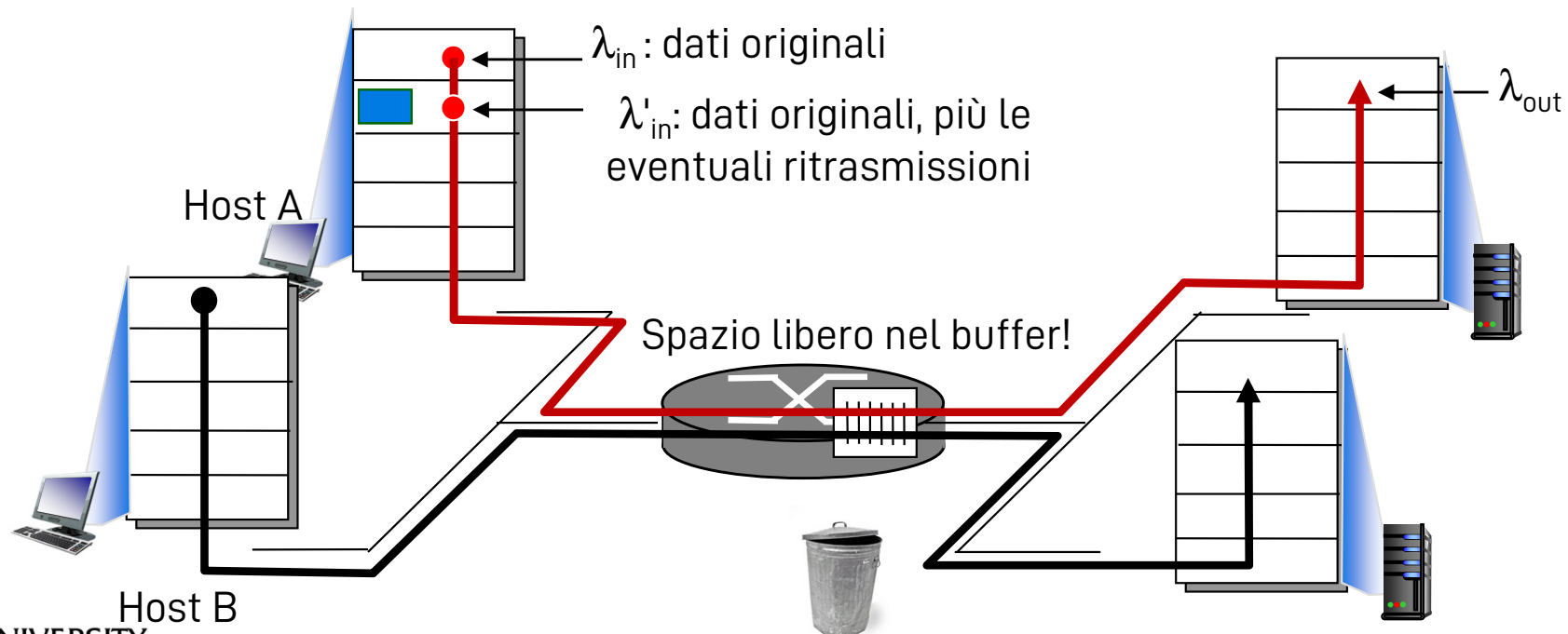
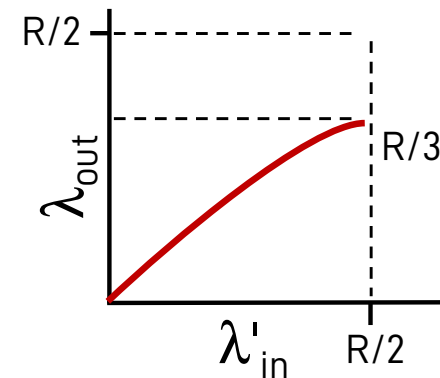


Cause/costi della congestione:

Scenario 2

Idealizzazione: il mittente reinvia pacchetti solo se è sicuro che si siano persi lungo la strada (ovvero, i timeout sono molto lunghi)

- ❑ Throughput potenzialmente migliore
 - ❖ Dipende sempre da perdite e timeout
- ❑ Take-home message: congestione → ritrasmissioni da gestire accuratamente

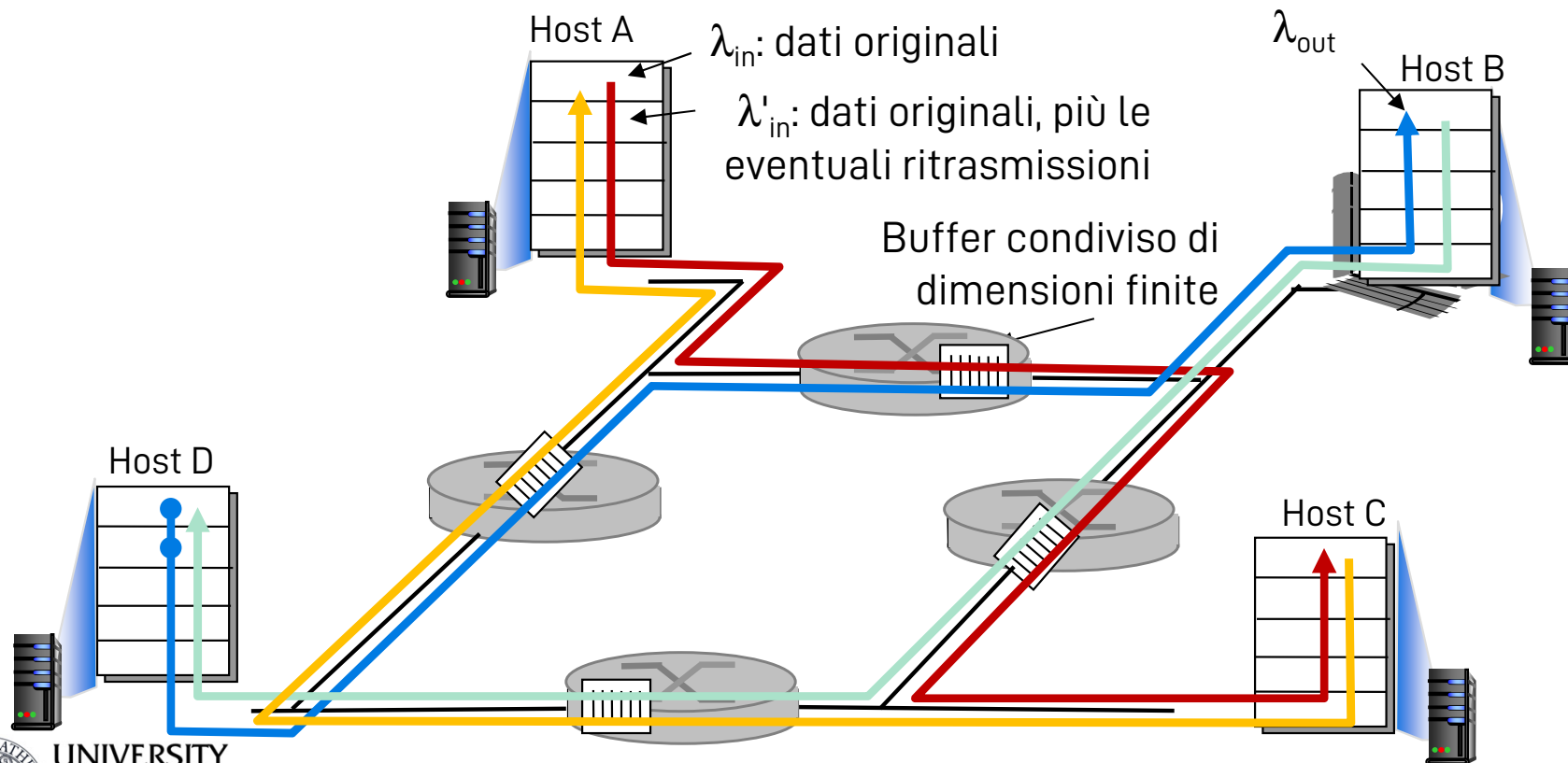


Cause/costi della congestione:

Scenario 3

❑ Rotte multi-salto con perdite

- ❖ **D:** che succede a λ_{out} quando λ_{in} e λ'_{in} aumentano senza controllo?
- ❖ **R:** quando λ'_{in} aumenta, tutti o quasi i pacchetti blu in arrivo alla coda in alto a destra vengono scartati, $\lambda_{out} \rightarrow 0$

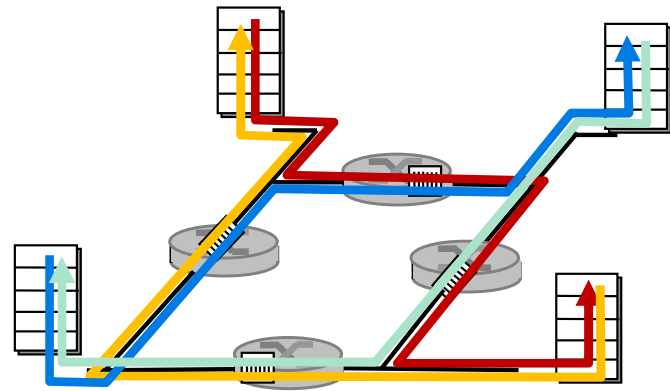
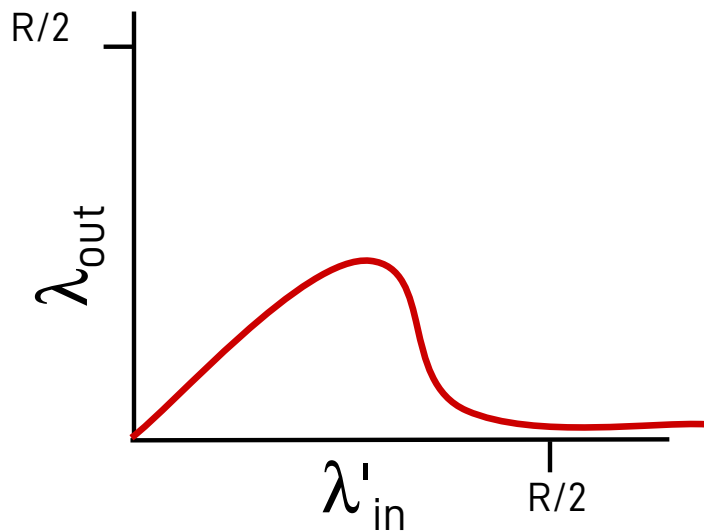


Cause/costi della congestione:

Scenario 3

Un altro "costo" della congestione:

- ❑ Ogni volta che si scarta un pacchetto, tutte le risorse usate per portare il pacchetto fino a quel punto risultano sprecate!



Cause/costi della congestione:

Riassunto

❑ Buffer infiniti

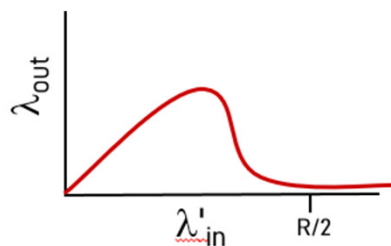
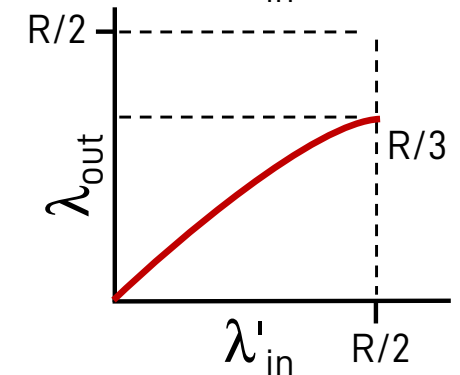
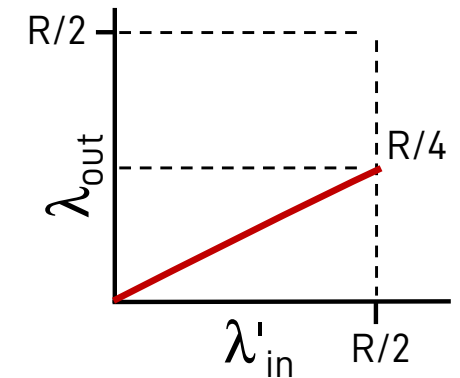
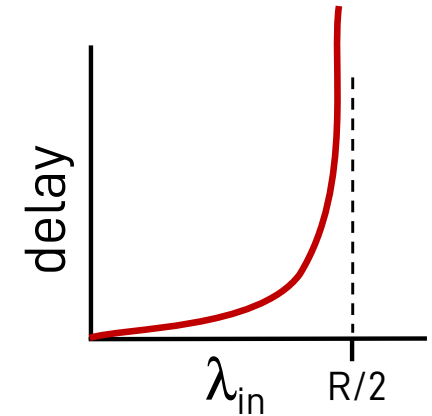
- ❖ No perdite, ma se tasso di arrivo = tasso di servizio...
- ❖ **Costo:** ritardi molto elevati

❑ Ritardi dovuti alla congestione: timeout

- ❖ **Costo:** spreco di risorse
 - Ritrasmissioni inutili
 - Nel grafico a destra: 1 ritrasmissione per pacchetto

❑ Pacchetti scartati a causa della congestione

- ❖ **Costi:** ritrasmissioni
- ❖ Più lavoro richiesto per ottenere lo stesso throughput percepito **a livello applicazione**
- ❖ Tale throughput potrebbe ridursi a zero sprecando gli sforzi fatti



Capitolo 3: Livello di trasporto

- ❑ 3.1 Servizi a livello di trasporto
- ❑ 3.2 Multiplexing e demultiplexing
- ❑ 3.3 Trasporto senza connessione: UDP
- ❑ 3.4 Principi del trasferimento dati affidabile
- ❑ 3.5 Trasporto orientato alla connessione: TCP
 - struttura dei segmenti
 - trasferimento dati affidabile
 - controllo di flusso
 - gestione della connessione
- ❑ 3.6 Principi del controllo di congestione
- ❑ 3.7 Controllo di congestione TCP

Controllo di congestione in TCP: attenzione!

- ❑ Non c'è un solo algoritmo in TCP per gestire la congestione
- ❑ Molte varianti sono state proposte nel tempo, ciascuna per rimuovere una limitazione delle versioni precedenti
 - ❖ TCP Tahoe, Reno, New Reno, Vegas, Westwood, CUBIC, BBR...
- ❑ L'implementazione di TCP spesso dipende dal sistema operativo
- ❑ Ora ci concentriamo su una versione specifica descritta nel libro
 - ❖ Anche questa versione non è del tutto completa
- ❑ In ogni caso, non dite mai a nessuno che “di TCP ce n'è uno solo”
- ❑ Le implementazioni di TCP ragionano in byte
 - ❖ Per semplicità, noi nel seguito ragioneremo in segmenti quanto più possibile

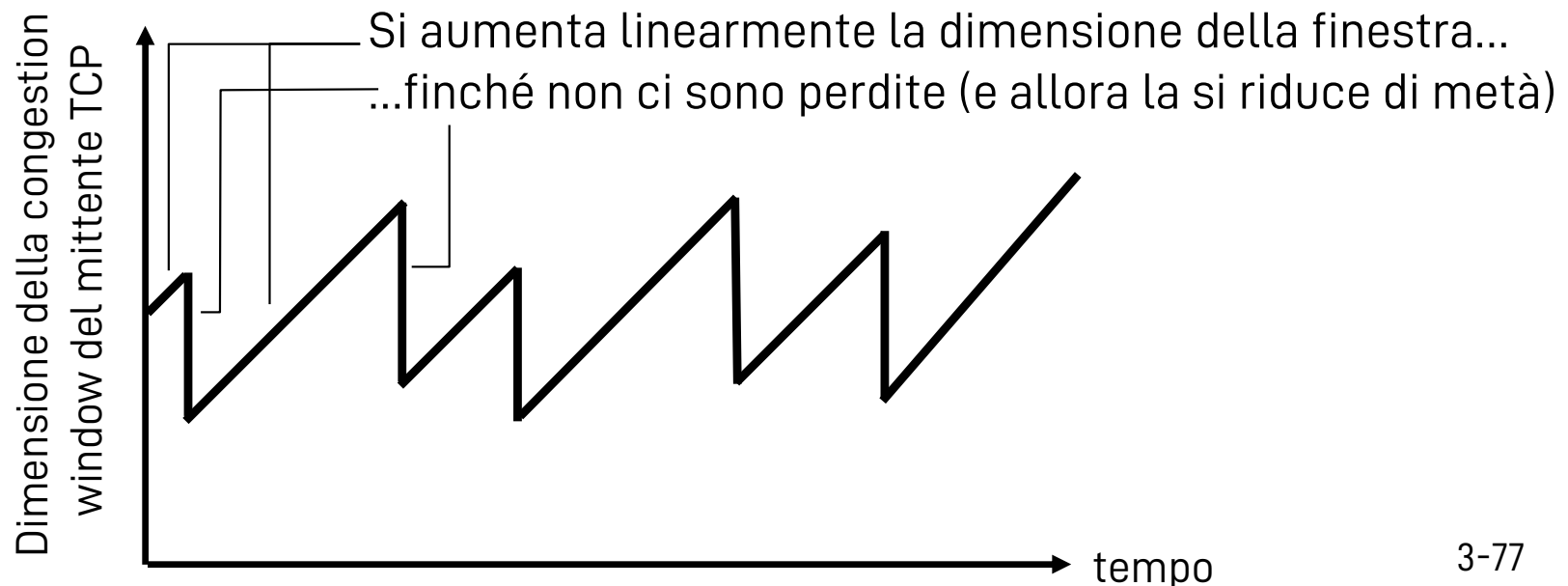
Controllo di congestione

- ❑ E' una delle caratteristiche più importanti di TCP
 - ❖ Adatta il tasso di trasmissione alle condizioni di rete
 - ❖ Evita di saturare e congestionare la rete
- ❑ Diversi approcci possibili
 - ❖ Controllo di congestione end-to-end
 - Non coinvolge la rete
 - Si capisce se c'è congestione osservando perdite di pacchetti e ritardi
 - Metodo usato da TCP
 - ❖ Controllo di congestione assistito dalla rete
 - I router forniscono feedback agli host sullo stato della rete
 - Es. un singolo bit per indicare la congestione
(SNA, DECbit, TCP/IP Explicit Congestion Notification (ECN), ATM)

TCP congestion control:

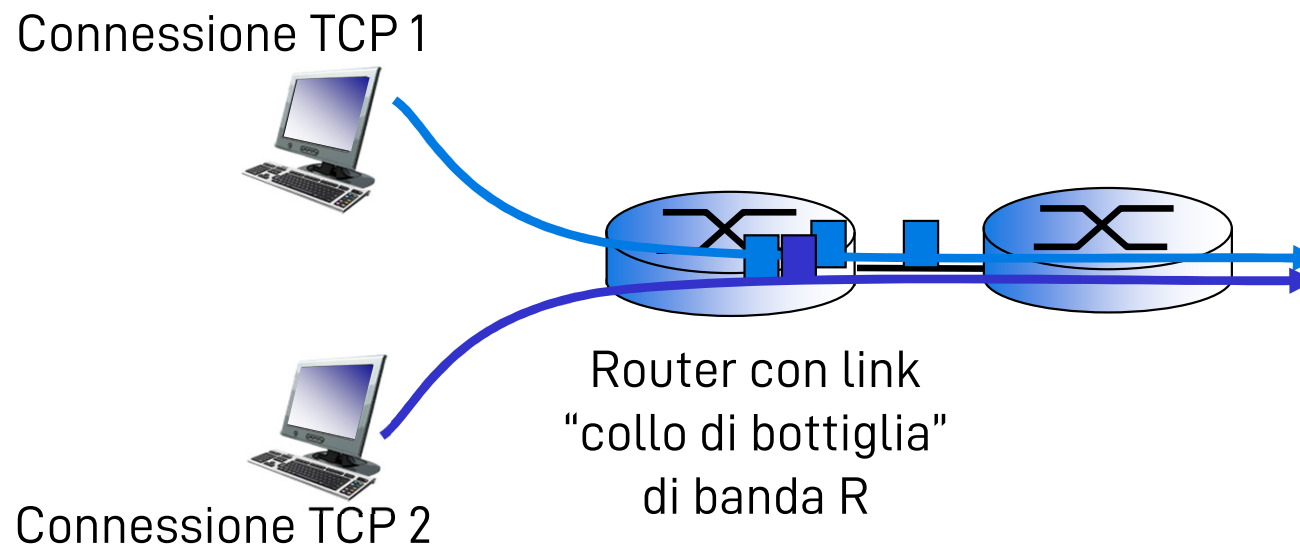
Additive Increase Multiplicative Decrease (AIMD)

- ❑ Approccio: il mittente aumenta il tasso di trasmissione (cioè la dimensione della finestra) cercando di occupare la banda disponibile, finché non si rilevano perdite
 - ❖ Additive increase: aumenta la finestra di 1 MSS ogni RTT finché non ci sono perdite
 - ❖ Multiplicative decrease: riduci la finestra (tipicamente della metà) quando si rileva una perdita



Perché usare AIMD?

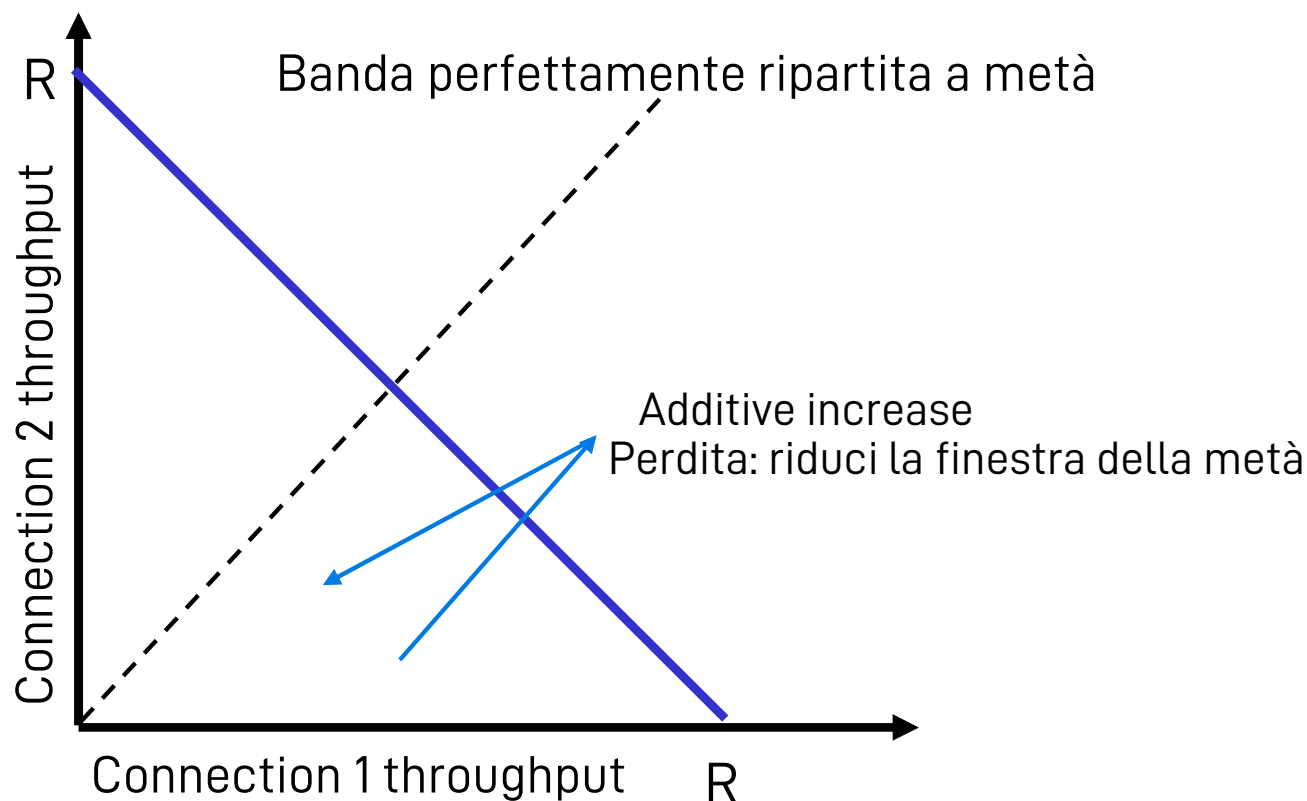
- ❑ Per ottenere equità (fairness): se K sessioni TCP si dividono uno stesso link di banda R che fa da "collo di bottiglia", le sessioni dovrebbero percepire tutte la stessa banda R/K



Perché usare AIMD? (cont.)

Due sessioni in competizione sullo stesso link di banda R : grafico in cui la banda della connessione 1 è sull'asse x , e la banda della connessione 2 è sull'asse y

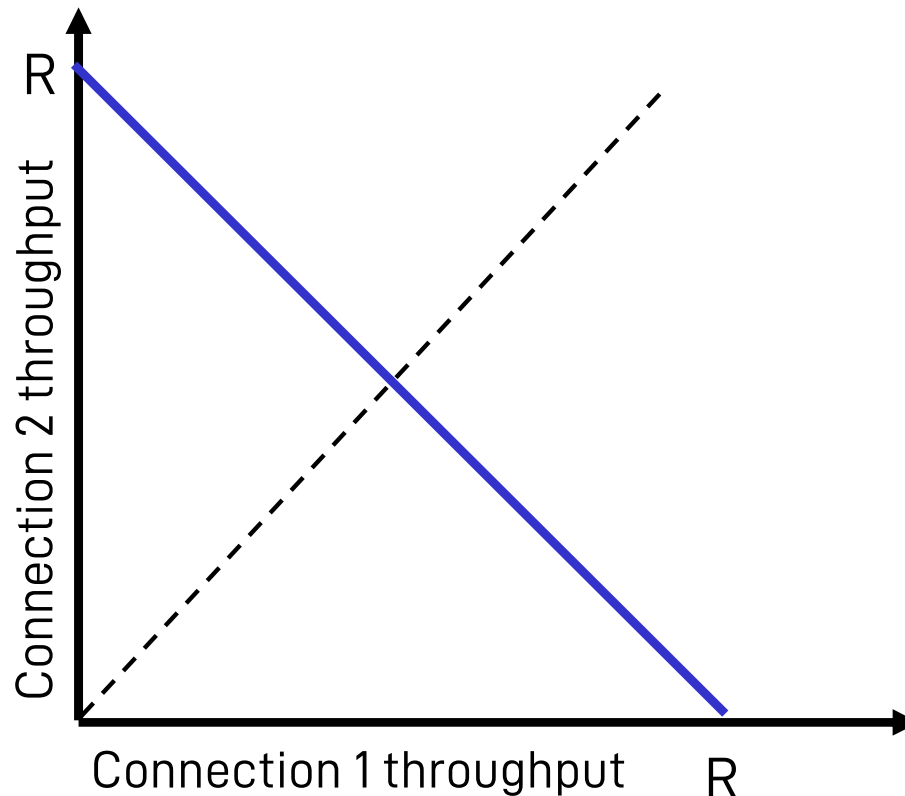
- ❑ Tracciamo sul grafico come cambia la banda delle connessioni nel tempo
 - ❖ Additive increase fa aumentare linearmente la banda di entrambe (pendenza = 1)
- ❑ Multiplicative decrease reduce il throughput proporzionalmente (cioè, maggior diminuzione per la connessione che stava usando più banda)



Perché usare AIMD? (cont.)

Due sessioni in competizione sullo stesso link di banda R : grafico in cui la banda della connessione 1 è sull'asse x , e la banda della connessione 2 è sull'asse y

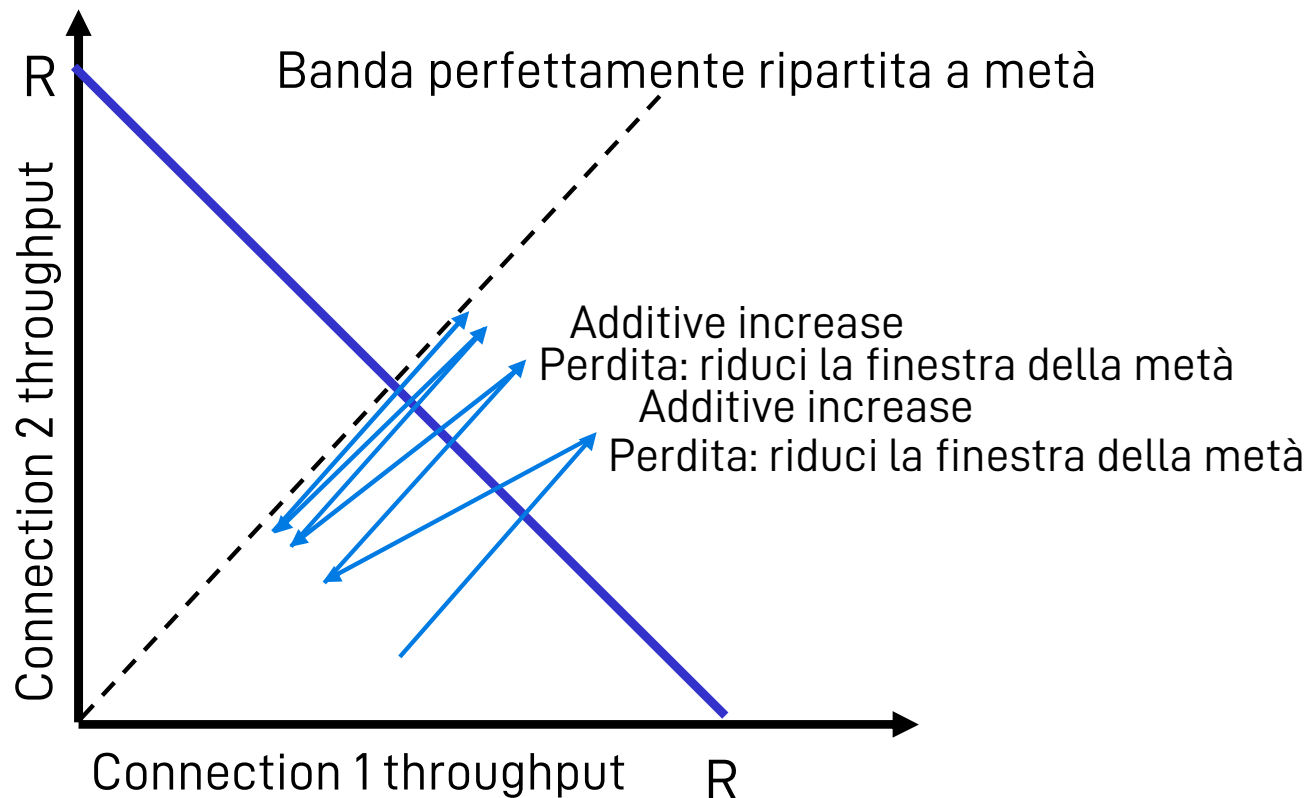
- ❑ Tracciamo sul grafico come cambia la banda delle connessioni nel tempo
 - ❖ Additive increase fa aumentare linearmente la banda di entrambe (pendenza = 1)
- ❑ Multiplicative decrease reduce il throughput proporzionalmente (cioè, maggior diminuzione per la connessione che stava usando più banda)



Perché usare AIMD? (cont.)

Due sessioni in competizione sullo stesso link di banda R : grafico in cui la banda della connessione 1 è sull'asse x , e la banda della connessione 2 è sull'asse y

- ❑ Tracciamo sul grafico come cambia la banda delle connessioni nel tempo
 - ❖ Additive increase fa aumentare linearmente la banda di entrambe (pendenza = 1)
- ❑ Multiplicative decrease reduce il throughput proporzionalmente (cioè, maggior diminuzione per la connessione che stava usando più banda)



Meccanismi per il controllo di congestione

- ❑ Il controllo di congestione gestisce l'adattamento della cosiddetta finestra di congestione (congestion window, CWND)
 - ❖ CWND = numero di byte che il trasmettitore può inviare nella rete
- ❑ In TCP ci sono diversi algoritmi per adattare la CWND
- ❑ In assenza di perdite
 - ❖ Slow start
 - ❖ Congestion avoidance
- ❑ Per migliorare l'efficienza di TCP in caso si verificano perdite:
 - ❖ Fast retransmit
 - ❖ Fast recovery
- ❑ **In ogni caso**, $|W_T| = \min(\text{CWND}, \text{RWND}) = \min(\text{CWND}, |W_R|)$

Ricapitoliamo il significato delle finestre

- ❑ Finestra: insieme di dati (byte, o segmenti)
- ❑ Finestra di ricezione, RWND
 - ❖ Insieme di dati che può essere gestito dal ricevitore
 - ❖ La sua dimensione è il limite massimo di dati che il ricevitore può immagazzinare nel buffer di ricezione
- ❑ Finestra di congestione, CWND
 - ❖ Insieme di dati che si possono inviare nella rete
 - ❖ La sua dimensione si adatta alla quantità massima di dati che il trasmettitore pensa di poter inviare senza sovraccaricare la rete
- ❑ Finestra di trasmissione
 - ❖ Insieme di dati che si possono inviare senza sovraccaricare la rete e senza saturare il ricevitore
 - ❖ Proprio per questo, $|W_T| = \min(\text{CWND}, \text{RWND}) = \min(\text{CWND}, |W_R|)$
 - ❖ **D**: da che relazione sono legati i valori $|W_T|$ e $|W_R|$?

TCP: Slow start e congestion avoidance

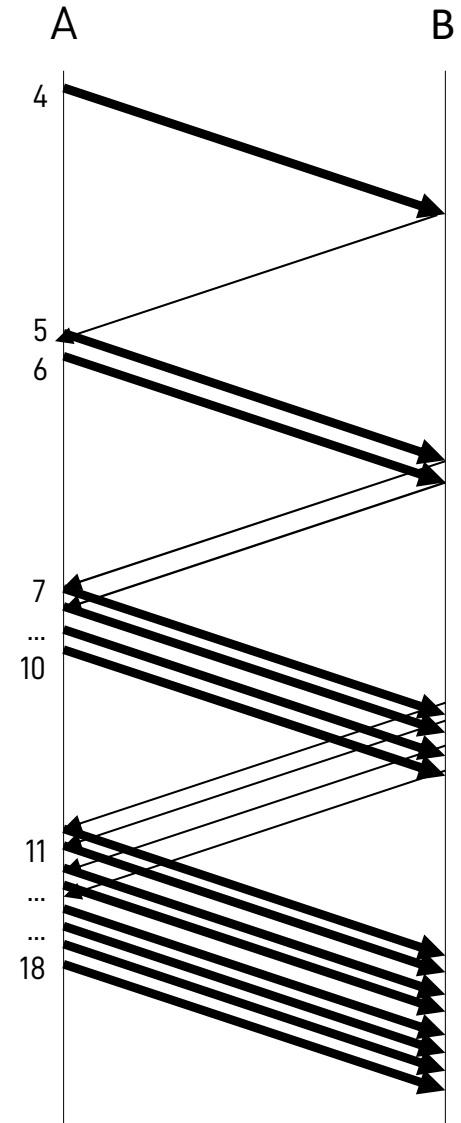
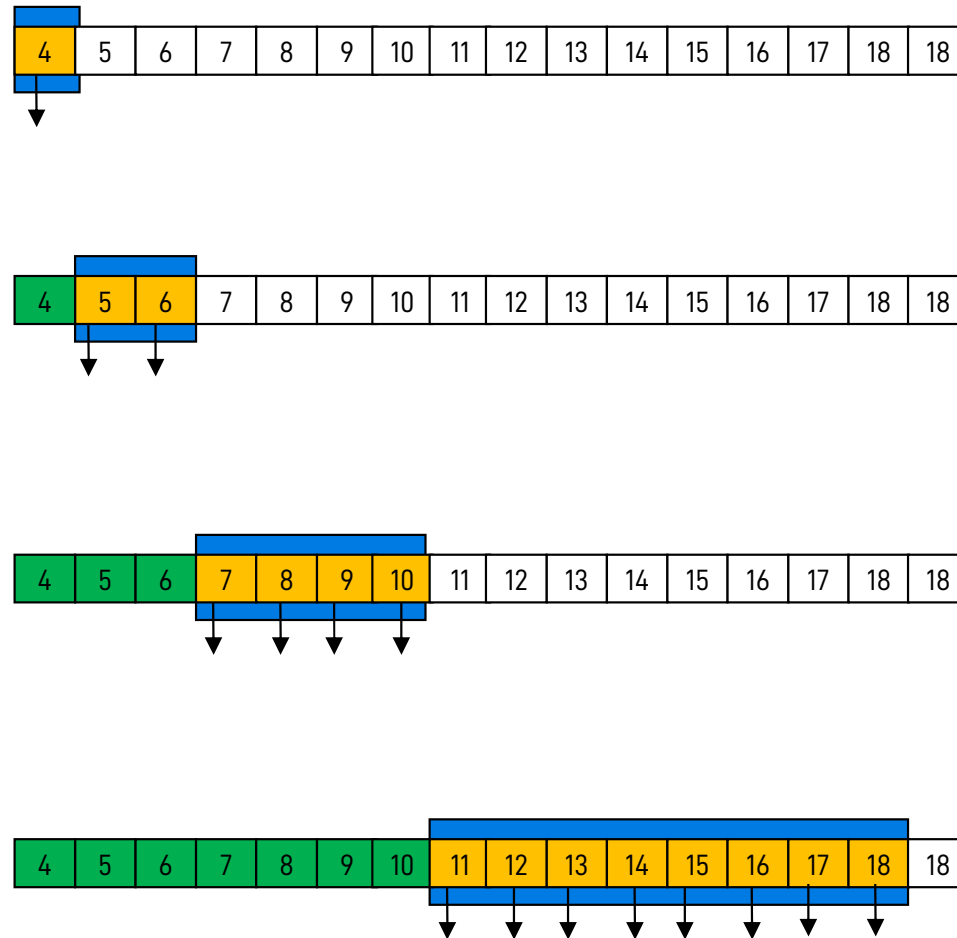
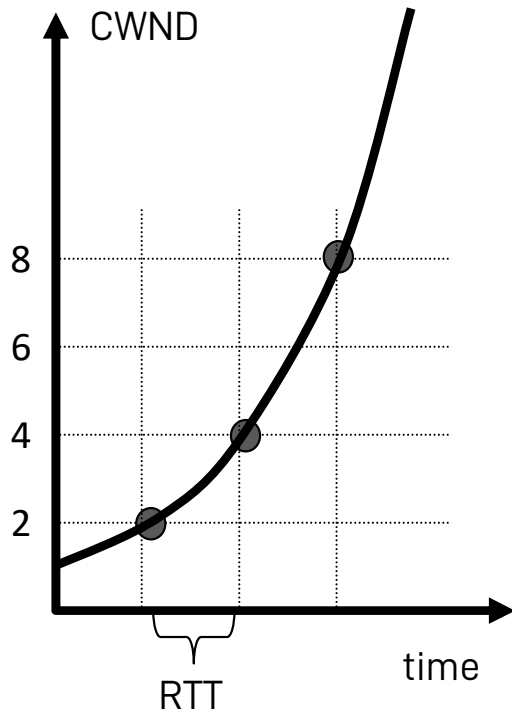
❑ Slow start

- ❖ Per ogni ACK valido ricevuto, aumento CWND di 1 MSS
- ❖ Così, CWND aumenta esponenzialmente nel tempo
 - Inizio con CWND = 1 MSS
 - Ricevo 1 ACK dopo 1 RTT $\rightarrow$ CWND = 2 MSS
 - Ricevo 2 ACK dopo 1 RTT $\rightarrow$ CWND = 4 MSS
- ❖ Quando CWND raggiunge o supera una soglia (slow start threshold, SSTHRESH) $\rightarrow$ passa alla modalità congestion avoidance

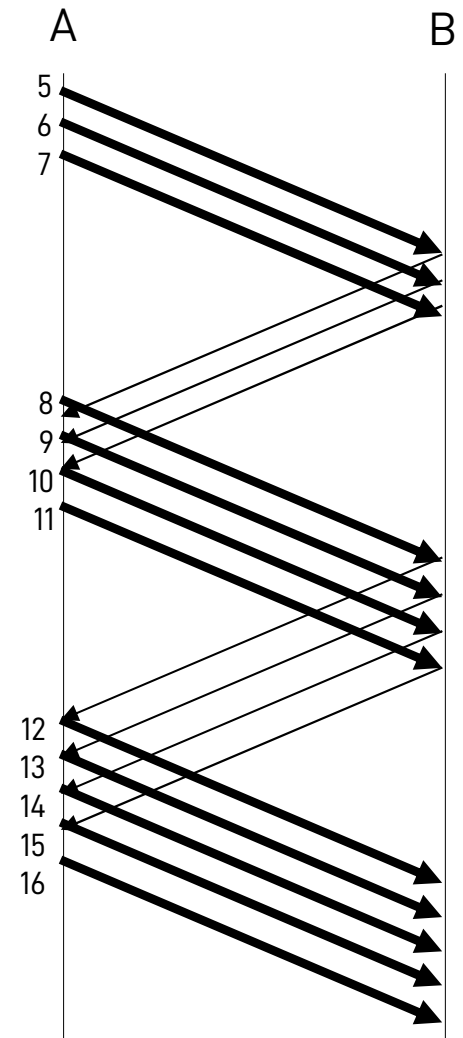
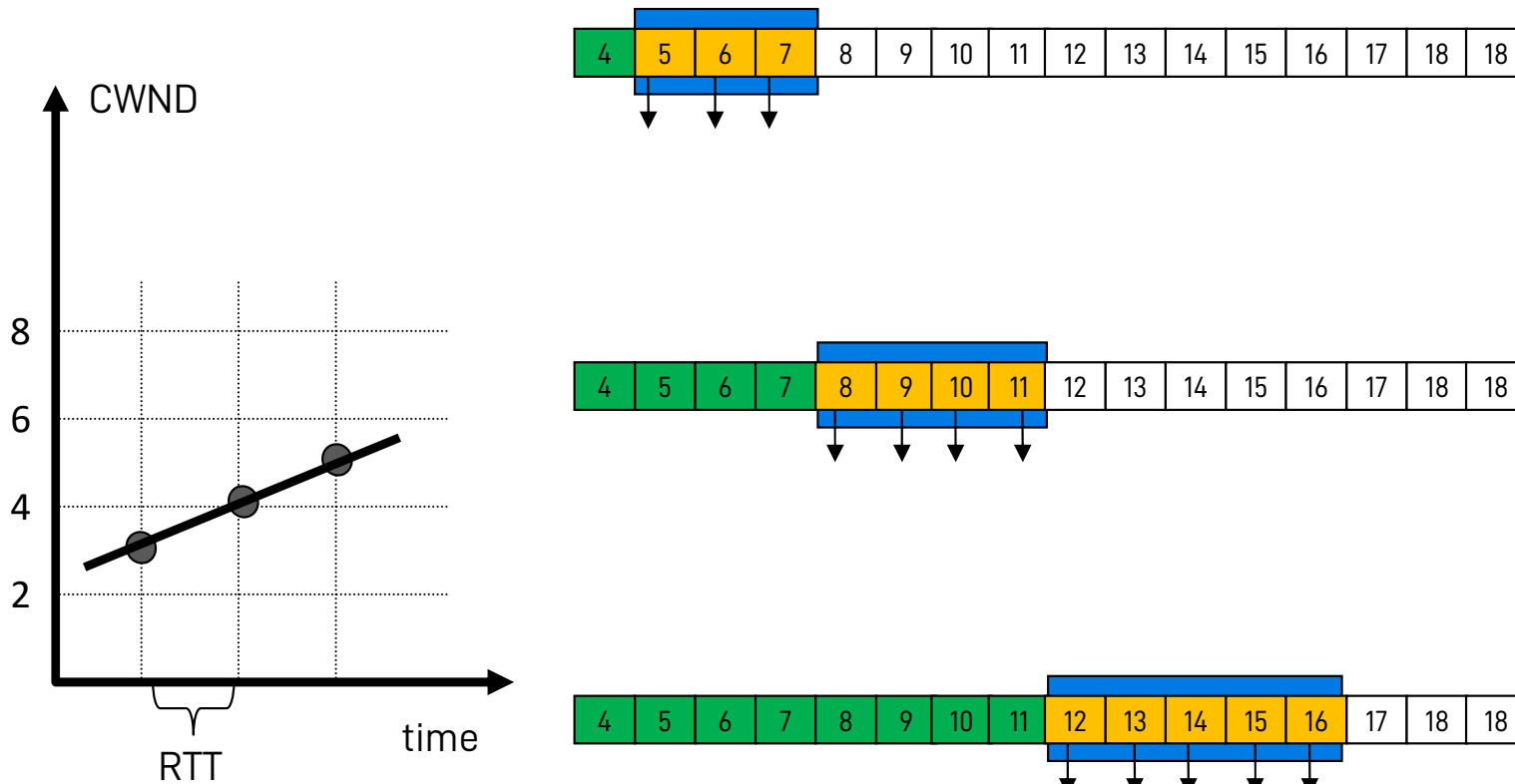
❑ Congestion avoidance

- ❖ Per ogni ACK valido ricevuto, aumento la CWND di $MSS * \frac{MSS}{CWND}$
 - 0, se ragioniamo in segmenti, aumento la CWND di $\frac{1}{CWND}$ segmenti
- ❖ In altre parole, per ogni RTT in cui ricevo esattamente tutti gli ACK attesi (in totale CWND ACK), aumento CWND di 1 MSS (1 segmento)
- ❖ Aumento lineare di CWND nel tempo

Esempio di slow start



Esempio di congestion avoidance



Parametri su cui potete agire



❑ Finestra di congestione (CWND)

- ❖ Aumentandola, vi fidate della capacità della rete di gestire il traffico



❑ Slow start threshold (SSTHRESH)

- ❖ Diminuendola, riducete la durata della fase di slow start, passate più rapidamente a congestion avoidance, e favorite un approccio più prudente



❑ Retransmission timeout (RTO)

- ❖ Aumentandolo, date più tempo alla rete di consegnare i segmenti e riducete il traffico delle ritrasmissioni



❑ W_{LOW} e W_{UP}

- ❖ Potete ritardarne lo spostamento (ad esempio per tenere memoria di pacchetti per i quali non avere ricevuto ACK)

Algoritmo della fase Slow start

❑ Inizializzazione:

- ❖ $CWND = 1 \text{ MSS}$ (o semplicemente 1 segmento)
- ❖ $SSTHRESH = RWND$ (o $RWND/2$ a seconda delle implementazioni)

❑ Se ricevo un ACK valido:

- ❖ $CWND = CWND + 1 \text{ MSS}$
- ❖ Sposto W_{LOW} al primo byte (o segmento) non ACKed
- ❖ Se $CWND \geq SSTHRESH \rightarrow$ passo alla fase Congestion Avoidance
- ❖ Trasmetto nuovi segmenti come consentito da $CWND$

❑ Se scatta un timeout:

- ❖ Abbasso $SSTHRESH = \text{MAX}(CWND / 2, 2)$
- ❖ Aumento $RTO = RTO * 2$ (questo passaggio manca nel libro)
- ❖ Reimposto $CWND = 1$
- ❖ Ritrasmetto il segmento che ha causato il timeout

Algoritmo della fase Congestion avoidance

❑ Se ricevo un ACK valido:

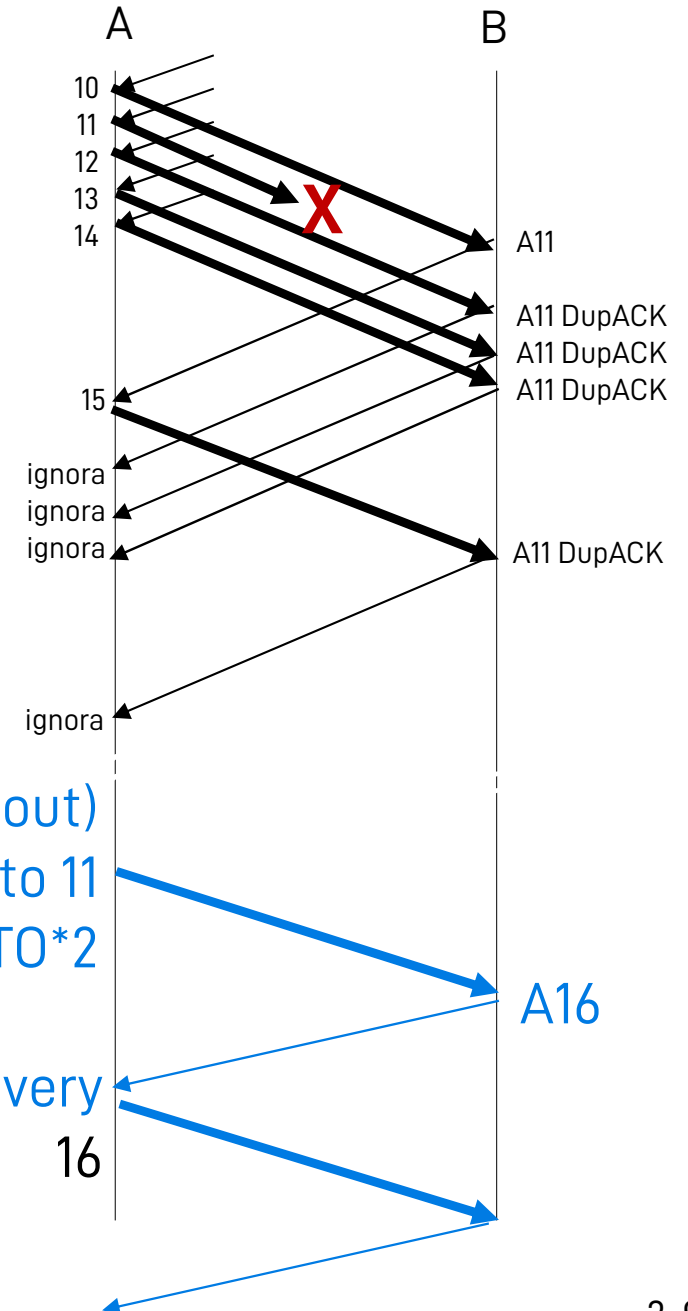
- ❖ $CWND = CWND + MSS * \frac{MSS}{CWND}$ (ricordate: valori in byte!)
- ❖ Sposto W_{LOW} al primo segmento non ACKed
- ❖ Trasmetto nuovi segmenti come consentito da CWND

❑ Se scatta un timeout:

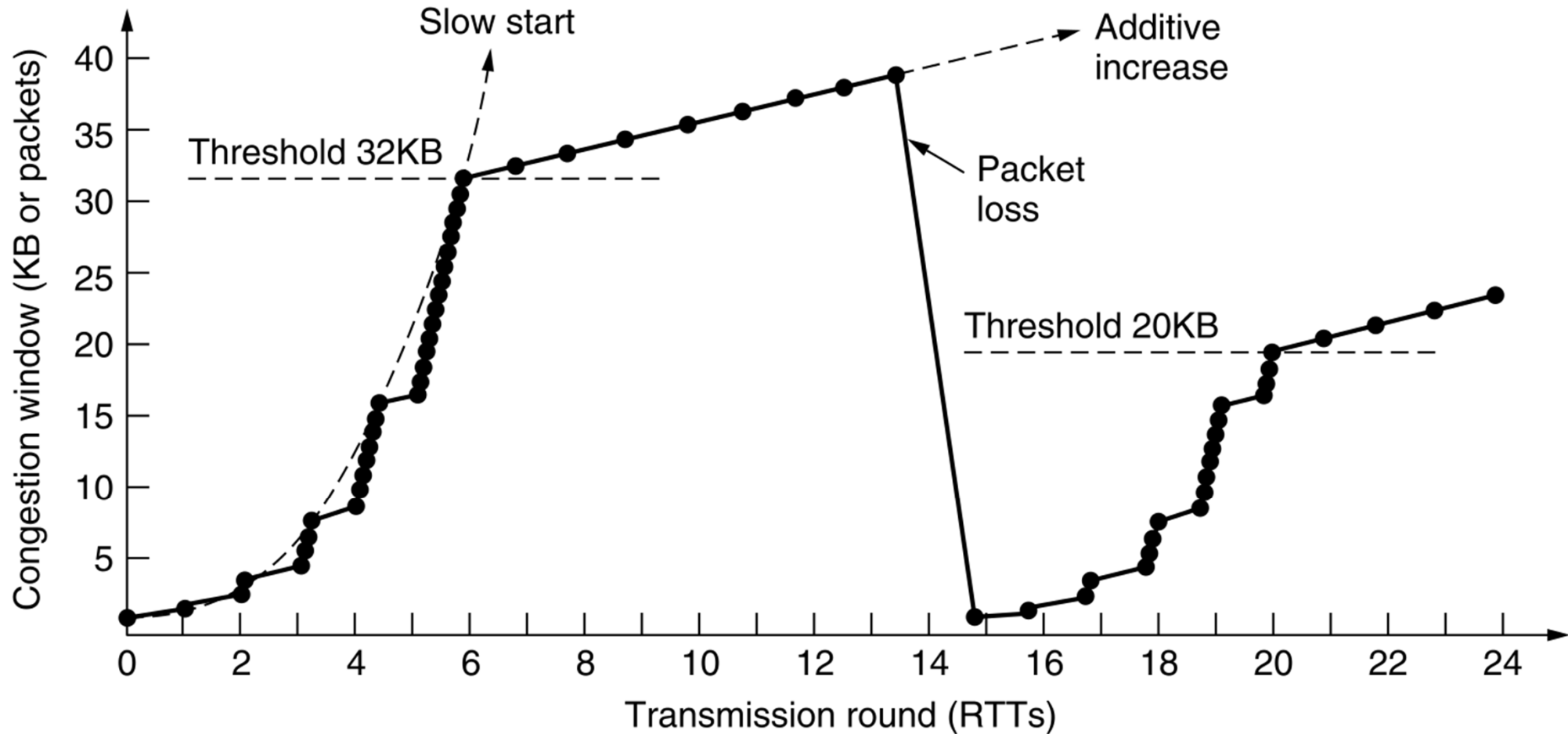
- ❖ Passo a slow start
- ❖ Abbasso $SSTHRESH = MAX(CWND / 2, 2)$
- ❖ Aumento $RTO = RTO * 2$ (questo passaggio manca nel libro)
- ❖ Reimposto $CWND = 1$
- ❖ Ritrasmetto il segmento che ha causato il timeout

RT0 è un buon indicatore di errori?

- ❑ Supponiamo che in questo scenario siamo in congestion avoidance
 - ❖ $CWND = 5, W_T = [10, \dots, 14]$
- ❑ Dopo il RTT per 11
 - ❖ Ritrasmetto 11
 - ❖ $CWND = 1$
- ❑ Dopo ACK 16
 - ❖ $CWND = 1, W_T = [16]$
- ❑ Abbiamo ignorato la sequenza di ACK duplicati
 - ❖ Ci stavano dicendo qualcosa?



Prestazioni di questa versione di TCP



Fast retransmit ...

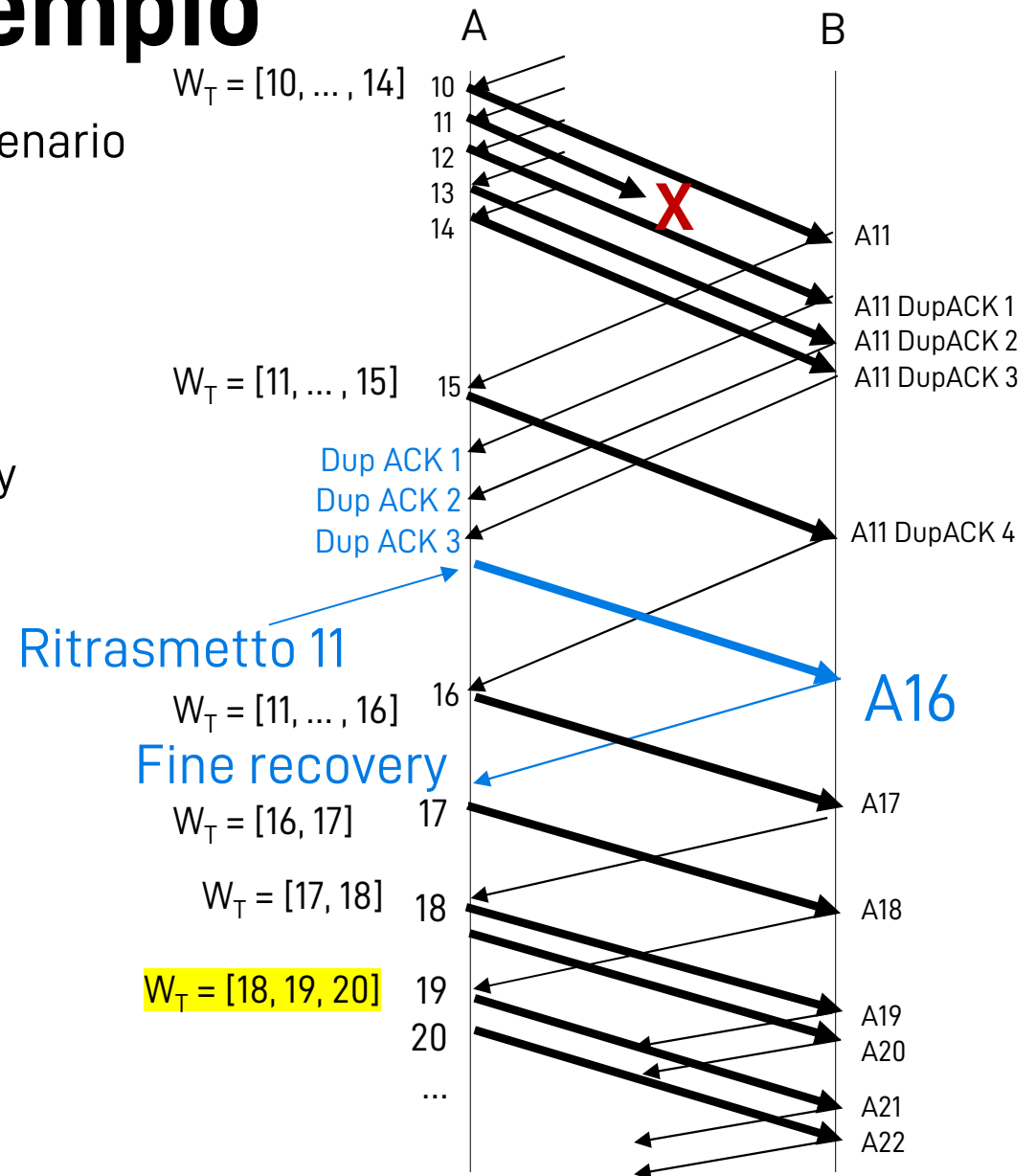
- ❑ Gli ACK duplicati, in fondo, ci dicono che la rete funziona
- ❑ Idea di base del fast recovery:
 - ❖ Diamo fiducia alla rete: se funziona, continuiamo a trasmettere
- ❑ Alla ricezione del 3° ACK duplicato
 - ❖ Ritrasmetto il segmento indicato dall'ACK ("fast retransmit")
 - ❖ Entro in una fase di "fast recovery"
- ❑ Mi ricordo il valore di W_{UP} per sapere quanti segmenti sono "in volo"
 - ❖ $RECOVER = W_{UP}$
 - ❖ Questo mi permetterà di capire quando la fase è completata

... e Fast recovery

- ❑ Alla ricezione del 3° ACK duplicato
 - ❖ Abbasso $SSTHRESH = CWND / 2$
 - ❖ Suppongo di aver perso solo quel segmento e imposto $CWND = SSTHRESH + 3 MSS$
 - Se quindi $CWND$ me lo permette, trasmetto altri segmenti
 - ❖ NON SPOSTO il puntatore W_{LOW}
- ❑ Se arrivano altri ACK duplicati
 - ❖ $CWND = CWND + 1 MSS$
 - Se $CWND$ me lo permette, trasmetto altri segmenti
 - ❖ NON SPOSTO il puntatore W_{LOW}
- ❑ Quando arriva un ACK valido (che include un riscontro per il segmento *RECOVER*)
 - ❖ $CWND = SSTHRESH$
 - ❖ Passo alla fase Congestion avoidance
 - ❖ Sposto W_{LOW} al primo segmento non ACKed
- ❑ Se arriva un ACK parziale (conferma un segmento precedente a *RECOVER*)
 - ❖ Ritrasmetto il primo segmento per cui non ho un ACK
 - ❖ Riduco $CWND = CWND - \text{numero di segmenti ACKed} + 1$
 - ❖ Sposto W_{LOW} al primo segmento non ACKed

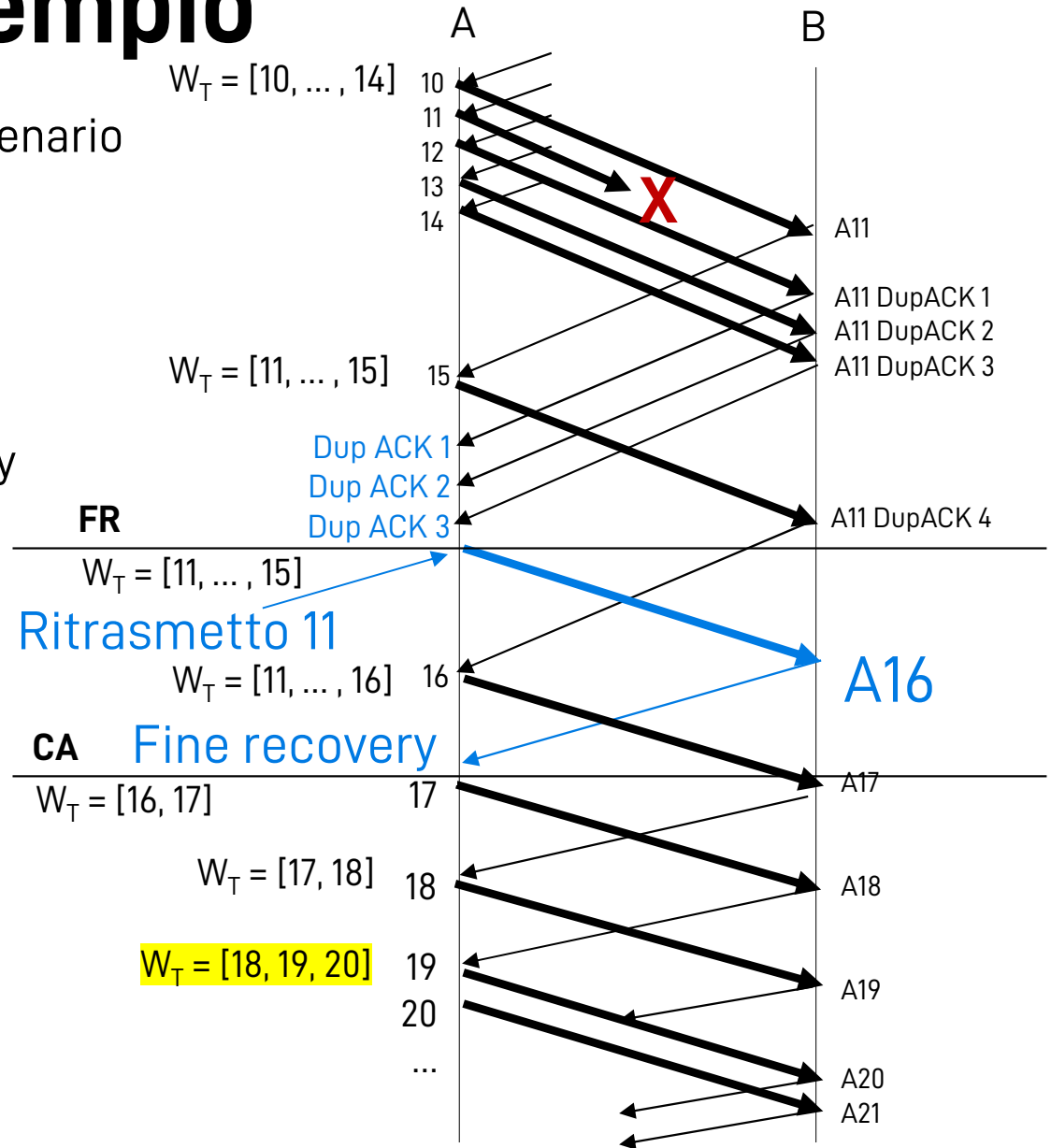
Fast recovery: esempio

- ❑ Supponiamo ancora che in questo scenario siamo in congestion avoidance
 - ❖ $CWND = 5, W_T = [10, \dots, 14]$
- ❑ Dopo ACK 11
 - ❖ $W_T = [11, \dots, 15]$: invio 15
- ❑ Dopo 3 ACK11 duplicati: Fast Recovery
 - ❖ $SSTHRESH = CWND / 2 = 2$
 - ❖ $CWND = SSTHRESH + 3 = 5$
 - ❖ $W_T = [11, \dots, 15]$
- ❑ Dopo il 4° ACK 11 duplicato
 - ❖ $CWND = CWND + 1$
 - ❖ $W_T = [11, \dots, 16]$: trasmetto 16
- ❑ Dopo ACK 16
 - ❖ $CWND = SSTHRESH = 2$
 - ❖ Passo a congestion avoidance
 - ❖ $W_T = [16, 17]$: trasmetto 17

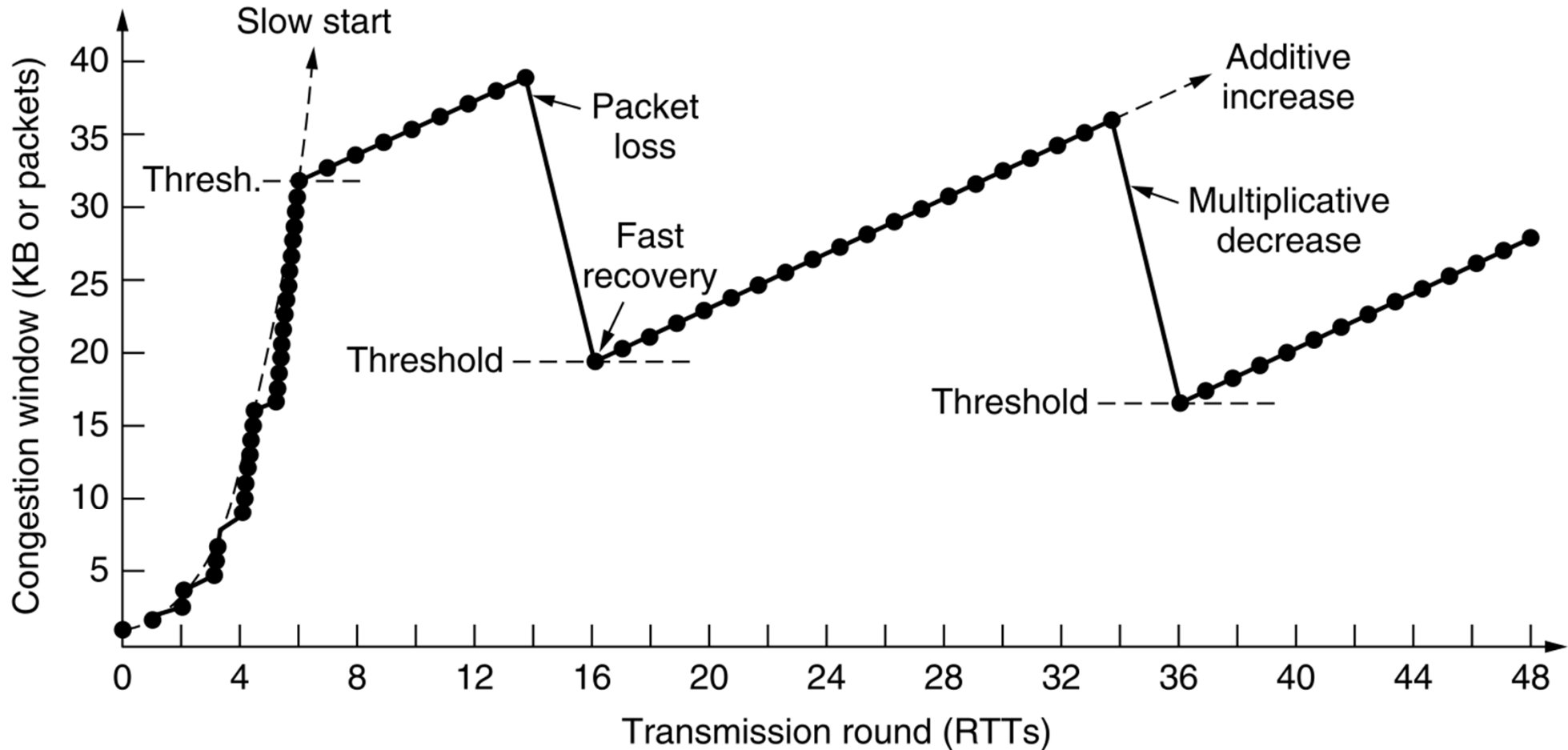


Fast recovery: esempio

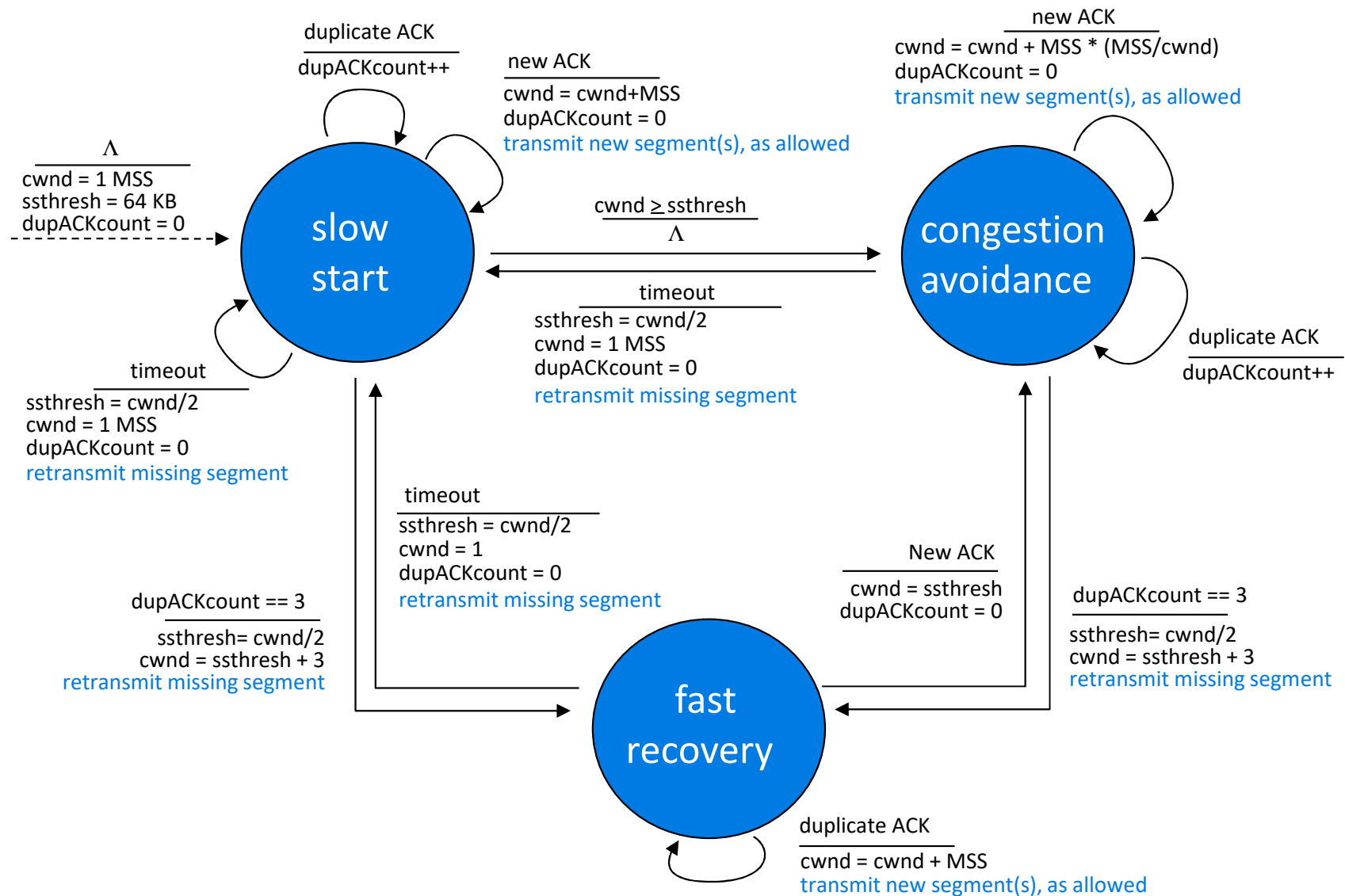
- ❑ Supponiamo ancora che in questo scenario siamo in congestion avoidance
 - ❖ $CWND = 5, W_T = [10, \dots, 14]$
- ❑ Dopo ACK 11
 - ❖ $W_T = [11, \dots, 15]$: invio 15
- ❑ Dopo 3 ACK11 duplicati: Fast Recovery
 - ❖ $SSTHRESH = CWND / 2 = 2$
 - ❖ $CWND = SSTHRESH + 3 = 5$
 - ❖ $W_T = [11, \dots, 15]$
- ❑ Dopo il 4° ACK 11 duplicato
 - ❖ $CWND = CWND + 1$
 - ❖ $W_T = [11, \dots, 16]$: trasmetto 16
- ❑ Dopo ACK 16
 - ❖ $CWND = SSTHRESH = 2$
 - ❖ Passo a congestion avoidance
 - ❖ $W_T = [16, 17]$: trasmetto 17



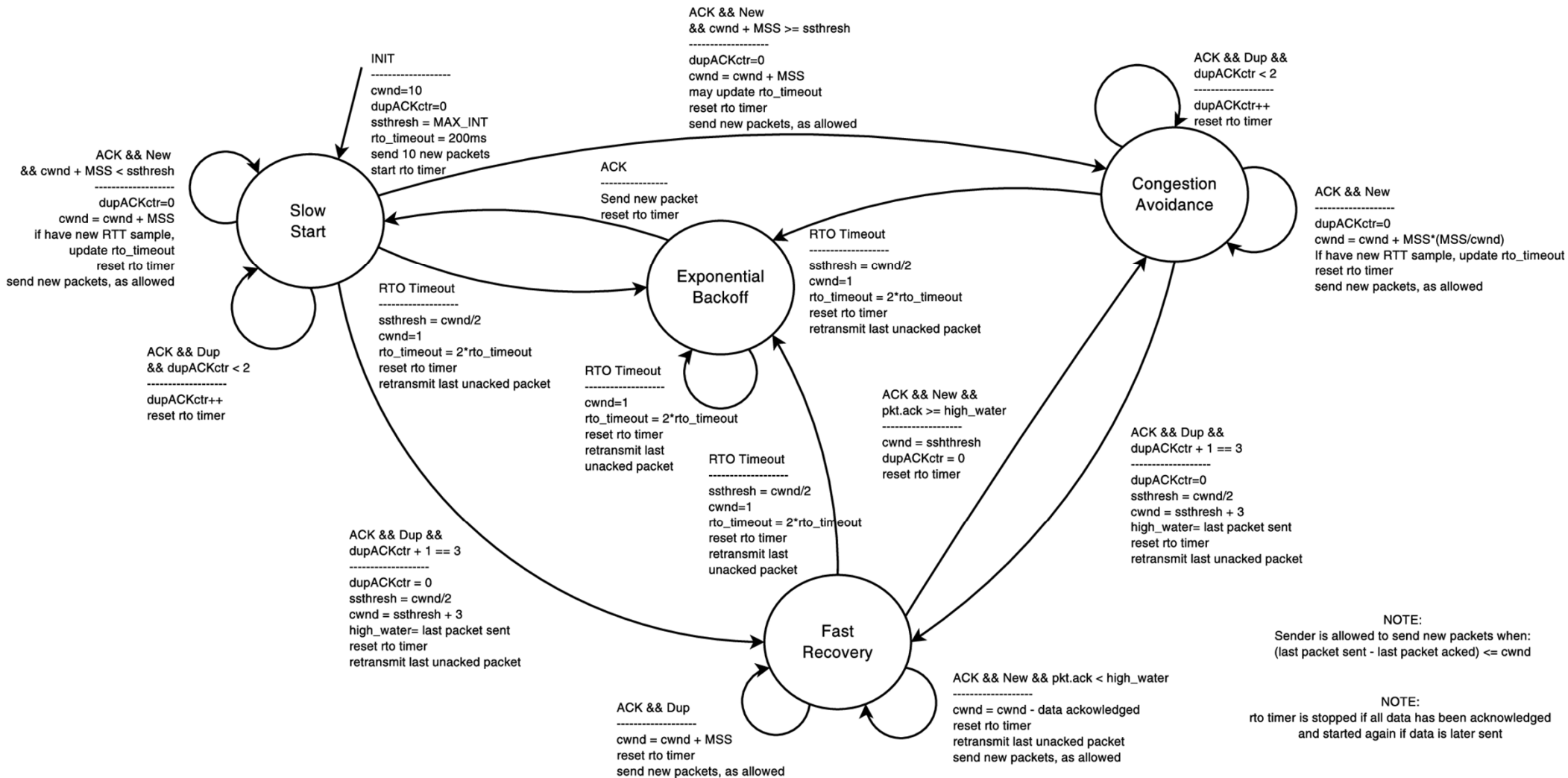
TCP fast retransmit e fast recovery



Macchina a stati dell'algoritmo di controllo di congestione in TCP



Un'altra macchina a stati TCP



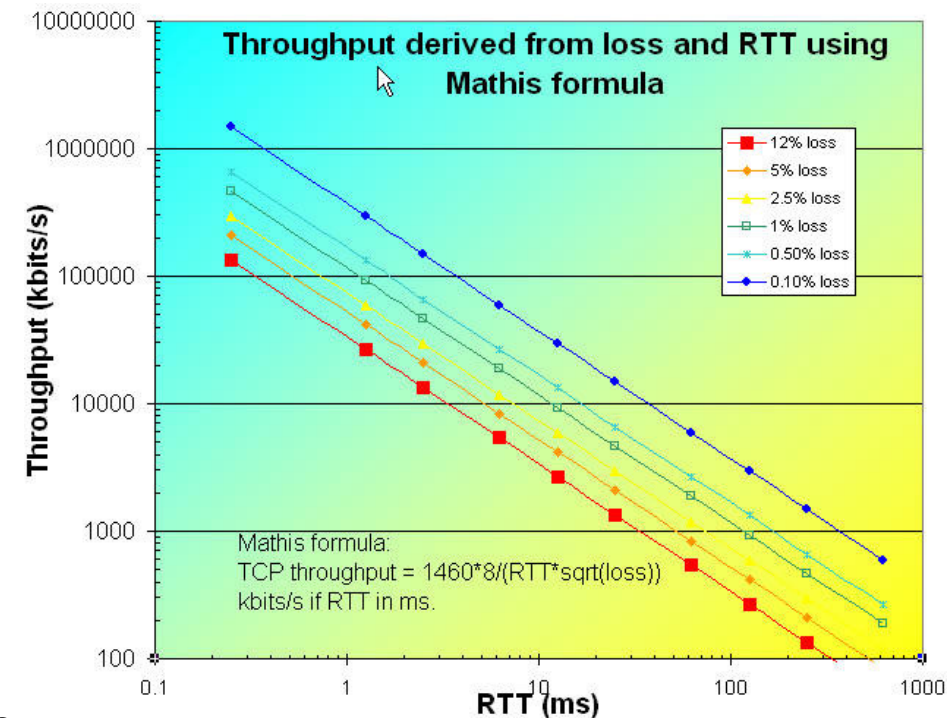
Source: S. Jero, et al. "Automated attack discovery in TCP congestion control using a model-guided approach." Proc. of Network and Distributed System Security Symp., San Diego, CA, USA. 2018.

Formula approssimata per il throughput di TCP

- From Mathis et al., "The macroscopic behavior of the TCP congestion avoidance algorithm", 1997

$$\text{❖ } \text{Thr}(RTT, p) \text{ [Byte/s]} < \frac{MSS}{RTT} \cdot \frac{1}{\sqrt{p}}$$

- ❖ p = Probabilità di perdere un segmento
- ❖ MSS = maximum segment size [Byte]
- ❖ RTT = round-trip time [s]



- Es.: $RTT = 220 \text{ ms}$, $MSS = 1460 \text{ Byte}$, $p = 10^{-3} \rightarrow \text{THR} = 1.68 \text{ Mbit/s}$

Problemi di equità (fairness) in TCP

Fairness e UDP

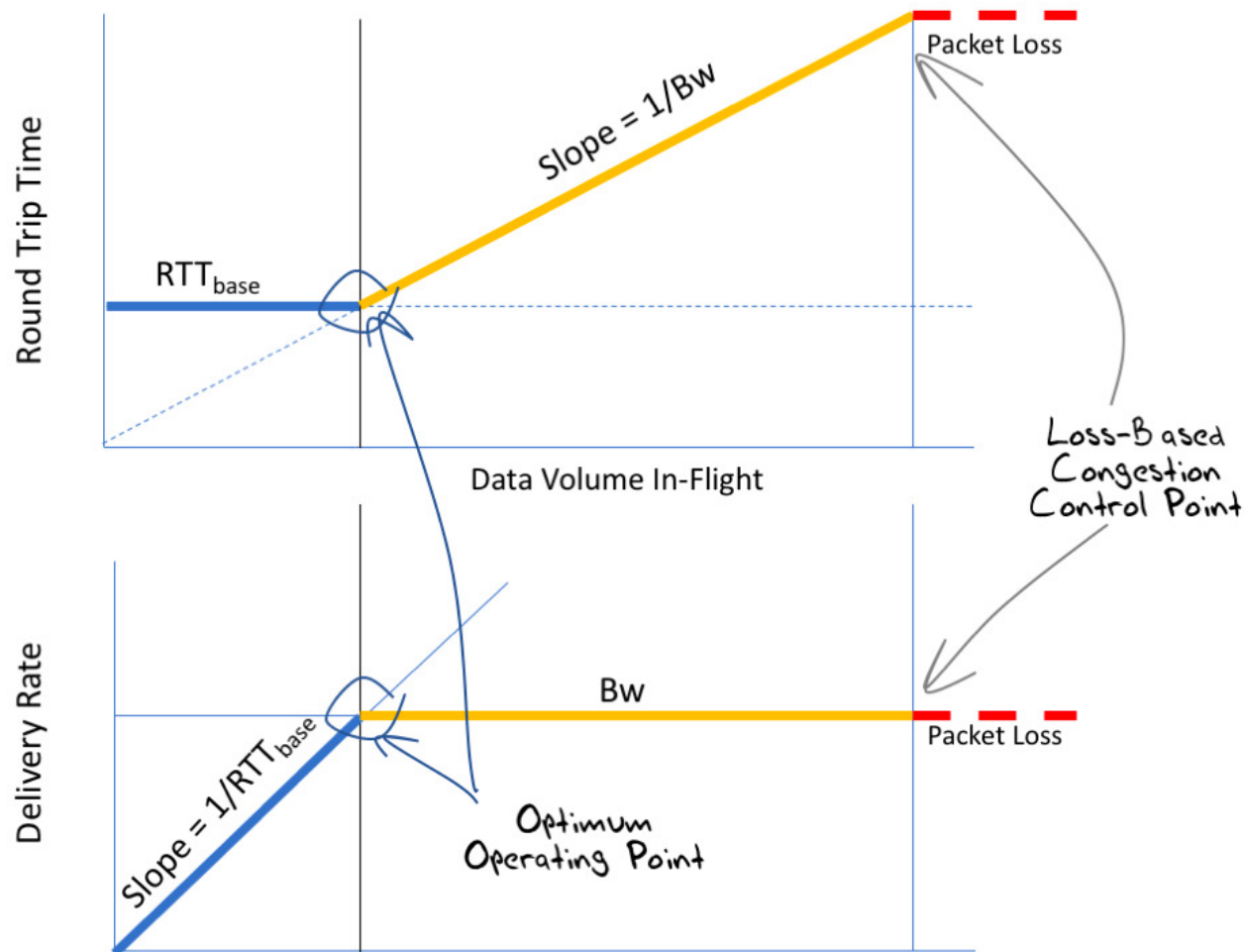
- ❑ Le applicazioni multimediali non usano TCP quasi mai
 - ❖ Non desiderano ridurre il tasso di trasmissione per via del controllo di congestione
- ❑ Invece usano UDP:
 - ❖ Inviano audio e video a un tasso costante
 - ❖ Tollerano o diversamente compensano le perdite di pacchetti

Fairness con connessioni TCP parallele

- ❑ Le applicazioni possono aprire più connessioni in parallelo tra due host
 - ❖ I web browser lo fanno
- ❑ Es.: link con banda R e 9 connessioni TCP esistenti
 - ❖ Una nuova applicazione apre 1 TCP, ottiene una banda $R/10$
 - ❖ Una nuova applicazione apre 9 TCP, ottiene una banda $R/2$

Futuro di TCP

- ❑ La versione di TCP che abbiamo studiato è loss-based
 - ❖ Ma quando avviene una perdita è già «tardi» per reagire



TCP e controllo di congestione

- ❑ Il controllo della congestione di TCP stabilisce di rallentare la trasmissione basandosi solo sulle indicazioni dei pacchetti persi
- ❑ Questo ha funzionato bene per molti anni
 - ❖ I piccoli buffer degli switch Internet e dei router erano ben adattati alla bassa larghezza di banda dei collegamenti Internet
- ❑ Ad oggi abbiamo bisogno di un algoritmo che risponda alla congestione effettiva, piuttosto che alla perdita di pacchetti
- ❑ Alcune soluzioni:
 - ❖ CUBIC
 - ❖ BBR
 - ❖ QUIC

CUBIC

- ❑ Algoritmo di controllo della congestione di rete in TCP
- ❑ In particolare, fa variare la lunghezza della finestra di congestione secondo una funzione cubica del tempo
 - ❖ Migliora la scalabilità e la stabilità su reti veloci e a lunga distanza
- ❑ Utilizzato da anni come impostazione predefinita nel kernel Linux
 - ❖ Windows lo ha adottato solo nel 2017

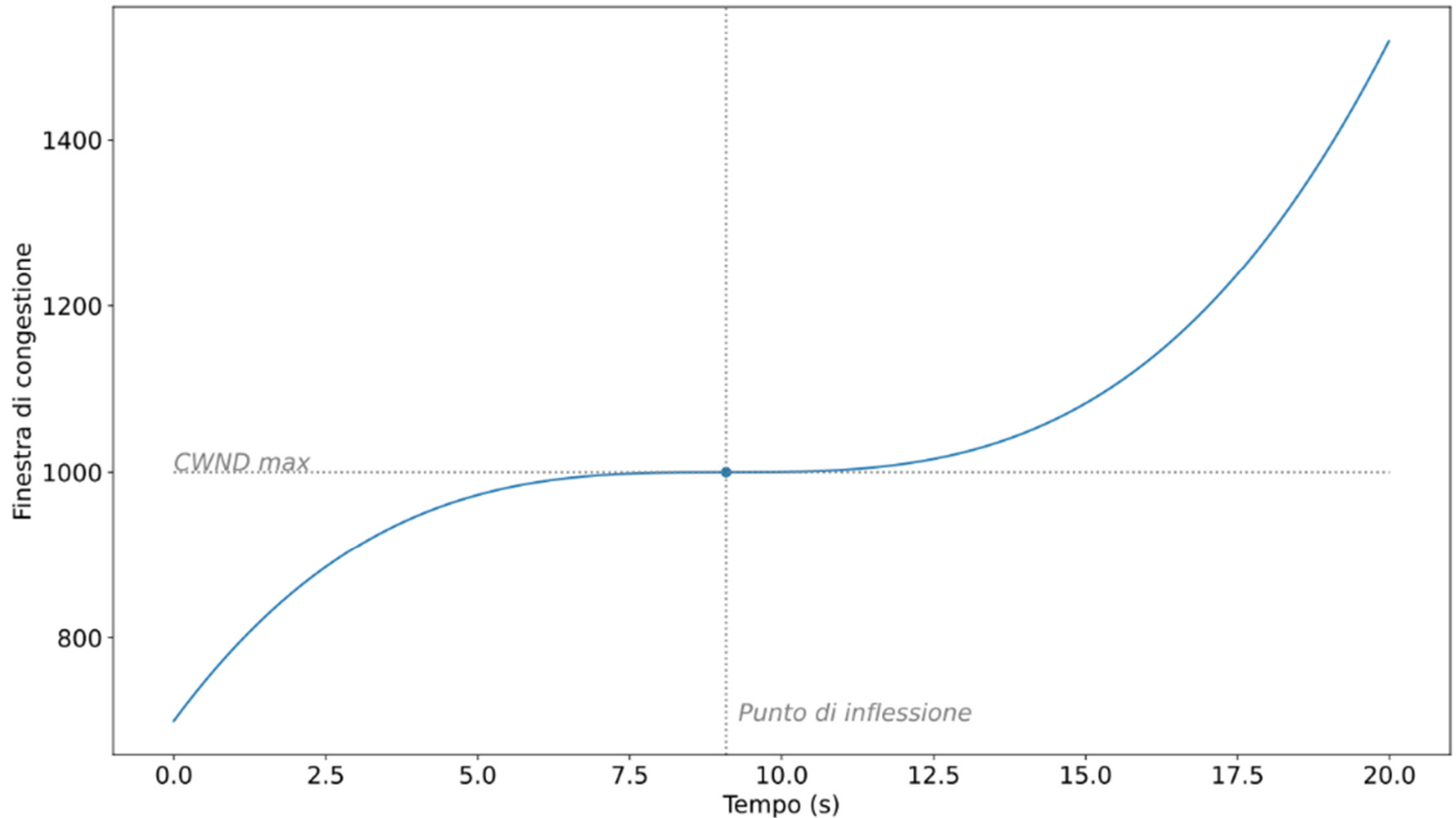
Principi di CUBIC

- ❑ Per un migliore utilizzo e stabilità della rete, CUBIC usa entrambi i profili concavo e convesso di una funzione cubica per aumentare la dimensione della finestra di congestione

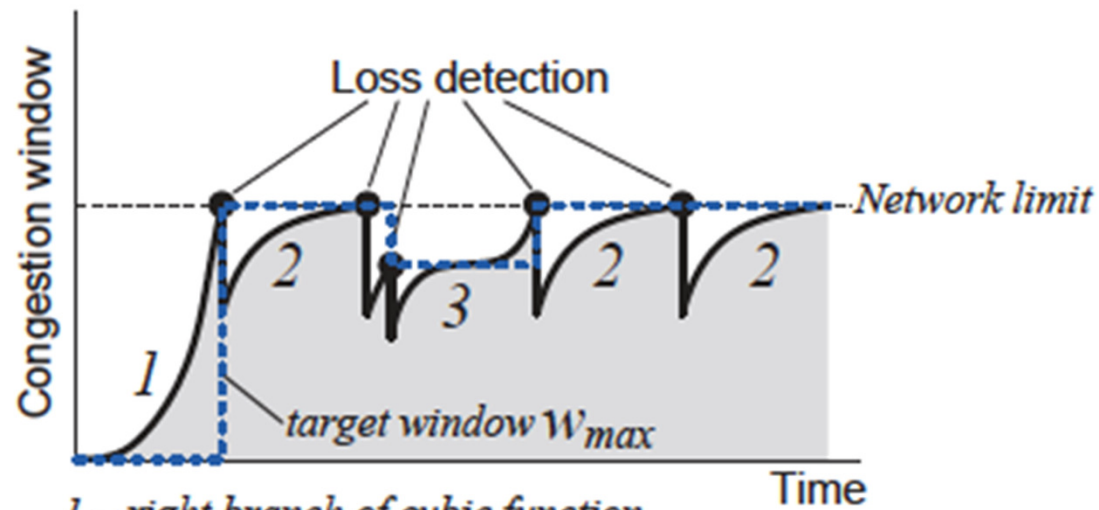
$$CWND_{cubic}(t) = C(t - K)^3 + CWND_{max}$$

- ❑ Dove $K = \sqrt[3]{\frac{CWND_{max}(1 - \beta)}{C}}$ e, da RFC 8312, $C = 0.4$ e $\beta = 0.7$
- ❑ Dopo una congestione, la finestra è scalata di β , non di 0.5
 - ❖ CUBIC bilancia scalabilità e velocità di convergenza
- ❑ TCP-friendly: non penalizza troppo i flussi TCP standard che condividono lo stesso bottleneck link
 - ❖ <https://www.cs.princeton.edu/courses/archive/fall16/cos561/papers/Cubic08.pdf>

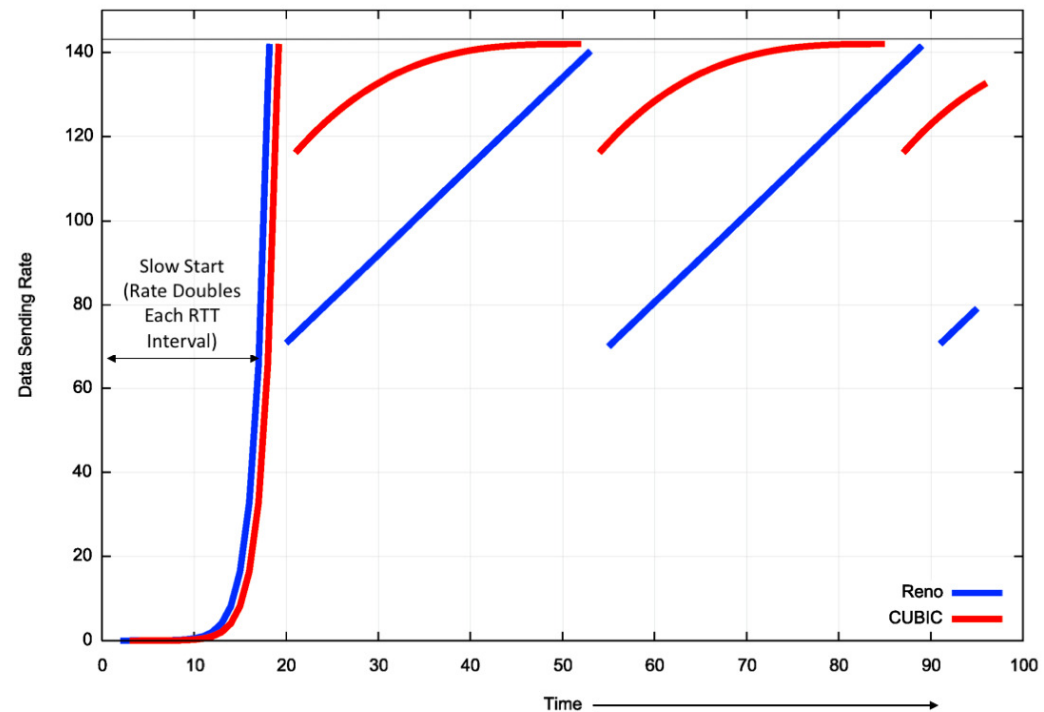
Funzione cubica per la CWND in CUBIC



CUBIC

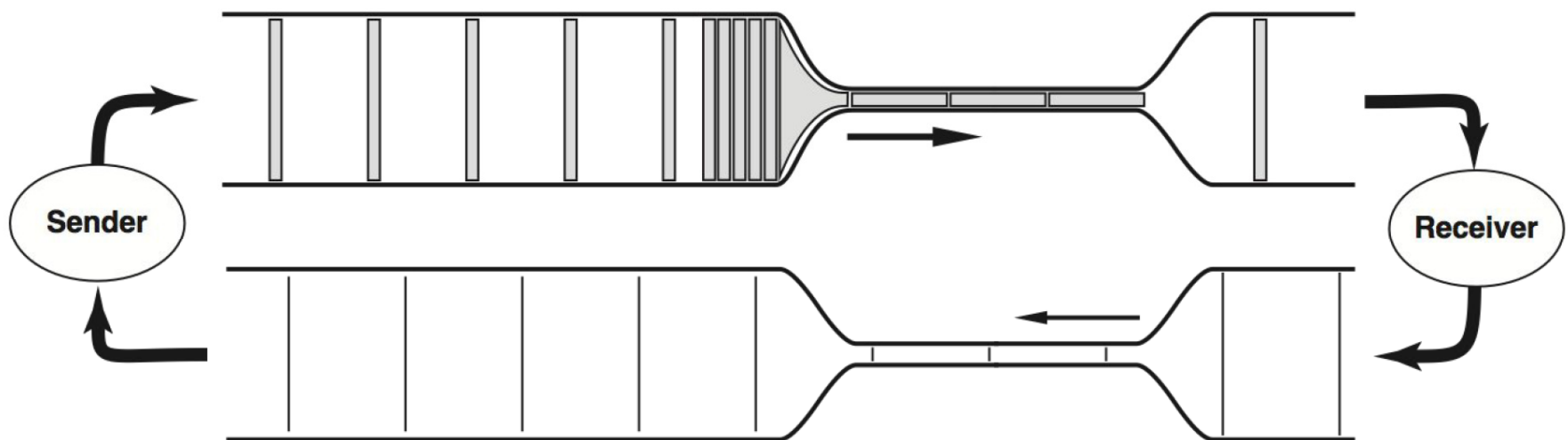


- 1 – right branch of cubic function
- 2 – left branch of cubic function
- 3 – left and right branches of cubic function



BBR

- ❑ Bottleneck Bandwidth and Roundtrip propagation time (BBR)
- ❑ Algoritmo per il controllo della congestione (Google, 2016)
- ❑ BBR non si basa più sulle perdite ma stima due parametri:
 - ❖ La banda sul link «collo di bottiglia» (bottleneck bandwidth)
 - ❖ L'RTT
- ❑ Idea: trasmettere pacchetti a una velocità che non «dovrebbe» incontrare accodamenti

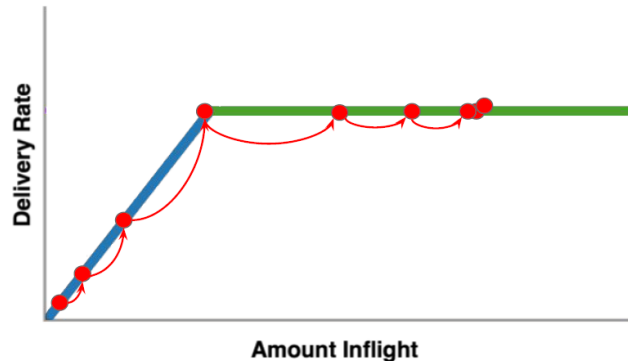


BBR

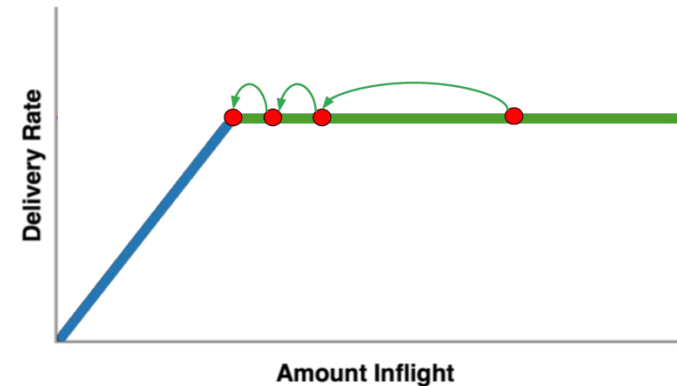
- ❑ Progettato per rispondere alla congestione effettiva, piuttosto che alla perdita di pacchetti
 - ❖ BBR modella la rete per inviare alla velocità della larghezza di banda disponibile
- ❑ Incentrato sul miglioramento delle prestazioni della rete quando la rete non è molto buona
- ❑ Server-side algorithm!
 - ❖ Non richiede al client di implementare BBR
- ❑ Concetto di *pacing*: invece di inserire nella CWND (e inviare) tutti i pacchetti consentiti, li inserisco al ritmo al quale può inviarli il nodo più lento (a monte del bottleneck link)

Ricerca del punto ottimale

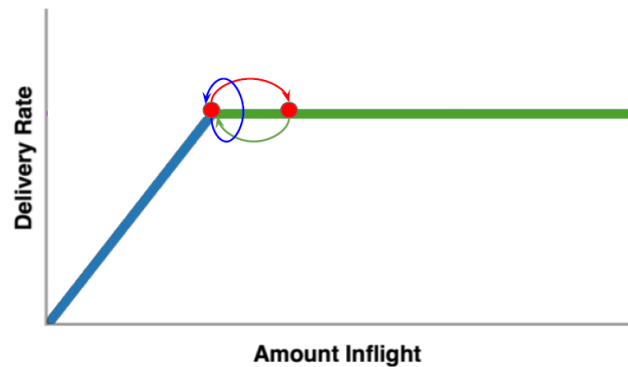
STARTUP: exponential BW search



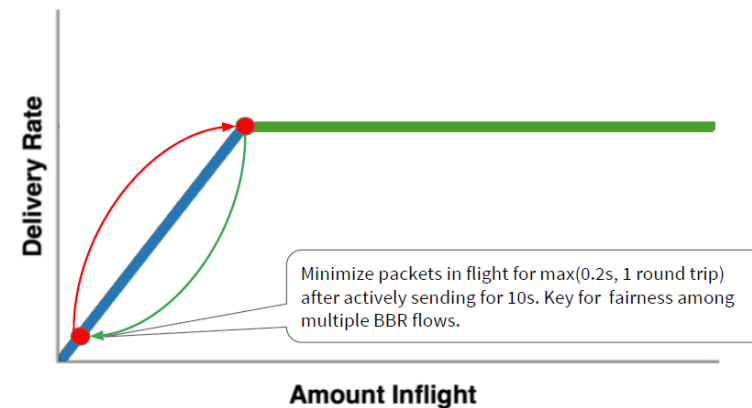
DRAIN: drain the queue created during startup



PROBE_BW: explore max BW, drain queue, cruise

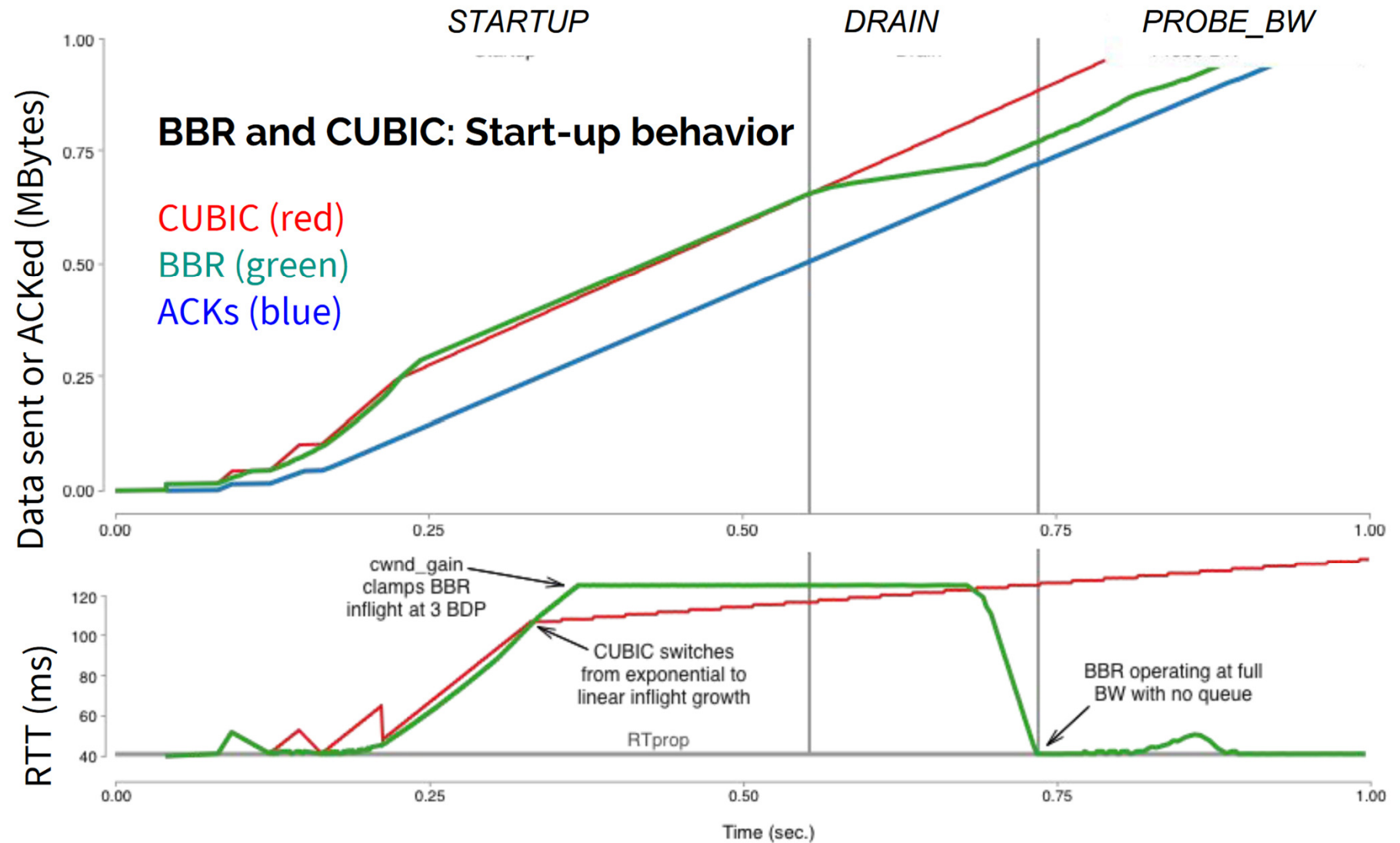


PROBE_RTT drains queue to refresh min_RTT



(effettuato cambiando il «pacing gain»)

Performance



Maggiore produttività

- ❑ Secondo Google, in un server con un collegamento Ethernet a 10 Gigabit che invia dati lungo un percorso con un RTT di 100 ms e tasso di perdita di pacchetti dell'1% si ha come throughput:
 - ❖ 3,3 Mbit/s con CUBIC
 - ❖ 9100 Mbit/s con BBR

- ❑ Ottimo insieme ad HTTP/2, che usa una singola connessione
 - ❖ Il risultato finale è un traffico più veloce sulle odierne dorsali ad alta velocità e una larghezza di banda notevolmente aumentata e tempi di download ridotti

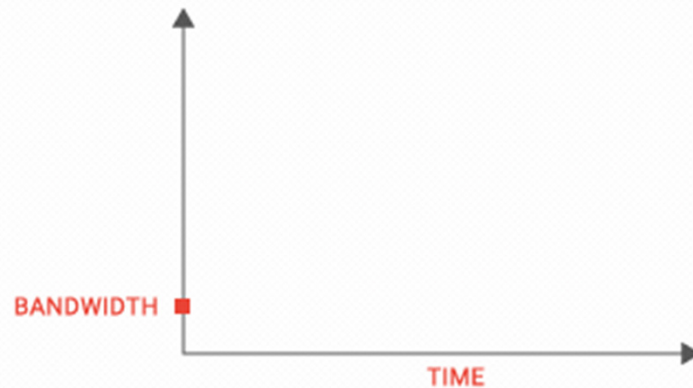
Latenza inferiore

- ❑ BBR consente riduzioni significative della latenza nelle reti dell'ultimo miglio che connettono gli utenti a Internet
- ❑ Consideriamo un collegamento con 10 Mbps, un tempo di andata e ritorno di 40 ms e un tipico buffer di collo di bottiglia di 1000 pacchetti
 - ❖ CUBIC ha un tempo di andata e ritorno medio di 1090 ms
 - ❖ BBR di solo 43 ms

BBR: Velocità di trasmissione

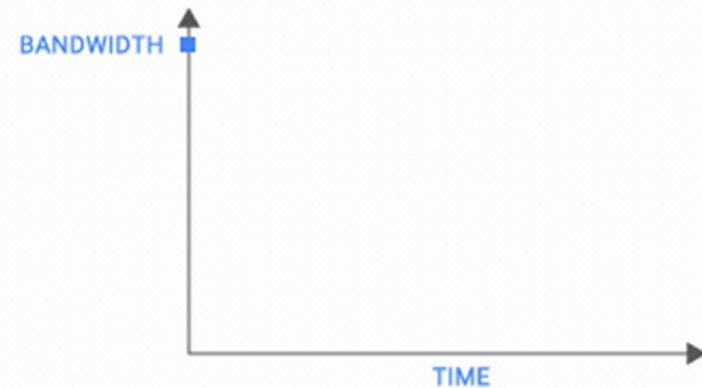
TCP before BBR

Today's Internet is not moving data as well as it should. TCP sends data at lower bandwidth because the 1980s-era algorithm assumes that packet loss means network congestion.

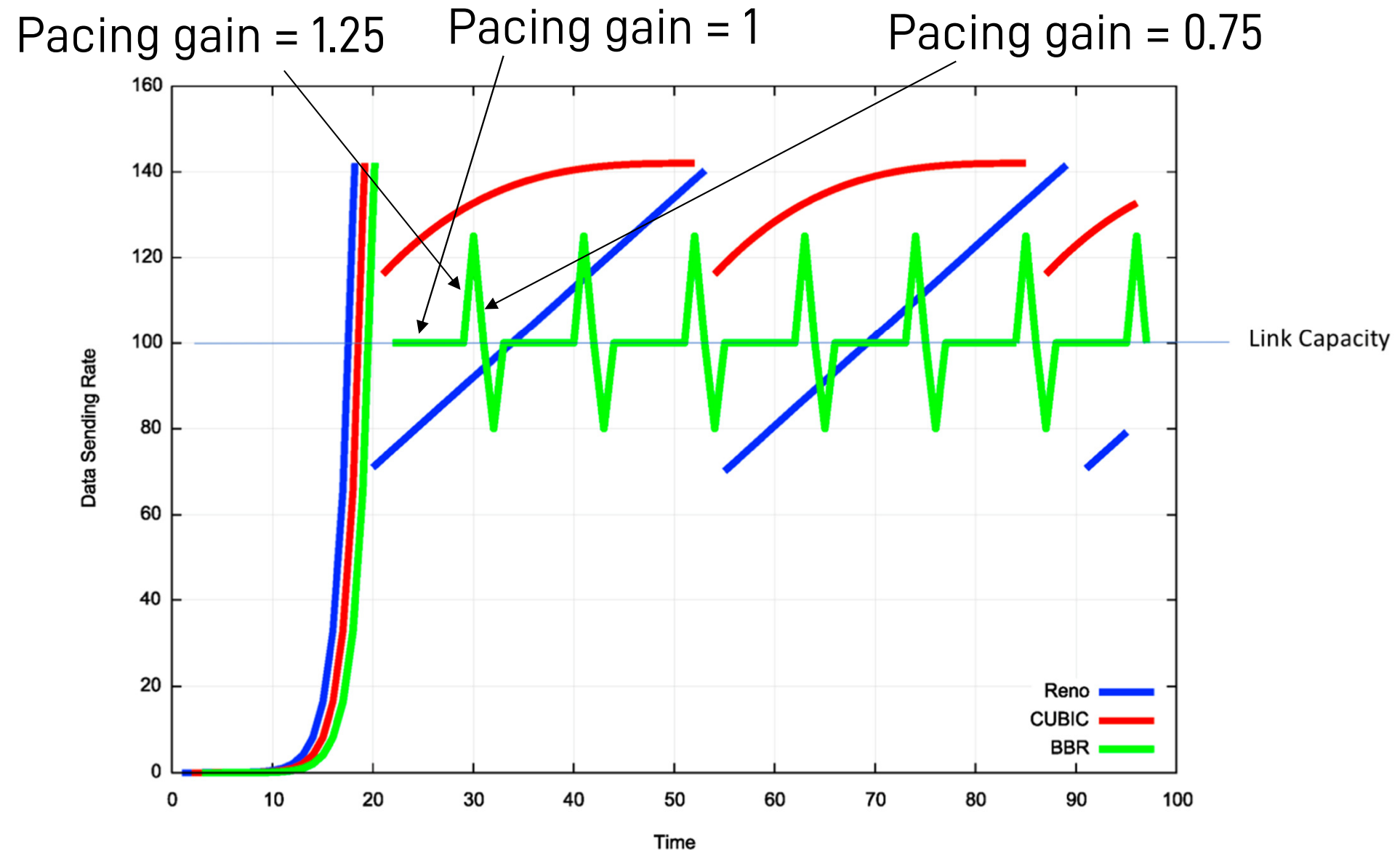


TCP BBR

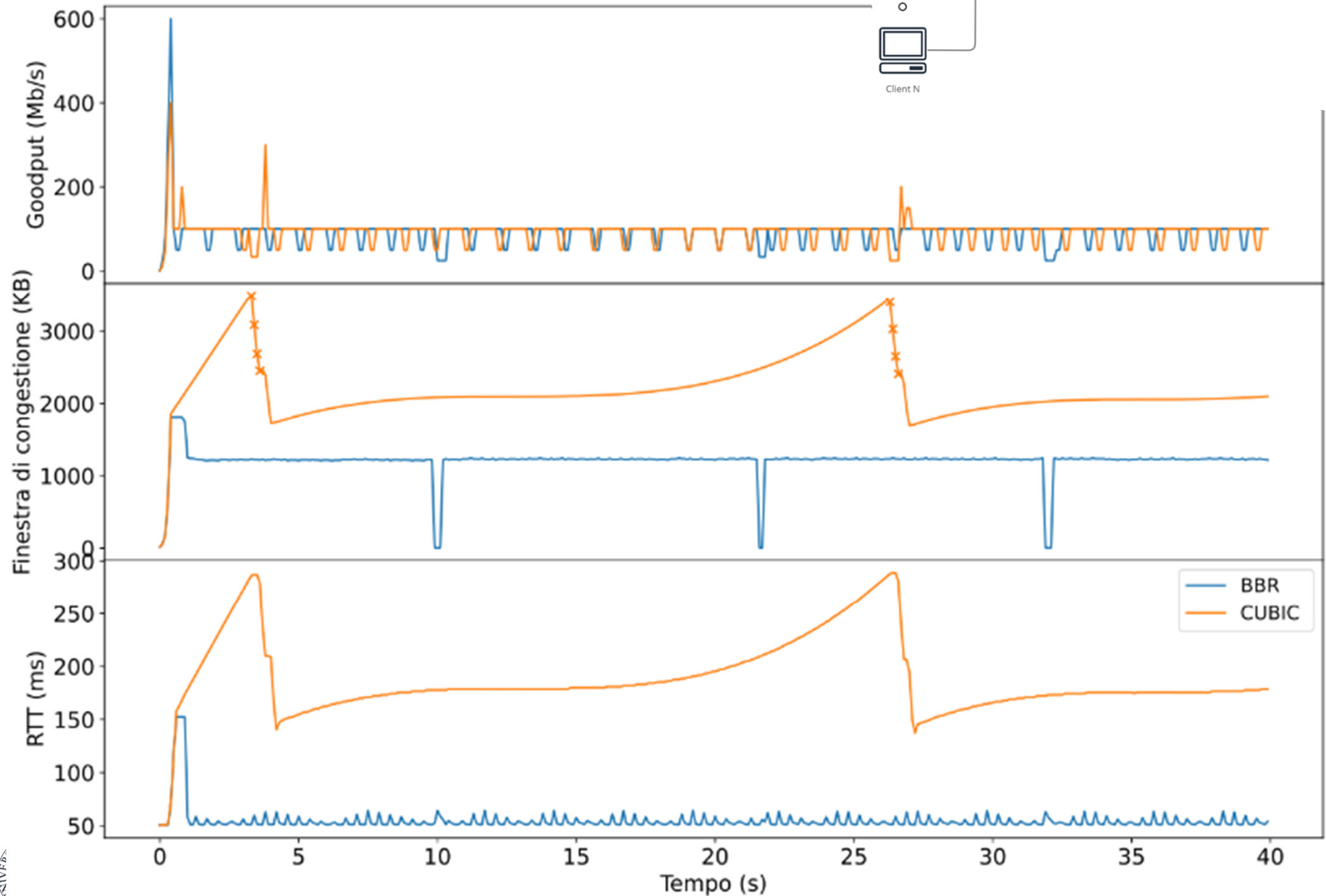
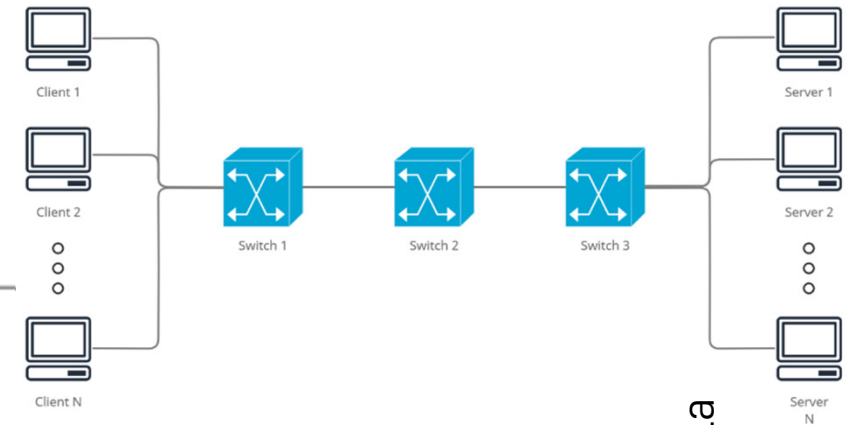
BBR models the network to send as fast as the available bandwidth and is 2700x faster than previous TCPs on a 10Gb, 100ms link with 1% loss. BBR powers google.com, youtube.com, and apps using Google Cloud Platform services.



Reno vs CUBIC vs BBR



BBR e CUBIC insieme



F. Vaccari, «Fairness di algoritmi per il controllo della congestione in protocolli TCP», Tesi UniTN, 2022.

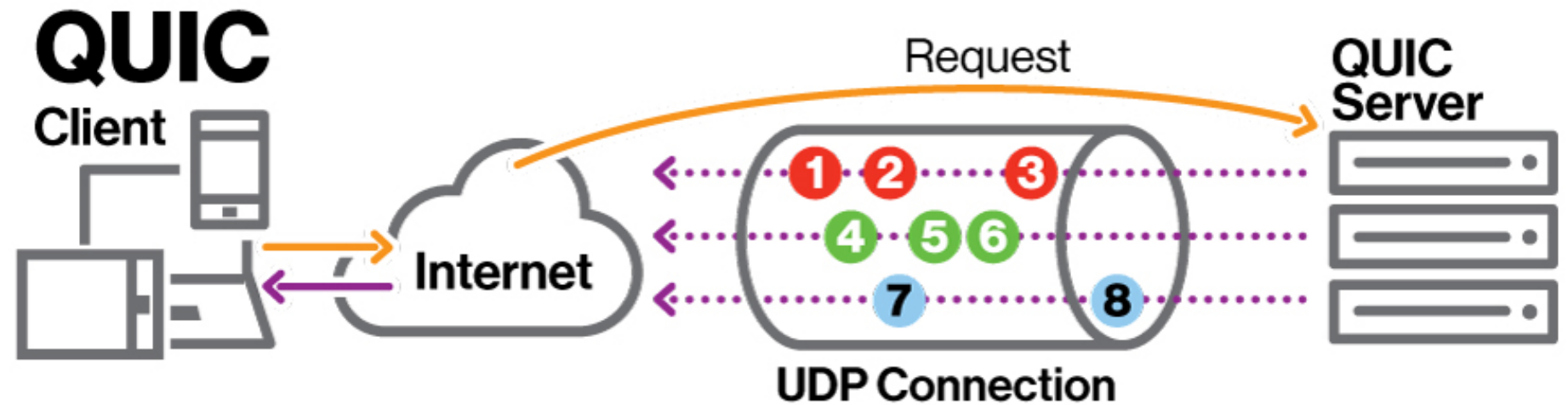
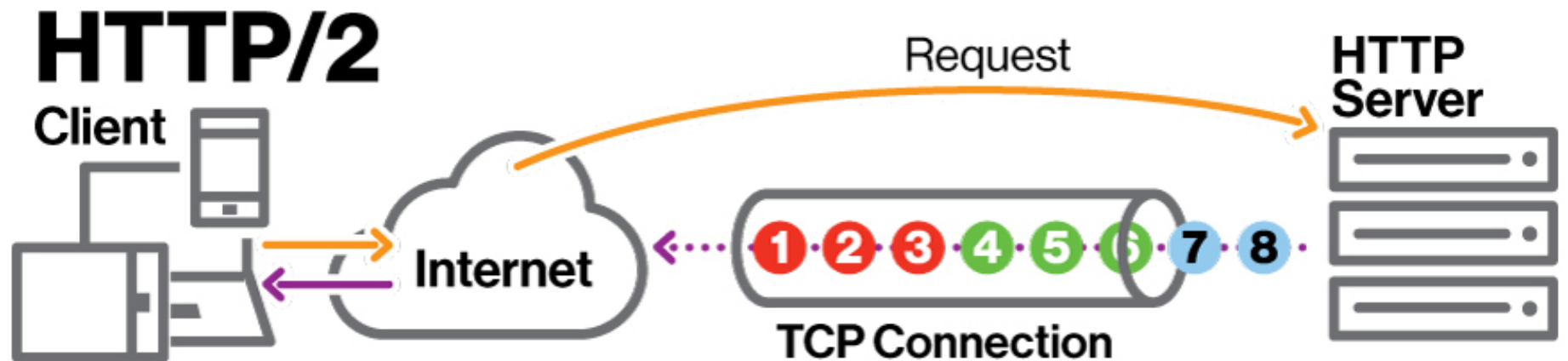
Per approfondimenti

- ❑ *BBR Congestion-Based Congestion Control*
(N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, **Van Jacobson**)
ACM Queue, 2016
<https://dl.acm.org/doi/pdf/10.1145/3012426.3022184>

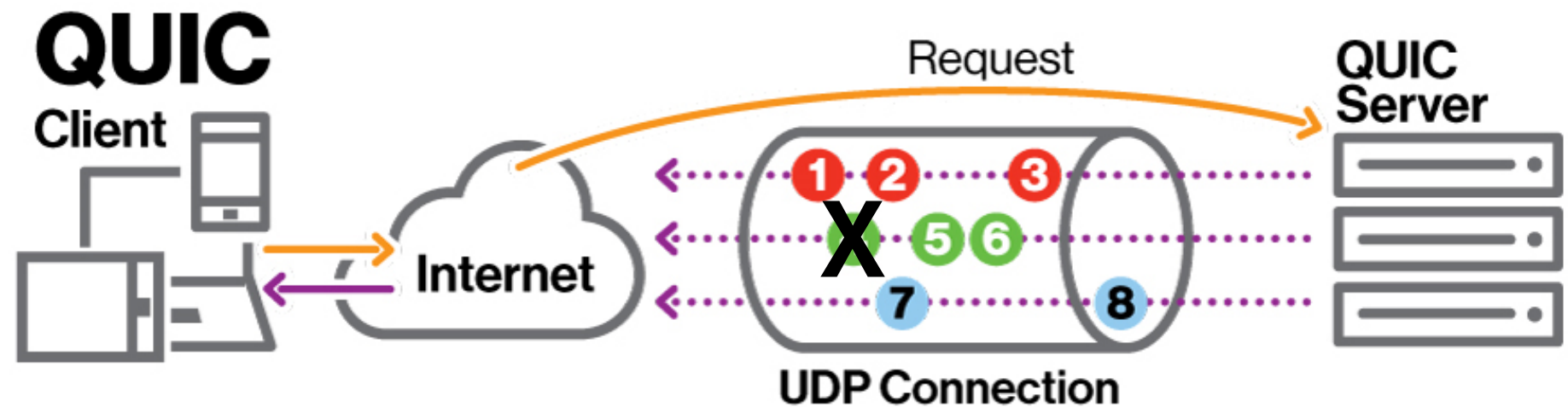
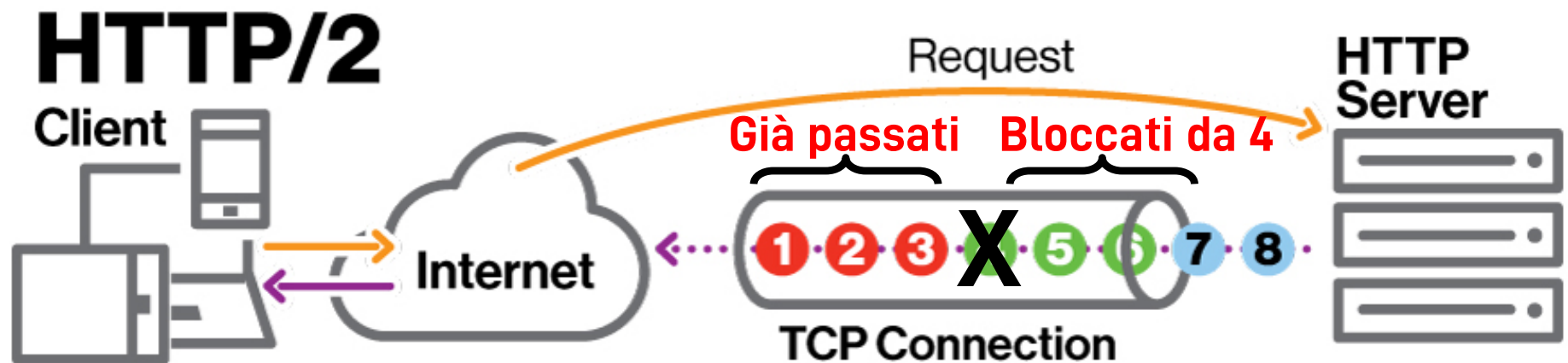
QUIC

- ❑ Inizialmente progettato da Google nel 2012
- ❑ Dapprima acronimo di Quick UDP Internet Connections, IETF lo considera semplicemente il nome del protocollo
- ❑ Due obiettivi
 - ❖ Evitare fenomeni di Head-of-Line blocking
 - Utilizzando di base UDP invece di TCP
 - ❖ Ridurre la latenza rispetto alle connessioni TCP
 - Compattando i messaggi per ridurre l'overhead di connessione
- ❑ Può essere implementato a livello applicativo anziché nel kernel
 - ❖ Chrome, Firefox, Safari, ...
- ❑ Protocollo di livello trasporto del futuro HTTP/3

HTTP/2 vs. QUIC



HTTP/2 (e HoL blocking) vs. QUIC

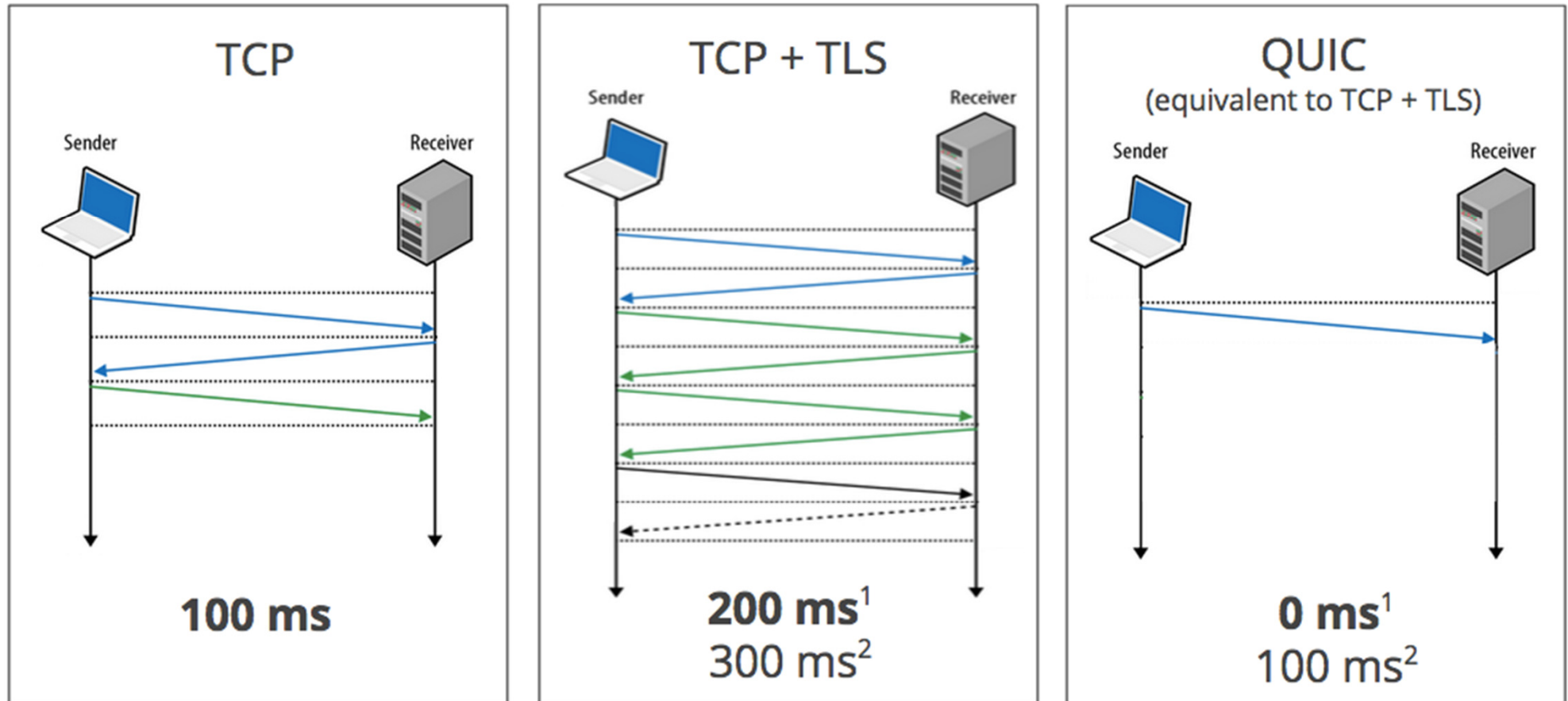


Ridurre l'overhead di connessione

- ❑ Molte connessioni HTTP richiederanno TLS per funzioni di sicurezza (autenticazione, crittografia, ...)
- ❑ QUIC incorpora nel processo di handshake iniziale lo scambio delle chiavi di configurazione e dei protocolli supportati
- ❑ Quando un client apre una connessione, il pacchetto di risposta include anche i dati per i pacchetti futuri necessari all'uso della crittografia in TLS
- ❑ Si elimina la necessità di impostare la connessione TCP e *poi* negoziare il protocollo di sicurezza tramite altri pacchetti
- ❑ Altri protocolli possono essere serviti nello stesso modo, combinando insieme più passi in una singola richiesta-risposta
 - ❖ Questi dati possono poi essere utilizzati sia per le richieste successive nella configurazione iniziale, sia per le richieste future

Dialogo «istantaneo»

Zero RTT Connection Establishment



1. Repeat connection
2. Never talked to server before

UDP o TCP?

- ❑ TCP è ampiamente adottato e spesso le infrastrutture internet sono «sintonizzate» per TCP e limitano o bloccano UDP
 - ❖ **D**: perché?
- ❑ Google ha condotto degli esperimenti esplorativi e ha scoperto che solo un piccolo numero di connessioni sono state bloccate in questo modo
- ❑ Perciò lo stack di rete di Chromium apre contemporaneamente sia una connessione QUIC sia una connessione TCP
 - ❖ Questo permette un fallback con una latenza trascurabile

Eventi di cambio rete

☐ **D**: quand'è che si cambia rete?

☐ Con TCP:

- ❖ Le socket sono identificate dalla quadrupla (IP sorgente, porta sorgente, IP destinazione, porta destinazione)

- ❖ Tutte le connessioni vanno in timeout e vengono ristabilite

☐ Invece, QUIC include un ID di connessione al server, che non dipende dalla fonte o dalla rete usata

☐ Si può così ristabilire la connessione inviando nuovamente un pacchetto contenente tale ID

- ❖ L'ID sarà ancora valido anche se l'indirizzo IP dell'utente cambia



Your connection was interrupted

A network change was detected.

ERR_NETWORK_CHANGED

Reload

Capitolo 3: Riassunto

- ❑ Principi alla base dei servizi del livello di trasporto:
 - Multiplexing, demultiplexing
 - Trasferimento dati affidabile
 - Controllo di flusso
 - Controllo di congestione
- ❑ Implementazione in Internet
 - UDP
 - TCP
- ❑ Protocolli di trasporto moderni
 - ❑ CUBIC
 - ❑ BBR
 - ❑ QUIC

Prossimamente:

- ❑ Lasciare la “periferia” della rete (livelli di applicazione e di trasporto)
- ❑ Entrare nel “cuore” della rete